AdRevora AI — Comprehensive Technical Overview

Current implementation and source handover — 12 October 2026

Latest edition: the repository now includes the reviewed frontend and executable revenue backend. This edition precedes the preserved earlier documentation baseline. Existing material and native structure are retained below.
Source commit: 1c52c1e14b33bf3d2f03d6e64a9e4408ea2a4a08. Owner-attributed normal push verified; repository visibility remains private and GitHub Actions remains disabled. The published static site and future backend deployment have separate release gates.
Current engineering evidence: 14/14 local core acceptance criteria; 89/89 JS/TS tests; 4/4 deployment tests; type, lint, build, dependency and security checks; 13 browser checks with zero application errors; independent source/documentation review approved.
Future production objectives: more than one million simultaneous dashboard users and a separate million-plus tracking workload, 99.9% monthly availability, measured latency/freshness, failure headroom and tested recovery. Architecture and release criteria follow; company-account activation and production acceptance remain open.

AdRevora AI

Source guide: README.md
Advertising → lead quality → appointments → customers → revenue → profit.
AdRevora AI is the product brand; AdRevora OS is the advertising intelligence workspace. The local connected core combines advertising networks, CRM journeys and verified financial data through one provider-independent system. The initial connected funnel is around Meta, GoHighLevel and coaching enrollments, followed by Google and other businesses.
Website: https://adrevora.zahidul-islam.com
Repository: https://github.com/Zahidulislam2222/AdRevora-AI
The repository includes the React frontend and executable Node revenue backend. Start with the local runtime, verified implementation and production-readiness roadmap. Current engineering evidence: 14/14 local criteria, 89 automated JS/TS tests, four deployment tests, real HTTP/browser flows and independent review. The production roadmap specifies 1M+ simultaneous dashboard users plus a separate tracking workload and a 99.9% availability objective with measurable release gates.

Start here

- Product owner or client; Starting point: Technical overview, ten-phase roadmap

- Frontend developer; Starting point: Frontend guide, design system
- Backend developer; Starting point: Backend contracts, integrations and data
- Security/privacy reviewer; Starting point: Security, US/EU compliance
- Platform engineer; Starting point: Capacity and reliability, operations, deployment
- Contributor; Starting point: Contribution gates, setup
- All readers; Starting point: Complete documentation index

Current delivery

The responsive frontend and fifteen routes are published over HTTPS. Product, solutions, integrations, security, resources, company, contact and legal pages share an accessible visual system with the workspace. The ambient film plays automatically; reduced-motion preferences and media failure use a static fallback. Smooth scrolling respects accessibility preferences.

- Website and workspace interface; Current state: Implemented and published; Next gate: Continued usability/accessibility review
- Reference workspace filters, drafts and CSV; Current state: Browser reference records; Next gate: Wider production workflows
- Local revenue engine at /engine/; Current state: Authenticated API, SQLite ledger, reports and persisted approvals; Next gate: Production identity, database and operations
- Contact brief; Current state: Validates and downloads locally; sends no message; Next gate: Approved delivery service/privacy controls
- Meta and HighLevel; Current state: Native read-only connectors; fixture-backed full-flow tests; Next gate: Company access, mappings and live verification
- Stripe; Current state: Signed payment/refund receiver; real test-key read-only check; Next gate: Own webhook secret and account-bound activation
- Attribution and recommendations; Current state: First/last eligible touch, unknowns, deterministic policy and real local AI; Next gate: Broader evaluations and production assessment
- Pixel/CAPI; Current state: Planned; Next gate: Tracking/privacy audit and approved implementation
- Google and additional businesses; Current state: Roadmap specified; Next gate: Stable pilot and expansion review
- Million-plus concurrency and 99.9% availability; Current state: Future workload/SLO acceptance targets; Next gate: Benchmarks and measured operation

The published website is the earlier static release. Source distribution includes the local core; production backend promotion is a separate release. Company Meta/HighLevel access is not available; connector tests use fixtures. Stripe readiness is test-mode read-only; no real payment/refund was created. The installed local model was actually exercised. Reference economics are not customer results. See the runtime guide and verification record for exact scope and remaining gates.

Run and verify

Use Node 24 LTS and npm. Use Python 3.11+ for deployment rendering/tests. Dependencies are pinned in the lockfile.

npm ci
npm run dev
To run the connected local engine, copy .env.example to ignored .env, set a bootstrap password of at least 16 characters, then run npm run build and npm run core. Use the printed /engine/ address. Each installation supplies its own private configuration and records access in its ignored recovery register. Provider setup is in the connector guide.
Use Vite's printed local URL. Before committing:
npm test
npm run typecheck
npm run lint
npm run format:check
python3 -m unittest discover -s tests -p 'test_*.py'
npm run build
npm run preview inspects the build locally. dist/ is the static deploy artifact; the repository root is never the web document root. Install/run security hooks as specified in CONTRIBUTING.md.

Repository map

- src/; Purpose: React components, routing, typed settings and maintained content
- public/; Purpose: Self-hosted artwork, film, fonts and licenses
- backend/; Purpose: Executable local API, database schema, ledger, connectors and bounded AI
- deploy/; Purpose: Implemented static deployment settings and renderer
- tests/; Purpose: Tracked automated regression tests
- docs/; Purpose: Architecture, roadmap, integration, governance, operations and evidence
- .github/workflows/; Purpose: Security workflow prepared for approved execution

Secrets, recovery records, project memory, personal scripts and artifacts are ignored. The frontend reads no application environment variables. Public settings belong to src/config.ts, maintained data to src/data/, deployment settings to deploy/settings.json. Backend variables enter backend/settings.ts and .env.example; maintained policy, prompts and mappings live in backend/data/.

Delivery and ownership

The repository remains private and owner-controlled. Public-shareable documentation does not change visibility or grant a source-code license. Preserve third-party notices; obtain commercial licensing/brand decisions before external distribution. The supplied business brief is a requirements reference, not evidence of an engagement.
Local security hooks are installed. Repository Actions remain disabled pending billing/execution approval; no remote CI success or automatic deployment is claimed. The static HTTPS artifact has publication evidence and current hero checks.
The ten phases coordinate UI, API, data, security, tests and documentation after frontend-first design. Legal approval, capacity proof and uptime measurements have separate release gates.

Production readiness and release roadmap

Source guide: docs/PRODUCTION-READINESS.md
Updated 2026-10-12. AdRevora AI connects advertising spend to customer journeys, collected cash and accountable recommendations. This document is the release map: current executable capabilities, the next account/production gates, and the future 1M+ simultaneous-user and 99.9% availability architecture.

Current engineering baseline

The repository contains a React frontend and Node HTTP/SQLite backend. The local engine implements password sessions, tenant roles, validated source snapshots, immutable money postings, successful refunds, consent-aware attribution, journey evidence, deterministic policy and persisted human decisions. Meta/GHL adapters are read-only. Stripe accepts signed account/mode/version-bound payment/refund events. Local AI selects allowlisted investigations; the server calculates financial prose.
The core met 14/14 local acceptance criteria: 89 JS/TS tests and four Python deployment tests pass; type/lint/build/security gates and real HTTP/browser flows pass; independent review approved after two reproduced defects were fixed. Thirteen browser checks cover login, finance, attribution, actual local AI, persisted decisions, journey evidence, connector configuration and four viewport widths. See the verification record for exact evidence and limitations.
The published static website is an earlier release. Backend source delivery does not promote a hosted service. Current connectors have contract/HTTP fixture evidence; real company account mappings, pagination and reconciliation need activation checks.

Release stages and acceptance

- Local core; Deliverables: Authenticated UI/API, ledger, adapters, policy and human decisions; Owner: Engineering; Required exit evidence: Local criteria, negative tests, real flows and reviewed source
- Account activation; Deliverables: Company-owned Meta/GHL access, versions/scopes/mappings, dedicated Stripe receiver; Owner: Integration and finance; Required exit evidence: Real bounded sync, durable event replay, account identity and cash/source reconciliation
- Production foundation; Deliverables: OIDC/MFA, TLS, PostgreSQL, vault, queue/outbox, tenant isolation and deployment; Owner: Platform and security; Required exit evidence: Threat review, authorization negatives, migrations, rotation, rollback and restore drills
- Controlled pilot; Deliverables: Lawful consent/data lifecycle, support/on-call, source freshness and operating dashboards; Owner: Product, privacy and operations; Required exit evidence: Legal/privacy sign-off, penetration test, pilot finance validation and measured service objectives
- Scale release; Deliverables: Regional cells, analytical read models, admission limits, failure headroom and distributed load; Owner: SRE and engineering; Required exit

evidence: Signed 1.1M dashboard plus 1.1M tracking workload, latency/error/cost/failure reports and independent acceptance

- Restricted automation; Deliverables: Approved provider writes, spend reservations, live authorization, audit and kill switch; Owner: Product, finance and security; Required exit evidence: Concurrent-budget, stale-approval, timeout/replay and reconciliation tests; explicit activation decision

Named individuals, backups and approval records belong in private governance records. The ten-phase plan owns dependencies, implementation artifacts and effort estimates; this matrix does not replace those phase exit gates.

Account activation and financial trust

Use the connector setup guide. Configure credentials outside source; URLs, versions, account IDs, limits and timeouts enter typed settings; CRM field mappings and financial/AI policies enter maintained data. Unknown consent, enrollment identity, currency or account binding must not become guessed matches.

Before a pilot, reconcile a bounded source period against provider exports and Stripe money. Record collection/refund timestamps, duplicate/reordered delivery, unknown revenue, missing fulfillment costs and source coverage separately. Local import control totals prove supplied-record arithmetic, not provider completeness. Obtain a historical completeness strategy before relying on dashboards for budgets.

Extend the financial model deliberately for fees, disputes/chargebacks, multicurrency, taxes and overhead. Net collected cash is not automatically total business profit. Keep one authoritative financial write path per tenant and version metric semantics. Schema changes and backfills need bounded, reversible migration/reconciliation plans.

Security, privacy and legal acceptance

Security policy owns controls and private reporting; compliance owns the primary-source applicability register. Before onboarding real business data, require:

- Controller/processor roles, approved jurisdictions/sectors, vendor contracts, lawful purposes, transparency, retention and cross-border transfer decisions.
- Consent receipts, withdrawal/opt-out propagation, verified rights requests, deletion after restore and financial/legal-hold exceptions.
- Enterprise authentication, MFA, explicit tenant authorization across API/jobs/cache/storage/exports, encrypted secrets and tested rotation/revocation.
- Verified webhook identities, bounded parsing, immutable/replay-safe money, restricted network egress and data-minimized logs/model requests.
- Versioned OWASP ASVS coverage, independent penetration testing, incident notification decisions and closure of launch-blocking findings.
- AI-role/classification review, applicable disclosures/media provenance, licensed models/assets and evaluation for unsupported claims and unauthorized actions.
- Payment-data scope review. Receive financial references and amounts, keep raw card data out of this analytics service, and assess any future collection flow separately.

GDPR/ePrivacy, applicable US state laws and the EU AI Act require decisions grounded in actual processing. Marketing, housing, insurance, communications, accessibility and minors each have separate expansion gates. A design checklist is engineering work; legal approval and certifications require their own evidence and accountable owners.

Future 1M+ simultaneous-user architecture

Use separate capacity planes: CDN/static assets, stateless authorized APIs, regional event collectors, durable queues, provider/account workers, transactional finance, incremental analytics, asynchronous exports and bounded AI jobs. Dashboard refreshes read materialized aggregates; they do not trigger provider sync or model inference.

The planning workload is 1.1M dashboard sessions at one request/10s and a separate 1.1M visitors at one event/30s. These assumptions imply 110,000 API requests/s and about 36,667 events/s before caching/batching. They are capacity inputs requiring measurement.

Partition by authorized tenant/region; bound noisy neighbors with per-tenant budgets and admission control. Pool/index transactional queries and move large analytical scans away from request paths. Queue backpressure, quotas and provider rate budgets must remain effective during AZ failure, traffic bursts and backlog recovery. Approved residency/transfer decisions determine regional placement and replication.

The local SQLite process remains the verification baseline. Production migration requires PostgreSQL constraints/RLS, reviewed migrations and data parity, an outbox/ durable queue, stateless sessions and analytical read models before distributed scale. Benchmark sustainable throughput per cell rather than guessing replica counts.

The capacity contract specifies arithmetic, payload/retention assumptions, p95/p99, load ramps, peak/soak duration, failure exercises and independent release evidence. Cloud/SaaS/load-test spending requires a separate approved budget; this roadmap provisions no resources.

Availability, recovery and operations

The production objective is 99.9% monthly availability for critical console/API journeys and separately for event acceptance. This exceeds a 99% floor. A 30-day time-based allowance is 43.2 minutes at 99.9%, compared with 432 minutes at 99%. Request-based budgets use eligible request counts and must not be substituted for elapsed downtime. Google SRE.

Define endpoint eligibility, regional coverage, maintenance and provider dependencies before contractual commitments. Measure external user journeys, latency/errors, queue lag, freshness, duplicate rates, unknown cash, DB saturation and restoration. Use multi-AZ durable services, readiness/draining, compatible migrations, canaries and tested rollback. Error-budget burn changes release priorities.

Recovery targets are RPO ≤5 minutes and RTO ≤60 minutes for agreed core data classes. Acceptance needs encrypted backup/PITR, isolated restore and finance reconciliation, privacy tombstone reapplication and an actual measured failure exercise. Assign a primary/backup maintainer, on-call rota, incident commander and security/privacy escalation. Provider outages preserve last trustworthy evidence with stale timestamps. See operations for the release and incident procedures.

Supply chain, economics and handover

Pin and review dependency updates; check registry provenance, licenses, advisories and asset notices. Produce dependency/SBOM and artifact manifests for production releases; record reviewed source, build identity, configuration version and rollback artifact. Separate commit attribution from cryptographic signature verification. Preserve required scans/hooks and require fresh-context review before release.
Forecast compute/HA headroom, DB/storage/I/O, queues, analytics, CDN/network, observability, identity, backups, AI, CI and operating staff using measured units. Keep tenant/event/report unit costs, quota/headroom decisions and spend caps visible to accountable owners. Rate limits and billing alerts alone do not guarantee a cap.
Company accounts retain source, domains, hosting, provider assets and recovery. Stage public files explicitly; verify secret/history exclusions and remote commit equality. Public documentation remains independently shareable while repository visibility and code licensing stay owner-controlled. Existing Google Docs keep their IDs and prior content; a validated new PDF replaces the canonical local mirror.

AdRevora AI — technical overview

Source guide: docs/TECHNICAL-OVERVIEW.md
Updated 2026-10-12. Product brand: AdRevora AI. Platform: AdRevora OS.
Latest milestone: frontend and executable backend source handover, with 14/14 local core criteria, 89 JS/TS tests, four deployment tests and independent review. See production readiness for account activation, legal/security acceptance and the future 1M+ concurrency/99.9% availability release path.

Purpose and initial business outcome

AdRevora combines campaign economics with lead quality, CRM stages, appointments, verified sales, collected revenue and known costs. It sits above supported advertising APIs and preserves a provider-independent financial and attribution model.
The first connected journey is Meta ad → webinar registration → lead → GHL contact → Platinum appointment → completed consultation → enrollment → verified revenue → AI recommendation. The supplied offer price is $5,000; payments, discounts, refunds and chargebacks determine actual money. Primary KPI: cost per Platinum enrollment. Secondary KPI: revenue per lead.
Later funnels cover real estate listings/closed transactions, property acquisition and profit, paid insurance policies and active producers. Definitions, regulated scope and provider permissions are reviewed separately for every expansion.

Implemented scope

The published frontend has fifteen routes: home, Platform, Solutions, Integrations, Security, Resources, About, Contact, Privacy, Terms, three resource articles, AdRevora OS and a not-found entry. React/TypeScript/Vite emit per-route HTML; Motion and Lenis support

transitions/scrolling with reactive reduced-motion behavior. Fonts, artwork and the automatic muted film are self-hosted.
Workspace filtering, campaign search, evidence, approvals, creative drafts, activity and CSV operate on reference records in browser memory. Integrations are disconnected. Contact preparation validates and downloads a local brief; no message is transmitted. The published artifact remains static. A separate local /engine/ now provides authenticated tenant workspaces, SQLite source ledger, payment/refund reconciliation, consent-aware first/last attribution, persisted approvals and actual free local AI. Native read-only Meta/HighLevel connectors are tested with fixtures. A real Stripe test key passed a read-only account check; the signed receiver awaits its own secret. Company provider access is not available, so live sync is unverified. See runtime, connector setup and current evidence.
A versioned static container behind HTTPS has verified isolation, headers, route behavior, restoration and file parity. Historical release gates include fifteen JS and four Python tests, fifteen live routes and seven hero viewport widths. See publication, hero verification and current documentation verification.

Users and authority

- Marketing Director; Primary needs: Winners/losers, creative fatigue, tests, funnel economics; Decision boundary: Approved advertising changes within company policy
- COO; Primary needs: Uncontacted leads, follow-up, appointments, CRM and technical bottlenecks; Decision boundary: Workflow/routing and operational changes
- CEO; Primary needs: Spend, sales, revenue, profit, allocation and major decisions; Decision boundary: Major budget, offers, pricing, markets and strategy
- Analyst; Primary needs: Scoped exploration, evidence and reporting; Decision boundary: No financial/provider execution
- Administrator; Primary needs: Membership, security and connection administration; Decision boundary: Human-controlled account management

Executive reporting does not imply permission to change provider ownership or account security. The local API enforces tenant membership and admin/analyst roles; enterprise business-unit roles remain a production requirement.

Architecture and stack

The local core reuses React and uses dependency-free Node HTTP/SQLite. It is a single-process verification architecture, not the million-user service below. Proposed production backend: modular TypeScript/NestJS API plus independent workers, PostgreSQL for transactions/ledger/identity/policy, a durable queue and transactional outbox, Redis-compatible cache, encrypted object storage, scoped analytical read models and an OIDC identity service. Exact versions/providers/regions are approved in phase 2. Next.js is an optional architecture decision, not a required rewrite. Add analytical storage or services when measured workloads justify them.

- Dashboard users → CDN and WAF
- CDN and WAF → React frontend
- React frontend → Tenant-authorized API

- Approved websites → Consent and event capture
- Consent and event capture → Regional ingestion
- Meta / GHL / later Google ↔ Provider adapters
- Provider adapters → Regional ingestion
- Regional ingestion → Durable queue and outbox
- Durable queue and outbox → Identity / funnel / finance workers
- Identity / funnel / finance workers → PostgreSQL source of truth
- PostgreSQL source of truth → Scoped analytical read models
- Tenant-authorized API → PostgreSQL source of truth
- Tenant-authorized API → Scoped analytical read models
- Scoped analytical read models → Evidence-bound AI workflows
- Evidence-bound AI workflows → Human approval and policy engine
- Human approval and policy engine → Restricted executor in later phases
- Restricted executor in later phases → Provider adapters
- Durable queue and outbox → Consent-checked conversion delivery
- Consent-checked conversion delivery → Meta / GHL / later Google

Interactive APIs, ingestion, provider sync, analytical reporting and AI have independent scaling/rate budgets. Dashboard refreshes read local aggregates rather than causing provider/LLM calls. Current static hosting and future regional services have separate capacity envelopes.

Tracking, identity and revenue

Preserve external IDs, historical names, UTMs and permitted click identifiers at capture. Keep contact links, appointment transitions, enrollment evidence, payments and refunds distinct. A won opportunity does not establish collected revenue. Metrics expose currency, cohort/date basis, model/window, data completeness and freshness.

Browser/server copies share a durable conversion ID for the same occurrence. Retries reuse that identity; separate conversions get new IDs. Receipt deduplication, domain-event idempotency and financial-posting idempotency protect different layers. Source counts and provider attribution counts remain separate.

First-touch and last eligible paid-touch views begin the internal attribution model. Unknown/partial matches remain visible. Multi-touch credit never exceeds one sale; network claims are not summed as separate revenue. Incrementality needs experiments. See integrations and data.

AI, security and compliance

The Director coordinates strategy, creative, campaign drafts, analysis, budget recommendations, retargeting and attribution. These are logical workflows, not a requirement for eight paid model sessions. Deterministic tools calculate money; AI explains evidence and drafts proposals. Recommendations use SCALE, MAINTAIN, TEST, FIX or PAUSE with freshness, limitations and expiration.

Publication, pausing, budgets and creative changes require human approval in the initial platform. Later bounded execution uses exact payload approval, live permission checks, budget

reservations, idempotency, reconciliation and a kill switch. Ownership, payment methods and security stay human-controlled. AI/governance.
Required security includes tenant isolation, least-privilege OAuth, MFA, secret storage, verified ingestion, safe exports, audit and independent assessment. Secrets and raw lead details stay out of prompts/logs. US/EU readiness uses an applicability register, privacy/consent/rights/retention/transfer controls and sector-specific review before relevant processing. Security and compliance define gates.

Scale and availability

The target covers more than one million simultaneous dashboard users AND more than one million simultaneous tracked visitors, independently measured. Request/event rates, regions, payloads, tenant skew, retention and peak duration define capacity. Incremental load/failure tests establish the accepted envelope.
The target is 99.9% monthly availability for defined critical console/API and event acceptance journeys. Routine releases target no user-visible interruption with compatible migrations, canaries, draining and rollback. Recovery, latency, freshness, errors and cost have measurable contracts in capacity/reliability.

Ten-phase delivery and architecture approval

- Frontend design and implementation, with explicit design acceptance.
- Architecture approval, authentication, tenancy and secure foundation.
- Meta/GHL read integrations and reconciled reporting.
- Tracking audit, consent-aware Pixel/CAPI and diagnostics.
- CRM journey, revenue ledger and attribution.
- AI analysis, creative/draft workflows, approvals and reporting.
- Initial connected production pilot, security and operational acceptance.
- Google/GA4, additional businesses and SaaS expansion.
- Million-plus capacity proof, resilience and controlled execution.
- Final acceptance, company handover and ongoing operations.

The initial connected MVP spans phases 1–7. Every later feature delivers frontend, backend, data, tests, security and documentation together. The build plan owns detailed dependencies and exit gates; frontend publication does not close other gates.
Approve twenty architecture items before integration development: diagrams, stack, Meta, Pixel, CAPI, GHL, schema, attribution, event map, AI, security, permissions, screens, scope, timeline, development cost, monthly cost, testing, deployment and docs. Draft coverage exists; approved migrations and live provider permission evidence follow in their implementation phases.

Estimates, ownership and handover

Baseline planning: 800–1,300 engineering hours for phases 1–7; 1,880–3,580 for all ten. At the assumed $60–$120 blended rate, arithmetic is $48,000–$156,000 and $112,800–$429,600 before contingency/external services. Suitable-team calendar allowances are 12–20 weeks for

the initial connected MVP and 9–18+ months overall, subject to discovery, reviews and measured scope. These are planning assumptions.
Pilot allowances and the large-scale cost model are in the roadmap. Million-user pricing requires measured demand and current vendor calculations; no purchase or paid execution follows from this document.
Company-controlled source/accounts/assets, API/schema/configuration guides, release manifests, recovery drills, dashboards, user/operator instructions and named maintenance owners form the handover. Start with contributing and operations. Public material excludes recovery and private incident records. The supplied brief is a requirements reference; distribution/brand rights need owner review.

Backend runtime and production architecture contract

Source guide: backend/README.md
Local implementation exists. Node 24 built-in HTTP/SQLite serves the connected React engine, password login, tenant-scoped ledger, reports, journeys, recommendations, decisions and source connectors. Run it, configure accounts, and check verification/limits. No new runtime dependency was added.
The local scope passed 14/14 acceptance criteria and independent review. Signed Stripe events, Meta/GHL normalized snapshots and the browser use the same ledger; financial facts remain deterministic when the model is unavailable. Source includes the backend type-check configuration, schema, maintained policy and regression tests. Company connection activation and production release are tracked separately in production readiness.

Implemented local modules

- settings.ts, data/; Responsibility: Typed environment boundary; validated financial/AI/mapping data
- schema.sql, store.ts; Responsibility: Additive local schema, WAL/FULL transactions, scrypt/hashed sessions, membership
- validation.ts, contracts.ts; Responsibility: Exact bounded imports, UTC timestamps, integer money and control totals
- ledger.ts; Responsibility: Replay-safe imports, source revisions, consent-aware attribution, reports/decisions
- connectors.ts; Responsibility: Read-only Meta daily spend and HighLevel explicit customer timelines
- stripe.ts; Responsibility: Raw-body signed successful-payment/refund ingestion; read-only readiness
- ai.ts; Responsibility: Aggregate-only local or verified-free inference; validated investigation codes
- server.ts, protocol.ts; Responsibility: Local HTTP, Host/Origin/auth checks, static build and protocol syntax

Implemented HTTP contract

All paths below are relative to /api/core. Requests/replies are JSON. Mutating browser requests require the exact configured Origin and an authenticated session except login; the Stripe receiver uses raw signature verification instead.

- GET /health; Behavior: Safe local liveness and schema version
- POST /login; Behavior: {username,password} → opaque HttpOnly cookie
- GET /session, POST /logout; Behavior: Authorized workspaces/settings; revoke cookie/session
- POST /workspaces/:id/import; Behavior: Validated normalized batch, transaction and replay check; admin
- POST /workspaces/:id/example; Behavior: Load maintained reference records; example workspace only
- GET /workspaces/:id/report; Behavior: start,end,model query; UTC inclusive start/exclusive end
- GET /workspaces/:id/journeys; Behavior: Optional opaque after contact-ID cursor; bounded lifetime evidence
- POST /workspaces/:id/recommend; Behavior: {campaignId,query,includeAi} → stored evidence-bound proposal
- POST /workspaces/:id/decide; Behavior: {recommendationId,choice} → one persistent admin decision; no execution
- GET /workspaces/:id/history; Behavior: Decision validity and latest audit events
- GET /workspaces/:id/connectors; Behavior: Admin-only configuration/last-success diagnostics; no secrets
- POST /workspaces/:id/sync; Behavior: Admin-only {provider,options}; configured imported workspace only
- POST /stripe-webhook; Behavior: Configured tenant/account/mode/API-version-bound raw signed events

Errors have a safe {error} message and appropriate 400/401/403/409/413/415/429/503 status; unexpected errors return a generic 500. Provider rejection returns 502. No OAuth callback, user/tenant administration UI, provider write, scheduler or public deployment is claimed. The engine is loopback-only and single-process.

Production proposal below

The remaining sections describe production requirements across phases 2–7. They are not claims about the local runtime.

Boundaries

Proposed starting point: modular TypeScript/NestJS API and independent workers. Approve versions, licenses and hosting in phase 2; extract services when measured isolation or scaling needs justify the operating cost.

- Identity/tenancy; Responsibility: OIDC, memberships, scoped roles, revocation; Constraint: Never trust browser tenant scope

- Connections; Responsibility: Approved OAuth, encrypted token references, refresh and sync state; Constraint: No provider tokens in browser
- Ingestion; Responsibility: Verify/validate, durably receive and deduplicate; Constraint: Bind tenant/account to verified installation
- Conversion delivery; Responsibility: Consent-aware outbox, CAPI mapping, retries and diagnostics; Constraint: Recheck permission before sending
- Funnel/identity; Responsibility: Contact links, stage history, appointments and enrollments; Constraint: No name-only identity merge
- Finance; Responsibility: Payments, refunds, costs, FX and reconciliation; Constraint: Opportunity value is not cash
- Attribution; Responsibility: Versioned touch eligibility, credit and unknown reasons; Constraint: Never invent identifiers
- Reporting; Responsibility: Scoped aggregates, freshness and asynchronous exports; Constraint: No full-history scan per page
- AI/approval; Responsibility: Evidence, bounded output and exact proposal approval; Constraint: Policy is enforced outside prompts
- Execution; Responsibility: Later approved provider writes; Constraint: No asset/payment/security administration
- Operations/privacy; Responsibility: Audit, retention, deletion/replay and jobs; Constraint: Preserve lawful financial retention

Draft API contracts

These resource families are design contracts, not current website endpoints. Version OpenAPI beside implementation. Tenant selectors supplement membership checks.

- Session; Contract: Read user/scopes/tenants, revoke; Protection: OIDC validation, secure cookies, CSRF
- Connection; Contract: OAuth start/callback, health, revoke; Protection: Role, state, exact callback and scope checks
- Reports; Contract: Summary/campaign/ad/creative economics; Protection: Unit scope, bounded dates/pagination
- Journey; Contract: Lead→appointment→enrollment→money; Protection: PII permissions and access audit
- Tracking; Contract: Health/rejection/dedup/consent diagnostics; Protection: Sanitized payloads, restricted access
- Recommendation; Contract: Five-state result and evidence snapshot; Protection: Version, freshness and uncertainty
- Draft; Contract: Internal campaign/creative draft; Protection: Validated schema; no implicit publication
- Approval; Contract: Approve/reject exact proposal; Protection: Payload hash, policy, expiry and replay protection
- Export; Contract: Async scoped report and expiring download; Protection: Rechecked authorization, safe CSV
- Webhook; Contract: Provider-specific installation receiver; Protection: Current signatures, bounded body, durable receipt

- Privacy; Contract: Verified rights request and progress; Protection: Identity verification and legal-hold review
- Health; Contract: Separate liveness/readiness; Protection: No sensitive public diagnostics

Use opaque IDs, UTC occurrence/receipt times, reporting timezone, schema versions, minor currency units/currency codes and correlation IDs. Internal retryable mutations require tenant/action-scoped idempotency keys; reject changed payload reuse. External webhooks use verified provider identities. Safe field errors and appropriate HTTP statuses must not leak cross-tenant existence. Long reports return a job receipt.

Authorization and database

Derive tenant scope from session membership, enforce it in services, RLS, constraints, cache/storage keys, jobs, exports and AI retrieval. Runtime database roles cannot bypass policies. Pool connections use transaction-local tenant context, with tests for context leakage. Include tenant scope in appropriate foreign/unique keys. Business units inside group reporting still require explicit permission.

Separate admin, marketing, operations, finance and privacy/support capabilities. Support grants are justified, expiring and audited. Automated workers get minimal roles. PostgreSQL RLS. The data guide specifies logical entities. Phase 2 adds reviewed migrations, indexes, ER diagram and reproducible local startup. Unique constraints protect provider identity, event occurrence, money posting and execution. Expand/migrate/contract changes preserve old/new API/worker compatibility. Backfills are bounded; assess locks and replication lag before large indexes or data rewrites.

Durability and external side effects

Transactions commit domain changes and outbox together. Queue delivery can repeat; handlers plus unique constraints produce one logical effect. An external timeout can hide success: reconcile ambiguous outcomes before a non-idempotent retry. Record attempts, receipt/occurrence times, consent, schema/policy and correlation identity.

Use bounded backoff/jitter, provider/account rate budgets and dead-letter review. Replay rechecks current privacy/permission. Revocation stops jobs and marks data stale. Drain workers safely during releases; separate interactive and batch capacity.

Configuration boundary

The local implementation already owns runtime variables in settings.ts. Extend that typed, validated boundary for production categories: service origins/ports, OIDC, DB, queues/cache/storage, encryption references, provider URLs/versions/scopes, retries/timeouts, retention, models/AI budget, action ceilings and telemetry. Constructors receive settings instead of inventing defaults.

Every implemented variable enters .env.example with safe values/ownership. Missing required production secrets fail startup; actual values live in the approved secret store. Version

maintained event/attribution/approval policies as data. Do not add unimplemented backend variables to the current frontend environment file.

Implementation and acceptance

Build real login/tenant isolation first, then reconciled Meta/GHL reporting, consented tracking, persisted revenue journey, evidence-bound AI/approvals and the controlled pilot. Test cross-tenant negatives, signature/replay, refresh races, duplicate payments, outbox crashes, consent withdrawal, deletion after restore, money/FX/cohort arithmetic, stale approvals, concurrent budget reservations and degraded providers.
Run bounded contract tests against approved provider environments. Never load-test Meta/GHL/Google. Contribution gates, security and operations govern release.

Frontend developer guide

Source guide: src/README.md
React 19, TypeScript 6, Vite 8, Motion 14 and Lenis 1 power the current frontend. Exact versions are in the manifest/lockfile. Use Node 24 LTS and npm ci at the root.
Source includes both the public website and authenticated revenue engine. The published static release and local backend have separate delivery paths; start the engine through the built backend server for authenticated API requests. Current evidence and production readiness describe acceptance.

Code map

- App.tsx; Responsibility: Home, routes, transitions and scrolling lifecycle
- components/SiteShell.tsx; Responsibility: Shared accessible header/navigation/footer
- components/Pages.tsx; Responsibility: Supporting pages, articles and contact brief
- components/Dashboard.tsx; Responsibility: Workspace views, filters, dialogs and local decisions
- components/CoreWorkspace.tsx, lib/core-api.ts; Responsibility: Authenticated /engine/ API, import/sync, evidence, AI and saved decisions
- data/core.ts, core.css; Responsibility: Revenue-engine content and responsive styles
- components/Primitives.tsx; Responsibility: Reusable interface primitives
- data/site.ts; Responsibility: Typed page registry, metadata, navigation and copy
- data/content.ts, data/interface.ts, data/templates.ts; Responsibility: Maintained content, labels and creative templates
- data/workspace.ts; Responsibility: Reference business records
- config.ts; Responsibility: Assets, motion, input, filenames and display settings
- lib/routing.ts; Responsibility: History routes and offline hash compatibility
- lib/contact.ts; Responsibility: Normalized validation and local brief preparation
- lib/metrics.ts; Responsibility: Economics and safe CSV export
- lib/motion-preference.ts; Responsibility: Reactive reduced-motion subscription
- ../vite.config.ts; Responsibility: Escaped route metadata and per-route HTML emission

Development

npm ci
npm run dev
npm test
npm run typecheck
npm run lint
npm run format:check
npm run build
npm run preview
Use the printed URL. Ten top-level public pages, three articles, workspace and not-found entry must support direct paths and back/forward. Former console links remain compatible. Unknown hosted URLs return a real HTTP 404.
Keep focus visible, controls named, dialog focus contained/restored and tabs keyboard operable. Verify 320px through wide desktop layouts and internal table scrolling. Target WCAG 2.2 AA; full accessibility assessment remains a separate gate.
The muted inline looping film autoplays without controls. Reduced motion at load and runtime disables decorative motion and smooth scrolling; media failure keeps the poster. No paid call runs on page view or video playback.
Contact preparation creates a local download and sends nothing. Preserve values when editing. The reference /workspace/ keeps approvals in browser memory and clears them on refresh. The /engine/ persists server decisions across reloads, binding approvals to evidence, policy and expiry. CSV output protects against spreadsheet formulas.

Backend connection points

The /engine/ page uses typed same-origin /api/core requests and persistent server decisions. Run a built frontend through npm run core; the Vite/static reference workspace remains separate. No provider key enters the browser. Keep provider requests out of components. Preserve disconnected/loading/empty/stale/error, permission-denied and insufficient-evidence states. Read currency, timezone, model, cohort and freshness from the API. Server authorization applies to every request; hiding UI is insufficient.
Backend contracts describe session, tenant, reports, journeys, recommendations and approvals. Provider secrets never enter bundles. Maintained copy belongs in data/; public behavior settings in config.ts. Any future public env variable must be safe, validated and listed in .env.example; none is required now.

Release

Deploy only dist/ and reviewed hosting configuration. Follow the static runbook, contribution gates and current hero evidence. Preserve local manual tooling; store private screenshots/manifests in ignored folders and publish scrubbed results.

Deployment tooling

Source guide: deploy/README.md
This directory implements the static website configuration renderer. The executable backend is in backend/ and runs locally through npm run core after a frontend build. This renderer does not provision or publish that backend. Hosted backend, database, queue and regional services follow the production-readiness gates.

Render locally

From the repository root with Python 3.11+:
python3 deploy/render.py deploy/settings.json artifacts/hosting
python3 -m unittest discover -s tests -p 'test_*.py'
The ignored output contains Compose, Nginx and Caddy configuration. Rendering starts no container, changes no DNS and makes no paid request. Unsafe hostnames and unpinned images are rejected. settings.json owns hostname, ports, image/release identity, resources, health, cache, headers and timeouts. Stable protocol syntax belongs to the renderer. Secrets never belong in the settings file.
Follow the static runbook: detect drift, build/test locally, package static output, validate/upload/promote, exercise HTTPS/browser flows and prove file parity. Retain the previous verified release and referenced hashed assets. Never use the preview server for production hosting.
Private recovery records hold actual server access/locations. Future API, queue and database operations follow operations and capacity/reliability.

Source and production release

A GitHub source push does not promote a running release. Core promotion requires approved identity/TLS/secrets/storage/backups, provider activation and real operational checks. Do not expose the current loopback-only HTTP server as a public service. The 1M+ concurrency and 99.9% uptime targets apply to the future measured architecture; the static container renderer makes no backend capacity commitment.

Connected local revenue engine

Source guide: docs/CORE-RUNTIME-GUIDE.md
The engine follows advertising spend → consented customer journey → enrollment → collected payment/refund → attribution → recommendation → recorded human decision. It is a single-process local application. The published website is an earlier static release; production service acceptance remains separate.

Start

Use Node 24 LTS with built-in SQLite support. Node 24.20.0 on Windows and 24.21.0 on Linux were exercised. Install the pinned dependencies with npm ci.

- Copy .env.example to ignored .env; verify Git ignores it first.
- Set CORE_BOOTSTRAP_USER and a unique password of at least 16 characters.

Record actual access privately in ignored CREDENTIALS.md, never in source.

- Run npm run build, then npm run core.
- Open the printed /engine/ address and sign in. Load the example journey to exercise reports, evidence, recommendations and persisted approval history.
- Select the imported workspace for actual normalized records or native connectors.

The current owner machine already has private configuration. Its recovery register contains the generated local account. Do not rerun bootstrap scripts to rotate a password accidentally. Environment bootstrap values only create the initial account; changing them does not change an existing password.

npm run dev is the static/reference interface and does not start the API. Provider secrets are read only by the backend. The core binds to configured loopback, checks exact Host/Origin, and must not be exposed publicly by changing a proxy casually.

Financial and attribution rules

- Integer minor units, one policy currency and explicit minor-unit scale; no implicit FX.
- Collected payments minus successful refunds equals net cash revenue. Enrollment or opportunity values are never treated as received cash.
- Recorded contribution subtracts ad spend and recorded fulfillment costs. Missing overhead, processor fees or taxes are excluded and must be considered separately.
- First/last eligible paid touch uses configured lookback, timestamps and consent. Unknown revenue is preserved; attributed plus unknown must equal ledger revenue.
- Reports use UTC inclusive start and exclusive end. They describe cash flow; funnel counts are not mature-cohort conversion rates. Campaign leads count distinct contacts created in the window with a paid touch in that window, consented at the touch and still consented now. Extending the end alone does not age out leads. A contact may count toward multiple campaigns.
- Meta daily spend is posted at the start of the advertising account's day. A UTC report can cut across account days; it cannot recover intraday spend from daily data.
- Last accepted import/snapshot measures local freshness. It does not prove complete provider coverage or that a quiet Stripe account has no missing events.

The maintained example has $10,000 collected, $500 refunded, $9,500 net, $1,000 spend and $2,000 recorded costs. $7,000 is attributed and $2,500 unknown, with $0 difference. These are reference records, not customer outcomes.

Imports and source sync

The import shape is {id,schemaVersion,source,controls,records}. Record families and fields are defined in backend/contracts.ts and validation.ts; the example JSON is a complete schema illustration. Imported batches use source: "manual" even when prepared from a source export. Each financial-family control must equal its batch sum. Every cross-reference must exist already or in the same transaction. Replays have one effect; changed identity/payload returns conflict with no partial writes.

Native Meta/HighLevel sync is admin-only, read-only and manually triggered. It binds to one configured imported workspace and one account per provider. A full response is validated before a transaction updates materialized records. Provider revisions are journaled; unrelated/manual records are protected. Failed sync leaves the last good ledger intact. CRM deletions that would orphan a payment fail for reconciliation. Configure sources using the activation guide.

AI and decisions

Enable the existing local Ollama server with configured model ID, or leave AI off. Run Ollama and the core on the same operating system/loopback network; the verified owner setup uses Windows Node and Windows Ollama. The application does not download models automatically or start paid inference.

The model receives aggregate metrics, selects allowlisted investigation codes and cannot alter policy actions, arithmetic, approval authority or provider state. Financial prose is rendered from server values. Invalid output/outage is explicit and the policy result remains available. A real model response initially invented financial prose; the restricted contract and regression tests address that defect.

An optional free-only OpenRouter adapter checks current model pricing every call, requires zero routing ceilings/no fallback and verifies zero reported cost. Existing account privacy blocked its real free request; settings were preserved. No paid route is used. Never infer paid authorization from the existence of an API key.

Approval/rejection persists with evidence, query/model, policy version and expiry. Changed/stale/expired evidence invalidates approval. Decisions execute no advertising change. Local roles are admin/analyst; enterprise identity/permissions remain future work.

Power-loss recovery

Durable data lives in ignored .local/; SQLite uses WAL and FULL synchronization. Initial account/workspaces and domain writes use transactions. Restart with the same settings/database; imports and decisions survive process restart. SQLite durability does not replace off-device backups or a measured disaster-recovery program.

Stop only this core process before making a cold backup. Copy the complete database directory, including any WAL/SHM files, as one consistent stopped snapshot. Never copy just the main file while a writer is active. Protect backups as private data. Do not run Windows and Linux writers against the same database concurrently.

On resume, read memory/MEMORY.md, memory/core-implementation.md and the private dossier, inspect Git status, verify ignores and actual running process, then resume the first pending gate. Never assume a pre-outage test or deployment succeeded. Manual scripts and evidence are kept in ignored manual-tests/ and artifacts/.

Limits before production

This core has no OIDC/MFA, public TLS service, encrypted vault, background workers, automated retention/rights workflow, independent penetration test, multi-region capacity or

contractual availability evidence. Stripe disputes/chargebacks, FX, fees, exhaustive historical provider backfill and automated daily delivery are not implemented. No ad executor exists. Company access/mapping/live verification and production review remain required; the code must not be presented as the finished ten-phase platform.

Meta, HighLevel and Stripe activation

Source guide: docs/CONNECTOR-SETUP.md
The connector code is implemented and tested locally. Company Meta/HighLevel access has not been supplied, so live compatibility is not claimed. Account setup, app permissions, field definitions and a real reconciliation run cannot be completed before that access exists. These steps configure the existing adapters; unsupported API/account behavior could still require a fix after live verification.
All secrets belong in ignored .env and the private recovery register. Use company accounts with read-only grants. No account purchase, advertising action or paid inference is authorized by these instructions.

Shared setup

Set CORE_CONNECTOR_TENANT to the imported workspace. Page size/count, total timeout, maximum report days and record/tenant ceilings are in the typed settings and env example. The local UI exposes configured/not-configured status and last accepted snapshot. Admin role is required for both status and sync. Example workspaces reject connector writes. Account binding prevents silently switching a populated workspace to another company account.
Sync is manual. Authentication errors, provider rate limits, non-advancing pages, changed source counts, unsafe amounts, wrong account/currency, missing relationships or ceilings fail without partially replacing the last successful source snapshot. Provider next URLs are never followed with bearer tokens. Source revisions are preserved in the local normalized journal.

Meta

Configure CORE_META_ORIGIN, CORE_META_API_VERSION, CORE_META_TOKEN and CORE_META_ACCOUNT (act_ followed by the advertising account ID). No API-version default is guessed; select a version currently supported for the company app. Grant the minimum approved Marketing API read access and verify the company's assets.
The adapter reads account ID/currency/timezone, campaigns, then campaign-level daily Insights (time_increment=1). It requests only the fields it uses. Source decimal spend converts exactly to minor units; unsupported fractions fail instead of rounding. Currency must match policy. Campaign rows and Insights use bounded native cursors. Account timezone determines day boundaries, including daylight-saving transitions.
From the engine, choose the report period and Sync Meta. Only completed account days are accepted; adjust the exclusive end to a completed account date if needed. The API options are {start:"YYYY-MM-DD",end:"YYYY-MM-DD"} with an exclusive end. A repeated sync replaces revised daily spend in that interval instead of doubling it. Campaign labels can refresh while original normalized snapshots remain journaled.

After access arrives, compare daily campaign totals with Ads Manager for the exact account timezone/currency/range, verify all pages and archived campaign coverage, then replay and check that totals remain stable. No Lead Ads, Pixel/CAPI, creative write or ad mutation is included in this adapter.
Contract references: official Meta Insights guide, Meta's generated AdAccount SDK and Insights fields. The developer guide returned HTTP 429 during research; generated SDK contracts were read directly. Actual app version/permission acceptance remains an account gate.

HighLevel

Set CORE_GHL_ORIGIN, CORE_GHL_API_VERSION, CORE_GHL_TOKEN and CORE_GHL_LOCATION. Use a location-scoped private integration token with approved contact read access. The adapter uses POST /contacts/search as a read operation; it does not use deprecated contact listing or write contacts. The current v3 OpenAPI requires the Version header but underspecifies its search body. Page and returned searchAfter cursor handling are bounded and fixture tested; verify actual responses, total semantics and pagination on the company's selected version before acceptance.
Edit backend/data/ghl-mapping.json, or set CORE_GHL_MAPPING_PATH to a private mapping file. Values are actual custom-field IDs, not labels. Two modes are supported:

- Scalar fields: configure every non-journey key for consent grant/withdrawal time, campaign/touch time, appointment/consultation time and enrollment ID/time. Each contact may leave evidence absent. Paired campaign/time and enrollment/time must appear together. Missing consent never becomes presumed permission.
- Timeline field: configure journey to a JSON-text custom field containing {consents:[],touches:[],stages:[],enrollments:[]}. Entries follow the normalized record contract; contact ID is bound by the adapter. This preserves multiple touches/enrollments and repeated consent transitions. Other mapping keys may be blank in this mode. All records are validated before acceptance.

Business timestamps need explicit timezone offsets or UTC, never date-only guesses. dateAdded supplies contact creation. The enrollment ID must also appear in Stripe payment metadata. Do not map opportunity monetary value to a payment. A stage label alone does not establish historical event time. The company must populate these source fields from its approved process; the connector cannot invent that evidence.
Sync Meta before CRM touches that reference its campaigns. Then Sync HighLevel. The complete bounded contact snapshot supplies source IDs and approved timeline fields; names, email, phone, raw attribution text and arbitrary contact fields are discarded rather than persisted or sent to AI. Validate source counts and spot-check consent history, event times, enrollment IDs and missing-data behavior.
References: official contact search, official v3 schema, private integrations.

Stripe

The inspected existing key is test-mode. A real read-only account request succeeded; charges were disabled and no AdRevora signing secret was configured. No other project's payment data

was copied. Create a separate AdRevora endpoint/secret only through an authorized account setup; do not repurpose another application's endpoint.
Configure mode, key, optional pinned API version, signing secret, imported tenant, authorized local actor, and connected-account binding if applicable. Set CORE_STRIPE_ENABLED=true only after these are supplied. For each successful PaymentIntent, metadata key CORE_STRIPE_ENROLLMENT_METADATA (default adrevora_enrollment_id) must equal a known CRM enrollment ID.
Register the exact endpoint with Stripe: /api/core/stripe-webhook on an authorized reachable service. The current core is loopback-only; local test forwarding or a reviewed production HTTPS deployment is needed. Supplying a key does not register a public webhook or expose this service safely.
Use snapshot-format events and subscribe to payment_intent.succeeded, refund.created and refund.updated. Only collected amount_received and succeeded refunds enter the ledger. Raw HMAC, timestamp tolerance, account, test/live mode and configured version are checked. Replays have one financial effect. Distinct authentic events for the same unchanged payment preserve the first accepted posting time and journal the new event identity; changed amount/currency/enrollment identity is rejected. Out-of-order refunds and unknown enrollment references fail for retry/reconciliation instead of acknowledging lost money. No charge/refund creation API exists. A rejected old delivery must arrive with a fresh valid signature; do not relax signature checks to replay stored raw requests.
Before live acceptance, verify the account's actual event payload/version, metadata, successful-payment/refund sequence, duplicate delivery, retry behavior and source reconciliation. Backfill older source records using validated normalized imports; the receiver is not an exhaustive historical Stripe crawler. Disputes, processor fees, taxes, FX and chargebacks require additional scope and must not be hidden in the current recorded-contribution figure.
References: Stripe signature verification, PaymentIntent, Refund.

Activation evidence to save

Record current versions/scopes privately, successful/failed source counts and exact date/currency/timezone comparison, consent/enrollment matching, test-mode Stripe replay/refund proof and a second sync showing stable totals. Keep private payloads and screenshots ignored. Update dossier/checkpoint before any deployment. Live activation is incomplete until these real-account checks pass, even if config status says configured.

Revenue intelligence core — implementation contract

Source guide: docs/CORE-IMPLEMENTATION-SPEC.md
Status: implementation started 2026-10-12. This document is written before code. The owner requested completion of the main product focus and durable interruption checkpoints. This authorizes local implementation, not paid services or invented company/provider access. The existing static website remains a separate release.

Before and intended after

Before: browser reference records and local decisions; no executable API, durable revenue ledger, session authentication or connected model. After this local milestone: authenticated, tenant-scoped HTTP application with durable imports, deterministic funnel/finance/attribution, evidence-bound recommendations and persisted human decisions. Real account integrations and production acceptance remain separate gates until provider access and operating approval exist.

Acceptance criteria

Review regression contract: campaign leads are distinct contacts created within the report window with at least one paid touch in that window, consented at the touch and still consented now. Extending the report end without adding records must not remove those leads or change revenue per lead. Enrollment attribution retains its event-anchored lookback. A policy revision invalidates earlier recommendation evidence.

- A reproducible local backend starts from validated settings and migrates a durable database; restart preserves records and decisions.
- Password authentication uses scrypt, hashed opaque expiring sessions, logout, same-origin mutation checks and tenant membership; cross-tenant attempts fail.
- A bounded, validated normalized import records campaigns, spend, contacts, touches, stage events, enrollments, payments and refunds transactionally. Replays have one effect; changed identifiers fail without partial writes.
- Money is integer minor units and one configured currency. Refunds cannot exceed received payments. Opportunity/enrollment values are not collected cash.
- First and last eligible paid-touch attribution are deterministic, time-bounded, and consent-aware. Unknown revenue is explicit and never silently allocated.
- Reports show source freshness, funnel counts, net collected cash, known costs, contribution, acquisition cost, and source reconciliation independently of provider attribution claims. Empty denominators remain unknown.
- Recommendations calculate evidence server-side, follow maintained policy, expose insufficient/stale evidence, and never execute advertising changes. A separately enabled local Ollama or verified-free adapter may select bounded investigation checks from aggregate evidence; all financial prose is deterministic; no paid inference or silent AI fallback is permitted.
- Approval/rejection binds tenant, campaign, evidence hash, model, policy and expiry; changed or expired evidence invalidates an approval. Audit persists.
- A connected browser interface supports login, import, reports, evidence, recommendation and decision flows, clearly separating examples from imports.
- Automated negative/regression tests, type/lint/security/build checks and real HTTP/browser flows have recorded results. Missing gates stay explicit.
- Dossier, README, backend guide, roadmap, configuration example and resumable project memory reflect actual progress and before/after evidence.

Account-ready connector extension (owner clarification, before implementation)

The owner has no Meta/HighLevel access yet and requests all preparable code now. Add read-only, manually triggered native API sync and private setup diagnostics; all account IDs, API versions, tokens and CRM field mappings are configuration. No paid API, provider mutation, background ad execution or invented consent.

- Meta campaign/daily-spend and HighLevel contact/custom-field adapters follow official contracts, bound pagination/time/record volume, reject wrong account, currency and ambiguous data, and never follow provider-supplied URLs with tokens.
- Sync updates revised source snapshots atomically, journals normalized revisions, preserves unrelated records, rejects cross-provider ownership and leaves the last good ledger untouched on errors. Examples cannot receive connector data.
- Configuration diagnostics and browser sync controls expose unconfigured access. Fixture-backed real HTTP tests exercise successful sync, corrections, replay, pagination, provider errors and role isolation. Live access remains unverified until credentials and company-specific mappings are supplied and tested.

HighLevel snapshots use explicit custom fields for consent and dated business events. A pipeline label or opportunity monetary value is not proof of consent, consultation, enrollment time or cash. Mapping these fields is account setup, not a reason to embed customer-specific business logic in source code.

Architecture decision and configuration ownership

Use the existing TypeScript stack with a small modular Node HTTP service and Node's built-in SQLite for a single-process local core. This adds no framework or database dependency. SQLite is not the proposed multi-region production design; PostgreSQL/OIDC/workers remain production architecture gates. HTTP/auth/domain boundaries permit replacing the local adapter without moving financial policy into UI code. Local password accounts are not enterprise OIDC/MFA acceptance.

- Bind host, port, origins, database location, limits, expiry, model URL/ID/timeout; Authoritative owner: backend/settings.ts, populated from env; .env.example documents every variable
- Local passwords/bootstrap access; Authoritative owner: Ignored .env and CREDENTIALS.md; only hashes in database
- Recommendation thresholds, messages, currency/attribution defaults; Authoritative owner: Validated backend/data/policy.json
- AI instructions/output contract; Authoritative owner: backend/data/ai.json; aggregate evidence only
- Reference journey; Authoritative owner: backend/data/example.json; maintained reference records
- Routes, cookie/header syntax, schema version; Authoritative owner: Named protocol module/schema; immutable syntax
- UI labels/layout; Authoritative owner: Maintained frontend content/data and CSS

The model adapter accepts explicitly configured local Ollama or verified-zero-price OpenRouter routes. Paid inference, provider mutations and ad spending are outside this milestone. Native read connectors are researched and fixture tested before account access; live operation requires granted scopes and real verification.

Research

- Node SQLite:

synchronous database and parameterized prepared statements; local scope only.

- OWASP session management:

opaque sessions, cookie protections and session lifecycle.

- OWASP CSRF:

exact origin validation in addition to SameSite cookies.

- Ollama structured output:

JSON-schema output must still be validated and cannot authorize actions.

Continuity and release gates

memory/core-implementation.md is the current checkpoint, updated at each stable step and before stopping. Keep test output and snapshots in ignored artifacts/. Do not commit, deploy, enable CI, alter Google Docs/PDFs or claim the full ten-phase program complete as a side effect. Before promoting frontend changes, compare the previous deployed artifact with the local baseline and record parity/drift. Fresh-context review remains required by the global workflow before completion.

Local core verification — 2026-10-12

Source guide: docs/CORE-VERIFICATION.md
Scope: local advertising-to-collected-revenue engine. This record does not certify the entire ten-phase platform or a production service. The implementation contract was written before code; it contains fourteen local acceptance criteria.

Before → after

- Static reference interface; browser-only state; Implemented local behavior: Authenticated HTTP engine, SQLite ledger, persistent decisions/audit
- No actual financial source handling; Implemented local behavior: Validated money/control totals, successful payment/refund receiver and unknowns
- Attribution definitions only; Implemented local behavior: Deterministic consent-aware first/last eligible paid touches and reconciliation
- No native account adapters; Implemented local behavior: Meta daily campaign spend and HighLevel explicit timelines, fixture tested
- No connected model; Implemented local behavior: Actual installed local Ollama call; allowlisted checks and deterministic financial prose

- Unrecorded implementation continuity; Implemented local behavior: Project-local checkpoints, dossier, credential recovery and preserved evidence

The maintained example reconciles $10,000 collected − $500 refunded = $9,500 net; $7,000 attributed + $2,500 unknown = $9,500, difference $0. Recorded contribution is $6,500 after $1,000 spend and $2,000 known costs. These are example values.

Evidence established

- 89 automated JS/TS tests passed across eight files after atomic-bootstrap and scoped test-discovery verification. Temporary audit copies are excluded. Domain cases cover negative tenant/role access, consent withdrawal, replay/collision, rollback, refunds/controls, maturity/staleness, all policy states and restart persistence.
- Independent review reproduced two defects: separate Stripe events conflicted on unchanged payment identity, and wider report windows aged out campaign leads. Both failed regression tests before correction and pass afterward. The reviewer approved the local scope with no remaining blocking findings; independent post-fix domain/provider/config checks passed 58/58, with earlier HTTP/connector checks 16/16.
- Real local HTTP exercises login/cookies, import/report/recommend/approve/history/logout and Host/Origin/body/method rejection.
- Fixture-backed real HTTP sync exercises Meta → HighLevel → signed Stripe payment → successful refund → fully attributed $450.00 net cash with zero difference. Native pagination, revised spend without double counting, failed sync rollback, account binding, example/analyst rejection and malicious next URL handling pass.
- Real Stripe test-key account read succeeded; charges disabled, webhook unconfigured. No real charge/refund or company CRM data was created/imported.
- Corrected real local model call returned HTTP 200 and zero vendor usage cost. Selected campaign prose is server-rendered $7,000.00 net, $800.00 spend and $4,700.00 recorded contribution. Unknown/duplicate codes and extra prose reject.
- OpenRouter free inference was rejected by existing account privacy policy. Privacy remained unchanged; no paid fallback was attempted.
- Before frontend edits, 26/26 deployed static files matched the captured local baseline. Production has not been changed in this task.

Final gates

- Tests; Verified result: 89/89 JS/TS; 4/4 Python deployment tests
- Type check; Verified result: Frontend and backend pass
- Lint; Verified result: Source, backend, automated tests and build config pass
- Build; Verified result: Vite build and all local route entries pass
- Browser; Verified result: 13 checks, zero application errors, 320/390/768/1440px; readable scrollable table verified
- Gitleaks; Verified result: Public-source and full Git history pass; private paths ignored/untracked/history absent
- Bandit; Verified result: Public Python audit pass; manual script finding fixed through the installed hook

- Semgrep; Verified result: 81 public files actually scanned, zero findings/errors after fixing initial zero-file scan
- Configuration; Verified result: Env ownership/example coverage, provider/model/secret boundary regression tests pass
- Formatting/edit hook; Verified result: Installed scanner passes; final formatting refresh recorded at handover
- Fresh-context review; Verified result: Approved for local scope; both reproduced defects fixed and independently rechecked

All fourteen scoped criteria have implementation and review evidence. Final runtime and security evidence is refreshed after the review fixes before handover. Real provider activation and production acceptance are separate from these local criteria.
Security audit: real secrets found in public source/history no. Actual private keys remain only in ignored recovery/environment files. Changeable provider URLs, versions, model IDs, thresholds, limits, paths and content use typed settings/data. Intentionally fixed constants are HTTP/cookie/provider field syntax, supported record/action enums and OWASP scrypt parameters. Currency scale is policy data. No new shipped runtime dependency was introduced.

Remaining acceptance boundaries

Company Meta/HighLevel accounts, real pagination/count behavior and custom-field mapping are unavailable. Own Stripe signing secret/public receiver registration, historical source completeness and live reconciliation remain account setup gates. Read the connector activation guide.
OIDC/MFA, hosted backend/PostgreSQL/workers, vault and privacy lifecycle, Pixel/CAPI, wider creative/director workflows, historical finance/chargebacks/FX/fees, independent assessment, pilot, load/recovery and measured SLOs are not closed. No advertising executor or paid API route runs. Source is local on the feature branch; no push, production promotion, remote CI enablement or Google Doc/PDF update is claimed.
This record captures the implementation gate before source publication. Subsequent GitHub and document/PDF synchronization belongs to the delivery verification; production acceptance remains governed by the release-readiness matrix.

Integrations, tracking, data and attribution

Source guide: docs/INTEGRATIONS-AND-DATA.md
Status: local Meta campaigns/daily Insights and HighLevel contact-timeline adapters are implemented, with bounded native pagination, source ownership, account binding, atomic revision journals and failure rollback. Company access is unavailable; live compatibility remains unverified. Stripe is the selected financial authority: successful collected payments/refunds have a signed receiver and real read-only test-key readiness proof. No real payment was created and the webhook is disabled. See connector setup for every account/mapping activation step. The wider tracking/OAuth/production contracts below remain future requirements.

Provider register and installation

Maintain a private versioned register per connector: company-owned app/account, verified API version, requested/granted scopes, access level/review status, supported endpoints, webhook verification, limits, token lifecycle, retention/terms and test evidence. Provider IDs are opaque strings. Do not scrape or use undocumented/deprecated functionality.

- Meta; Initial data: Accounts, campaigns, ad sets, ads, creatives, Insights, authorized Lead Ads; Write boundary: Internal drafts first; approved executor later; Implementation prerequisite: Current version, company assets, app review/permissions and authorized test data
- GHL; Initial data: Contacts, opportunities, pipelines, calendars/appointments, tags and approved fields; Write boundary: Selected events/fields only after operations approval; Implementation prerequisite: Installation/location scopes, stage definitions and webhook verification
- Google Ads/GA4; Initial data: Campaign spend, approved conversions and reporting; Write boundary: Approved conversion ingestion and later scoped changes; Implementation prerequisite: Developer/access approval, conversion route and consent requirements
- Finance/enrollment source; Initial data: Enrollment evidence, payments, refunds, fees/costs; Write boundary: Source-owned records; controlled reconciliation; Implementation prerequisite: Approved authoritative source and immutable posting identity

OAuth exchanges happen on the backend, with state binding, exact callbacks, minimum scopes, encrypted secret references and serialized refresh. Never store provider tokens in browser storage or prompts. Revocation disables jobs and requires explicit reconnection. Webhook tenant/account binding comes from verified installation metadata, not body trust. Meta's documentation returned HTTP 429 during earlier planning research. Local connector implementation subsequently used official contracts/generated SDK evidence and fixture-backed HTTP checks recorded in connector setup. No company-account version or permission set is verified. Before live activation, verify current official docs, configured versions/scopes and actual account access. Meta Marketing API. GHL OAuth and webhooks are starting points for future production installation/event expansion; verify event-specific support/signatures. The local GHL connector is a manually triggered read-only contacts search with explicit field mapping. For Google, select the supported ingestion path for the actual account at implementation; see conversion management.

Synchronization and trusted reporting

Capture bounded historical backfill, pagination/cursors, account currency/timezone, query/API versions, receipt timestamps and sync checkpoints. Reconcile a bounded account/date sample against each provider UI/export with documented metric definitions, reporting lag and attribution windows. Refresh approved partitions for late conversions.

Jobs have per-provider/account limits, bounded retries, backoff/jitter, circuit breakers, dead-letter review and reconnect states. Persist webhook receipts before acknowledging

success. A duplicate receipt can update attempt telemetry without repeating business effects. Dashboard refreshes use scoped local read models; exports and backfills run as jobs.

Audit tracking before modifying assets

Inventory accessible company datasets/pixels, domains, websites, funnels, tag containers, CMS plugins, existing CAPI senders and GHL automations. Record ownership, event triggers, consent state, diagnostics and the route from business occurrence to receipt. Find missing triggers, duplicate tags, obsolete senders and incorrect values/timezones.
Reuse suitable assets. New datasets require an approved ownership/business rationale. Snapshot configuration and rollback before migration/removal. Inspect actual network payloads and provider diagnostics; a homepage tag is not a complete revenue architecture. Do not place advertising tags in the authenticated AdRevora console by default.

Event dictionary

Internal names are proposals; approve standard/custom external mappings, allowed fields, source values, event-age limits and optimization eligibility against the current account.

- Landing viewed; Authority: Approved public page; Browser: After required consent; Backend/CRM: Optional receipt; External decision: PageView if permitted
- Event registration/lead accepted; Authority: Successful form or Meta Lead Ad; Browser: Website success only; Backend/CRM: Preserve form/provider identity; External decision: One approved Lead/registration mapping
- Attendance; Authority: Verified attendance system; Browser: Never inferred from a page view; Backend/CRM: Source integration; External decision: Internal first; optional approved custom mapping
- Platinum appointment; Authority: Confirmed GHL booking; Browser: Only same confirmed booking; Backend/CRM: Appointment identity/status; External decision: Schedule/custom mapping if permitted
- Consultation completed; Authority: Defined CRM stage and operator evidence; Browser: No; Backend/CRM: Verified transition; External decision: Internal first
- Platinum enrollment; Authority: Verified enrollment source; Browser: Optional exact confirmation; Backend/CRM: Enrollment identity; External decision: Avoid duplicate payment reporting
- Collected payment; Authority: Approved finance/payment record; Browser: Optional same payment; Backend/CRM: Actual value/currency; External decision: Purchase if permitted
- Refund/chargeback; Authority: Finance source; Browser: No; Backend/CRM: Reversing ledger posting; External decision: Only supported adjustment route

Every event stores tenant/unit, stable identity, schema, occurrence/receipt times, source provenance, consent/purpose and allowed evidence. Business stages are distinct; a lead is not a booking and an enrollment is not automatically a payment.

Deduplication and CAPI

For browser/server copies of one conversion, propagate one opaque durable event ID and the same provider event name within the same dataset. Browser and server senders must preserve this identity; queue retries reuse it. A subsequent booking/payment gets a new identity. Never use an email as the event ID or randomly regenerate it on retry.
Deduplicate provider receipts, logical events and financial postings independently. CAPI deliveries use an outbox, current consent checks at queue/send time, approved matching fields, provider-required normalization/hashing, valid occurrence time and source/value/currency. Hashing is not anonymization. Strip sensitive query strings and insurance/financial details from URLs and external payloads.
Validate browser+server reception/dedup with Meta Test Events and diagnostics on approved assets. Test repeated delivery, duplicate tags, second real conversion, browser-only, CRM-only, denied/withdrawn consent, invalid fields, delayed events and partial failure. HTTP success alone is insufficient. Meta deduplication reference. Record supported behavior against current docs/account tests before production.

Logical schema

Phase 2 turns these domains into reviewed migrations and an ER diagram. All tenant relationships preserve tenant scope; uniqueness includes provider/account where needed.

- Access; Entities: tenants, business_units, users, memberships, roles, sessions
- Connections; Entities: installations, external_accounts, secret_refs, granted_scopes, cursors, receipts
- Advertising; Entities: campaigns, ad_sets, ads, creatives, offers, markets, historical_names, daily_insights
- Tracking; Entities: events, deliveries, consent_receipts, touchpoints
- CRM; Entities: contacts, identity_links, merges, leads, lead_forms, opportunities, stage_history
- Funnel; Entities: appointments, appointment_transitions, consultations, enrollments
- Finance; Entities: orders, invoices, payments, refunds, chargebacks, revenue_postings, costs, FX
- Attribution; Entities: model_versions, runs, credits, evidence_links, unknown_reasons
- AI/actions; Entities: analysis_runs, evidence_snapshots, recommendations, drafts, approvals, reservations, executions
- Operations/privacy; Entities: outbox, job_runs, audit_events, reconciliation_exceptions, retention/deletion_tasks

The diagram is conceptual; tenant constraints, refund references, consent and remaining entities are specified in migrations. Use precise decimals/minor units, original currency, versioned FX and distinct occurrence/receipt/account-reporting times. Partition/archive high-volume events based on measured workload and approved retention.

Identity and attribution

Capture unit/provider/account/campaign/ad-set/ad/creative IDs, historical names, UTMs, approved click/browser identifiers, page/form, creation time and permitted first/last touches. Link GHL contact, appointments, opportunity, enrollment and finance evidence. Validate parameters; names and UTMs alone do not prove an exact ad match.

Prefer explicit lead-form IDs and validated ad/click links. Contact matching is scoped, confidence-based and correctable with history. Handle merges without duplicating money. Do not fingerprint or infer an unobserved cross-device journey. Expose deterministic, partial and unknown outcomes with reason codes and coverage.

Initial models: first eligible touch and last eligible paid touch. Window, eligibility, lookback and refund/cohort policy are versioned product data. Later multi-touch credits sum to at most 100% of a posting. Keep network-reported views separate: Meta and Google may both claim the same sale. Attribution credit does not prove causal lift.

A $5,000 collected payment links to verified enrollment/contact and eligible recorded touch, then the actual account/ad/campaign. If evidence is missing, retain $5,000 in total revenue with unknown attribution. If two campaigns touched the person, show model-specific credit and the touch timeline. Refunds append reversing postings and restate metrics without deleting the original transaction.

Financial definitions

- CPL; Definition and required label: Approved ad spend / distinct defined leads
- Cost per appointment; Definition and required label: Spend / qualifying unique appointments; cancellation policy
- Cost per enrollment; Definition and required label: Spend / verified enrollments; refund and cohort basis
- CAC; Definition and required label: Agreed acquisition costs / new customers; disclose ad-only basis
- Revenue per lead; Definition and required label: Net collected or recognized revenue / cohort leads; basis
- ROAS; Definition and required label: Attributed revenue / ad spend; model/window/currency
- Profit; Definition and required label: Revenue less refunds, ads, fulfillment/COGS, fees, commissions and agreed costs
- Attribution coverage; Definition and required label: Eligible revenue/sales with adequate evidence / all eligible revenue/sales

Missing costs mean unavailable or explicitly partial contribution, not audited profit. Zero denominators produce insufficient-data states. Never sum currencies without approved FX. Distinguish acquisition cohort from calendar-period cash reporting.

End-to-end acceptance journey

On approved accounts, prove ad identity → consented Pixel/CAPI event → lead → GHL contact → appointment → completed consultation → enrollment → real money posting → report → AI

recommendation. Also prove unmatched sale, multiple touches, installments, refund, repeated webhook, delayed outcome, contact merge and consent withdrawal. Reconcile source totals daily, expose exceptions and retain a sanitized evidence trace.

AI workflows, approval and governance

Source guide: docs/AI-AND-GOVERNANCE.md
Status: the local revenue engine implements deterministic five-state policy, evidence-bound persistent approvals and a real local model adapter. Model output is restricted to validated investigation codes; all financial prose comes from the server. The live model test caught false currency/trend prose despite valid JSON, so unconstrained explanations were removed and regression tests added. OpenRouter supports only verified-zero-price routes and was blocked by existing account privacy; local Ollama is the verified current path. No paid fallback or provider action exists. See core evidence.
The broader creative/director/budget workflows below remain production requirements. Local SCALE/MAINTAIN/TEST/FIX/PAUSE results are advisory human-review states.

Workflow responsibilities

- Ads Director; Inputs and output: Reconciled economics → ranked priorities and executive brief; Initial authority: Recommend
- Strategist; Inputs and output: Offer/funnel/audience policy → objective, CTA, budget/KPI brief; Initial authority: Draft
- Creative Director; Inputs and output: Approved claims and creative history → hooks/copy/concepts/scripts; Initial authority: Draft
- Campaign Builder; Inputs and output: Approved brief/assets → internal campaign/ad-set/ad draft; Initial authority: No publication
- Analyst; Inputs and output: Entire funnel → evidence and alternative diagnoses; Initial authority: Analyze
- Budget Optimizer; Inputs and output: Mature cohorts, cost/uncertainty → five-state recommendation; Initial authority: No budget write
- Retargeting Manager; Inputs and output: Verified stage/exclusions → next-action segment plan; Initial authority: Propose
- Attribution Agent; Inputs and output: Deterministic credits/ledger → explain trace and unknowns; Initial authority: No invented identity/money

Implement a bounded workflow/state machine before introducing distributed agents. Use a provider-neutral model gateway with configured model IDs, schema validation, timeouts/retries and per-tenant cost caps. Historical learning starts with versioned queries, creative metadata and evaluated prompts; customer-data model training needs separate terms, purpose and approval.

Recommendation contract

Every recommendation includes tenant/account/campaign, reporting/cohort window, source timestamps, attribution model, financial basis, sample size, conversion lag, evidence hash, model/prompt/policy versions, uncertainty, expiration and rationale.

- SCALE; Interpretation: Economics support a bounded human-reviewed increase
- MAINTAIN; Interpretation: Current economics justify continuation within approved limits
- TEST; Interpretation: Run a defined hypothesis with a capped budget and success/stop rule
- FIX; Interpretation: A tracking, landing-page, follow-up, appointment, offer or sales issue needs review
- PAUSE; Interpretation: Evidence supports human review of a stop; no automatic initial action

Good CTR with poor landing conversion suggests a page/offer issue; strong lead volume with weak appointments suggests follow-up; strong appointments with low closes suggests sales/offer issues. Separate these hypotheses from conclusions. Low sample counts, stale data, unknown revenue and immature cohorts reduce confidence. Two enrollments alone do not establish a reliable scaling rule.

Human authority

- Publish/pause, launch a test, change creative/budget within policy; Required approver: Marketing Director
- Routing, sales process or significant operations; Required approver: COO
- Major spend, new offer/pricing/market, significant vendor or strategy change; Required approver: CEO
- Asset ownership, users/partners, payment methods, security; Required approver: Human administrator only; outside AI executor

Approvals bind exact tenant, business unit, account, action/payload hash, evidence, policy version, ceiling, approver and expiry. Changed payload/evidence or revoked permission invalidates approval. Recheck live provider state and authorization just before execution; rejected/expired approvals never become executable by prompt text.

Future spending controls

A separate restricted executor has only approved write capabilities. Enforce account, company and campaign ceilings, shared reservations/locks for concurrent requests, pacing, cool-down, permitted delta, max action count and a global/company kill switch. A daily platform budget is not a guaranteed invoice cap: account for provider delivery behavior, attribution lag and pending changes. Reserve worst approved exposure and reconcile live spend; fail closed when trustworthy spend or policy is unavailable.

Record idempotency identity, before/after state, execution result and uncertain external outcomes. Reversal uses compensating action; served ads, spent money and delivered data cannot be literally undone. Stop new changes and escalate when reconciliation fails. Test

simultaneous requests, timeout-after-success, replay, stale approval, ceiling changes and kill-switch behavior before enabling automation.

Model security and evaluation

Treat CRM text, imported ads and web content as untrusted data. Separate instructions from evidence; allowlist tool schemas/actions. No secrets, raw contact payloads or cross-tenant retrieval enters prompts. Minimize aggregates and assess vendor training, retention, residency and subprocessors. Log safe IDs/versions/cost rather than prompts containing sensitive source material.
Evaluation covers arithmetic grounded in tools, unsupported claims, wrong tenant, privacy-sensitive content, missing disclaimers, prompt injection, ambiguous attribution, low samples, model refusal/outage and unauthorized execution. Store a versioned dataset and release thresholds; a model upgrade needs evaluation rather than an automatic swap.

Ownership and audit

Company-controlled accounts own code, domains, cloud, Meta/GHL assets, database, creative history, prompts and workflows. Developers receive scoped access, revoked at handover. Audit captures actor, tenant, request, evidence, approvals, before/after, correlation, timestamps and outcome; sensitive details follow retention/access policy.
Define named product, technical, finance, security/privacy and operations owners in private governance records. The requirements brief does not establish client consent or permission for production accounts. Compliance and operations define expansion/release controls.

US/EU compliance and privacy implementation plan

Source guide: docs/COMPLIANCE.md
Research reviewed: 2026-10-12. This is the engineering/legal work register for the connected platform and its future releases. The published release is static frontend; the repository also includes an authenticated local revenue runtime. Company-account activation is pending. The local runtime stores source identifiers, consent timelines and financial records, so data minimization and access controls apply even without names or emails. Applicable obligations require qualified review, implemented controls and operating evidence before relevant processing starts.

Scope and accountable decisions

Before onboarding, record legal entity/establishment, target countries/states, user and lead locations, business sector, age scope, collected fields, purposes, vendors, data flow, hosting/transfer destinations and controller/processor responsibilities. The company controls its marketing purposes; AdRevora's future SaaS role depends on its contracts and actual processing. Do not assume one role for every activity.
Maintain a versioned applicability register with legal basis, owner, controls, evidence, decision date, next review and release gate. Counsel owns legal interpretation; privacy owner

rights/consent; security owner safeguards; finance owner retention and money; marketing owner claims/disclosures; product owner accepted scope. Reassess when adding a network, country, business, data type, AI use or sales communication channel.

EU/EEA data protection

The GDPR provides the primary review framework for roles, lawful basis, transparency, rights, privacy by design, security, processor terms, impact assessment and international transfers. Engineering controls:

- Purpose/basis; Required implementation evidence: Field-level inventory, lawful-purpose decisions, notices and minimization
- Roles/vendors; Required implementation evidence: Controller/processor mapping, agreements, subprocessor/change review
- Rights; Required implementation evidence: Verified intake, scoped access/correction/deletion/export/restriction/objection workflows
- Retention; Required implementation evidence: Approved schedule per data class, deletion jobs, financial/legal-hold exceptions
- High-risk processing; Required implementation evidence: DPIA/DPO/representative/profiling and automated-decision applicability decisions
- Transfer; Required implementation evidence: Approved mechanism and transfer assessment where required, vendor/access locations
- Security/breach; Required implementation evidence: Risk-based safeguards, incident ownership and notification decision evidence

Track rights deadlines by jurisdiction rather than hardcoding one global period. A qualified owner determines whether Article 22 or other decision safeguards apply. Under GDPR Article 33, notify the competent authority where required without undue delay and, where feasible, within 72 hours of awareness; assess affected-person duties separately. Incident escalation must enable those decisions rather than wait for a complete technical postmortem. Keep legally required financial records under explicit restricted retention; deletion is not indiscriminate destruction.

Device tracking and marketing delivery

Assess national ePrivacy implementations for device storage/access, pixels and links. The EDPB final technical-scope guidance is a starting point. Where required, obtain valid consent before nonessential tracking, provide usable withdrawal and keep browser/server decisions consistent.

Consent receipts record purpose, notice/policy version, timestamp, source and permitted identifiers without collecting unnecessary data. A marketing outbox checks permission both when queued and before sending. Withdrawal suppresses pending sends and future eligible processing. CAPI, hashing and server-side architecture do not bypass choices. Test no-consent, reject-all, accept-selected, withdrawal, region/unknown state and retries. Keep authenticated console analytics separate from customers' advertising tags.

US state privacy

Use an applicability matrix for each targeted state, thresholds, exemptions and data activities. California review starts with the Attorney General CCPA overview and CPPA FAQs. Covered processing may require access, correction/deletion and sale/sharing or targeted-ad opt-outs, including qualifying opt-out preference signals such as Global Privacy Control. Review sensitive/minor data and service-provider/contractor terms. CPPA's current regulation register also covers audit, risk-assessment, automated-decision and insurance developments; verify applicability and operative dates for the actual business.

The CPPA adoption notice states the regulations became effective January 1, 2026, with phased compliance. Covered risk-assessment work begins in 2026; significant-decision ADMT duties begin in 2027; audit certification deadlines are phased by business size in 2028–2030. The privacy owner must verify the final regulations, thresholds and applicable deadlines before activating any covered processing. Advertising analytics alone does not settle ADMT classification.

Engineering acceptance includes signal handling through capture/outbox/replay, required links/notices, verified requests, vendor propagation and a legal exception record. Do not infer that a GLBA-related exemption removes all state privacy duties for a company or every dataset. Maintain state breach-notification decisions privately.

AI governance

Assess provider/deployer roles, prohibited practices, AI literacy, transparency and risk classification under the EU AI Act framework. The Commission's Article 50 guidance states transparency obligations apply from 2 August 2026, with a limited legacy-system marking transition. Its enforcement timeline, updated 6 October 2026, distinguishes later high-risk implementation dates. Recheck the enacted rules and classification before launch rather than treating all AI as one category.

Inventory interactive AI and generated media, determine applicable user disclosures and content marking, and preserve provenance. Review insurance, recruiting, education or eligibility uses separately; marketing recommendations do not authorize hiring, underwriting or other consequential decisions. Keep initial scope outside those functions. Human oversight, prompt/model versions, evaluation, vendor terms and incident response are implementation gates. AI/governance.

Sector, advertising and communications

- Housing/real estate; Required review and controls: Fair-housing duties, protected-characteristic proxies, audience exclusions and claims; Primary starting point: HUD digital-advertising guidance context
- Insurance/finance; Required review and controls: Applicable federal/state regulator, GLBA scope, licensing, disclosures and data restrictions; Primary starting point: FTC GLBA overview

- Advertising claims; Required review and controls: Substantiation, endorsements/results claims, material disclosures, generated-media rights; Primary starting point: FTC substantiation policy
- Email/calls/texts; Required review and controls: Channel-specific permission, opt-outs, suppression, records and sender requirements if introduced; Primary starting point: FTC CAN-SPAM guide, FCC telemarketing
- Accessibility; Required review and controls: Applicable accessibility duties and WCAG 2.2 AA engineering target; Primary starting point: W3C WCAG 2.2
- Children/vulnerable groups; Required review and controls: Age scope and sensitive targeting restrictions; no child-directed product in initial scope; Primary starting point: FTC COPPA resources

Historical guidance is context; counsel verifies continuing status and governing law. Provider advertising policies, special categories, audience restrictions and required business verification are additional release requirements. They do not replace law. Assess any DSA/platform or other sector obligations from the product's actual service role before expansion; do not apply a blanket checklist without an applicability decision.

Data lifecycle and release evidence

Payment data and regulated expansion

The revenue ledger needs verified payment references, currency, amounts and refunds; it does not require card numbers or card-security codes. Keep payment collection in an approved processor-hosted flow if introduced, outside analytics/model payloads. Determine the merchant/service-provider scope and required evidence with the payment owner. Stripe's integration security guide describes shared PCI responsibilities and lower-risk integration choices. Using a processor does not itself certify this project or remove the business's obligations.

Do not import health records, sensitive underwriting details or child-directed data into the initial marketing workflow. Any such sector expansion needs a separate field inventory, applicable-law review, contracts and controls before collection.

Retention and launch records

Classify public marketing content, business analytics, identity/contact data, financial records, consent, audit and secrets separately. Default external AI input to approved aggregates; exclude raw CRM/free text and sensitive financial/insurance fields. Hashes and pseudonymous IDs remain protected where identifiable. Protect exports and logs.

Rights/deletion jobs reach transactional DB, analytics, cache, object exports and vendors where required. Keep deletion tombstones and reapply them after backup restore before replay. Enforce purpose on replay so withdrawn events are not resent. Test legal holds and permitted financial retention without reactivating marketing.

Before connected launch, retain an approved applicability/data-flow register, notices/terms and company identity/contact, vendor agreements/transfer decisions, consent/network traces, rights/retention tests, security review, breach exercise, AI classification/evaluations and sector

sign-off. Name responsible owners and review cadence. Certifications and a statement of universal legal compliance require evidence beyond software design; public pages are updated to match the processing actually enabled.

Security policy and implementation requirements

Source guide: SECURITY.md

Private reporting

Contact the repository owner privately through GitHub or the established project channel. Do not publish credentials, customer data or infrastructure details in issues. Include affected commit/component, impact and safe reproduction. A dedicated monitored security mailbox, response SLA and bounty are not configured; establish response ownership before wider onboarding.

Current protections

The deployed scope is static frontend. No advertising token, customer database, login or connected AI runs in the browser. Assets are self-hosted. HTTPS, restrictive headers and a nonroot/read-only/resource-limited privately bound container are verified within that scope. See publication evidence.

The separate local engine uses loopback binding, exact Host/Origin checks, bounded JSON, scrypt passwords, hashed expiring HttpOnly/SameSite sessions, tenant roles, transactional financial validation and signed Stripe events. Meta/HighLevel tokens remain server-only. Names, email, phone and arbitrary CRM fields are discarded; source identifiers and approved timelines remain protected in the ledger, while AI receives aggregate evidence only. AI cannot return arbitrary prose, tools or financial actions. Provider pagination never follows an external next URL with credentials. These controls are local scope; OIDC/MFA, TLS backend deployment, vault/rotation, privacy lifecycle, recovery and independent assessment remain production gates.

Local Gitleaks, Bandit and Semgrep supplement tests/review. Repository Actions remain disabled. Re-evaluate CSP when adding approved backend/auth origins; do not replace specific rules with wildcards. Passing scans do not establish security certification.

Required platform controls

- Session; Controls: OIDC, MFA, secure cookies, CSRF, revocation; Acceptance evidence: Real login/revocation and session negatives
- Tenant; Controls: Membership scope, endpoint checks, RLS, tenant-aware jobs/cache/storage; Acceptance evidence: Cross-tenant read/write/export/retrieval tests
- Provider; Controls: Least-privilege OAuth, state/callback checks, encrypted tokens, refresh locking; Acceptance evidence: Install/revoke/expiry and permission register
- Ingestion; Controls: Verified signatures, bounded payload, replay/idempotency, durable receipts; Acceptance evidence: Tampered/duplicate/out-of-order tests

- AI; Controls: Minimal inputs, untrusted-content isolation, allowlisted tools, output schemas; Acceptance evidence: Injection and unauthorized-action evaluations
- Money/actions; Controls: Exact approval, live policy check, reservations, ceilings, kill switch; Acceptance evidence: Race/replay/stale-approval/reconciliation tests
- Runtime; Controls: TLS, private data services, restricted egress, minimum privileges, patching; Acceptance evidence: Configuration/security assessment
- Files; Controls: Scoped expiring exports, safe CSV, upload validation; Acceptance evidence: Unauthorized download and unsafe-file tests
- Secrets; Controls: Approved vault, separated environments, rotation, redacted logs; Acceptance evidence: Leak scan and rotation/recovery drill
- Availability; Controls: Multi-AZ, overload controls, backup/restore, on-call; Acceptance evidence: Failure/load/restore reports

Use a versioned OWASP ASVS baseline; version 5.0.0 is the researched starting point for a production coverage matrix. Record control IDs, applicability, implementation owner, test evidence and open findings. Threat-model cross-tenant access, token theft, forged events, duplicate money, injection, overspend, privacy replay and provider compromise. Keep detailed assessments private.

Lifecycle and launch gate

Production releases retain a reviewed dependency/license inventory and SBOM, signed or otherwise verifiable artifact provenance, patch ownership and rollback identity. Audit dependencies and container images at release; triage advisories by exposure and exploitability. Keep CI credentials least-privileged and separate build/deploy permissions. Never include private recovery records in release archives.

Classify/minimize data; enforce consent, retention, rights and deletion after restore. Hashing identifiers does not make them anonymous. Restrict sensitive insurance/financial fields from advertising/AI payloads. Review vendor terms, data use and transfers.

Assign dependency patch ownership. Every release requires tests, types/lint/build, secret/SAST/configuration scans, real flows and independent review. Connected production pilots require independent penetration testing, incident/recovery drills and closure of launch-blocking findings. Any future SOC 2/ISO audit needs separate operating evidence.

Contain incidents, preserve safe evidence, rotate affected credentials, reconcile external effects and assess notification duties with the privacy owner. Recover from verified artifacts and record corrective gates. See operations and compliance.

Scalability, availability and recovery

Source guide: docs/SCALABILITY-AND-RELIABILITY.md

Status: future acceptance contract. The published static website is not a benchmark of the connected platform. Keep measurements, estimates and approved targets distinct.

Signed workload before capacity purchase

The owner requires more than one million simultaneous dashboard users and more than one million simultaneous tracked visitors. Plan an overlapping 1.1M of each; measure interactive work and tracking work independently.

- Dashboard; Planning assumption: 1.1M active sessions, one API request per 10 seconds; Arithmetic rate: 110,000 API requests/second
- Tracking; Planning assumption: 1.1M visitors, one event per 30 seconds; Arithmetic rate: About 36,667 events/second
- Combined before caching/batching; Planning assumption: Both workloads at once; Arithmetic rate: About 146,667 operations/second
- Tracking sustained 24 hours; Planning assumption: Same event rate; Arithmetic rate: 3.168 billion events/day

At an assumed decimal 1 KB/event, raw tracking alone is about 3.168 TB/day before indexes, replicas, queue overhead or logs. Peak concurrency does not imply all-day traffic. Sign arrival distributions, peak/soak duration, regions, session activity, endpoint mix, report/export jobs, payload/event sizes, tenant skew, retention, data history, cache hit/miss and network conditions. Idle connections are a separate test.

Scaling boundaries

- Static UI/assets; Growth method: CDN and immutable hashed files; Guardrail: Keep HTML/artifact compatibility; verify headers
- Interactive API; Growth method: Stateless regional replicas, autoscaling and scoped caches; Guardrail: Per-tenant admission limits and no synchronous analytics scan
- Event ingestion; Growth method: Regional collectors and partitioned durable queues; Guardrail: Acknowledge only after durable acceptance
- Provider sync/CAPI; Growth method: Account-scoped workers, batching where supported; Guardrail: Provider budgets and consent; not one call per page view
- Reports/analytics; Growth method: Incremental materialization and analytical store; Guardrail: Async heavy jobs, query limits and freshness
- Transaction/finance; Growth method: Pooled PostgreSQL, indexed scoped queries, tenant cells; Guardrail: Single authoritative financial write path per tenant
- AI; Growth method: Async evidence-based jobs and cached authorized results; Guardrail: Token/cost/concurrency caps; no inference per visitor
- Storage/export; Growth method: Object lifecycle and expiring scoped downloads; Guardrail: Residency, deletion and access enforcement

Begin the connected pilot with multi-AZ compute, DB and durable queues. Introduce regional cells when benchmarks justify them. A tenant routes to its authorized cell; keep failure blast radius bounded. Data residency and cross-region replication require privacy approval. Avoid synchronous global financial writes. Tenant skew and noisy neighbors require per-tenant limits and isolation, not only fleet-wide scaling.

Capacity planning records sustainable throughput per instance/cell, DB connections, queue lag, bytes/event, cost/tenant and failure headroom. An autoscaler cannot create capacity beyond cloud quotas, DB limits or provider permissions; review those first.

Service objectives

Proposed production targets become accepted only after measurable evidence:

- Console/API availability; Target: At least 99.9% per calendar month; Measurement: Successful eligible critical requests / eligible requests, plus external user-journey probes
- Event acceptance availability; Target: At least 99.9% monthly, separately; Measurement: Durable accepted eligible events / eligible attempts; explicit rejection/retry counted by agreed contract
- Interactive latency; Target: p95 ≤500ms, p99 ≤1.5s; Measurement: Agreed endpoints at signed load; cache hit/miss separately
- Event acceptance latency; Target: p95 ≤300ms regional; Measurement: Time to durable acknowledgement
- Platform errors; Target: <0.1% under agreed steady load; Measurement: Defined endpoint failures/time windows
- Internal freshness; Target: 99% processed within 5 minutes; Measurement: Received valid event → available read model
- Recovery; Target: Core-data RPO ≤5min, RTO ≤60min; Measurement: Approved regional disaster drill, per data class
- In-region AZ failure; Target: No loss of acknowledged durable events; Measurement: Queue/DB guarantee plus actual failure exercise
- Routine deployments; Target: No user-visible failed critical requests caused by release; Measurement: Canary, draining, compatibility and rollback evidence

99.9% corresponds to 43.2 minutes in a 30-day time-based month; request-based error budgets use eligible request counts instead. Do not interchange the two. Final contracts define valid attempts, maintenance, customer/network errors, regions and provider journeys. Keep user-visible end-to-end failures visible even when a provider is at fault. A cloud SLA is not measured application availability. Google SRE SLO guidance.

For comparison, a 99% time-based objective allows 432 minutes in a 30-day month. The roadmap selects the stronger 99.9% objective. Neither percentage is an observed uptime claim for the local engine. Agree the operating window, exclusions and measurement before any SLA or customer commitment.

Keep last trustworthy data with timestamps during provider outages; stop unsafe actions when source freshness fails policy. Isolate AI/provider work from core event acceptance. Use error-budget burn to freeze risky releases and prioritize recovery.

Benchmark and resilience gates

Ramp through 1k, 10k, 100k and 1.1M active dashboard users with matching tracking load. Use distributed generators, representative data/tenant skew, cold/warm caches, auth, expensive endpoints and real payload sizes. Final proposal includes at least a two-hour peak and 24-hour representative soak, plus bursts. Production-scale generators and vendor traffic require a separately verified budget and spend cap.

Use controlled provider adapters for load generation; separately perform bounded real provider contract tests. Never send the million-user load to external networks or CRM. Measure p50/p95/p99, error rate, saturation, DB/queue lag, backlog drain, autoscaling, cache misses,

egress and observed unit cost. Exercise AZ loss, replica/worker termination, DB failover, provider/AI outage, queue duplication, region recovery and canary rollback.
Acceptance requires the signed workload, generator/config versions, artifact/infra identity, data size, complete results, safety margin, failures/remediation and independent review. Capacity claims stop at the largest successfully exercised envelope.

Backup, restore and deployment continuity

The local HTTP/SQLite process is a functional verification baseline. Before scaling it into a hosted system, migrate with reconciled data to tenant-constrained PostgreSQL, external session/identity services, durable jobs/outbox and analytical read models. Measure a cell's safe envelope before deriving fleet size; include AZ-loss headroom, DB connection ceilings and hot-tenant isolation in that envelope.
Encrypted backups/PITR belong to company accounts with least-privilege recovery access. Record retention and geographical/legal constraints. Test restore in isolation, reconcile finance totals, reapply privacy deletion tombstones and verify replay cannot send withdrawn marketing events. Measure actual RPO/RTO and publish scrubbed results.
Use expand/migrate/contract schemas, compatible old/new workers, graceful draining, canaries, readiness probes and rollback thresholds. Keep source/artifact/config identity and local/deployed digest parity. External side effects require compensating reconciliation; restoring a DB does not reverse ads already served or cash spent.

Cost and operating team

Pilot budget allowances are in the roadmap; they do not price this million-user workload. Price compute/HA headroom, DB/storage/I/O, queue/analytics, CDN/WAF/network transfer, observability, identity, backups, AI, CI and 24/7 staffing using measured units and current regional vendor calculators. Record baseline/peak/ stress scenarios and per-tenant/event/report cost. Alerts do not impose hard caps.
Assign SRE, incident commander, backup maintainer and an on-call rota before contractual uptime. Operations defines incident and release ownership. No new resources or paid load tests are activated by writing this contract.

Release, operations and developer handover

Source guide: docs/OPERATIONS.md
Production remains static HTTPS frontend. A separate local core now runs under the runtime/recovery guide. Native source activation follows the connector setup; no background sync or external writes run. The production data procedures below remain required before promotion. Company-owned accounts and reviewed local source remain authoritative; private recovery records hold actual access details.

Current website runbook

Use deployment tooling and the static runbook. Build locally, deploy only static output/generated configuration, retain the previous release, verify real browser/HTTP behavior and prove every deployed file hash. Do not hand-edit live code. A documentation-only source push does not promote a website release.

Future environment and delivery contract

Separate development, staging and production identity, data, secrets, queues and quotas. Use reserved test records and approved provider test environments; never copy customer data into development without a documented lawful minimization process. Adopt declarative company-owned infrastructure, verified native inventories, short-lived CI grants and pinned/scanned artifacts. Remote Actions stay disabled until actual billing and execution are approved; source pushes do not enable workflows.

Each release records source commit, dependency lock, artifact/container digest, configuration/schema version, migration plan, acceptance/gates/review, operator, observability and rollback trigger. Snapshot deployed manifests/files before changing anything; reconcile live-ahead drift into local first. Promote staging, then a bounded production canary with approval. Verify served identity and local/artifact parity.

Use readiness, graceful connection/worker draining, backward-compatible migrations and old/new worker tests. Retain rollback artifacts and restore procedures. Database recovery is not a reversal of external financial/ad effects; reconcile/compensate them explicitly.

Observability and alerting

- Critical API/event SLO and burn rate; Owner/action: On-call SRE; incident/freeze release
- DB saturation/replication, queue age/backlog, ingestion rejection; Owner/action: Platform owner; isolate overload and drain safely
- Provider freshness, rate limits, auth/revocation; Owner/action: Integration owner; reconnect or degrade read-only
- Ledger/source difference and attribution coverage; Owner/action: Finance/data owner; quarantine/reconcile exceptions
- Approval execution, spend reservation and policy denial; Owner/action: Marketing/security owner; stop unsafe changes
- AI schema/evidence/cost failure; Owner/action: AI owner; deterministic fallback and suspend affected workflow
- Consent/rights/deletion failures; Owner/action: Privacy owner; suppress affected processing and assess obligations
- Backup/restore and certificate expiry; Owner/action: SRE; repair before recovery/security exposure

Use structured metrics/traces/logs with correlation/tenant-safe IDs. Redact tokens, contact fields and full provider bodies. Keep cardinality/retention/cost bounded. Public status communications expose service impact, not private infrastructure.

Incident procedure

- Assign incident commander, technical lead and communications/privacy contacts.
- Contain dangerous execution, compromised access or affected processing; preserve safe evidence and audit continuity. Use scoped/global kill switches where appropriate.
- Determine affected tenants, time, money/data and service impact. Keep a factual timeline.
- Restore a verified artifact or fail over under the approved runbook. Reconcile queues, payments and uncertain provider mutations before re-enabling work.
- Privacy/legal owners decide notification duties and deadlines. Use approved channels; do not send unsolicited customer communications as a tooling side effect.
- Verify recovery against SLO/freshness/data/finance checks; remove temporary privileges.
- Record cause, contributing conditions, corrective tests and owners/deadlines. Update private incident/defect logs and publish only an approved scrubbed summary.

Recovery and data lifecycle

Schedule encrypted backups/PITR and isolated restores. Measure RPO/RTO per data class, verify finance totals and file/artifact hashes, reapply deletion tombstones and consent before replay, and validate current schema/permissions. Test retention/legal holds. A successful backup job without a restore test is insufficient acceptance evidence.
Provider outages use durable queues, bounded retries and stale timestamps. Preserve last trustworthy metrics; fail closed on advertising execution when authority/economics are unknown. Replaying a dead-letter message requires cause review, current consent, idempotency and bounded scope. Secrets rotate through the approved vault and private recovery register, with revoked old grants and tested dependents.

User workflow handover

Marketing reviews full-funnel evidence and creative/test proposals, then explicitly approves a bounded action. COO reviews follow-up/appointments/CRM bottlenecks before routing changes. CEO reviews spend→sales→revenue→profit and major allocation decisions. All users can inspect freshness/model/unknown coverage rather than receiving unexplained scores. Permissions and a clear escalation path apply to every workflow.

Acceptance and maintenance checklist

Use the production-readiness matrix to connect these operating responsibilities to account activation, pilot and scale release gates. The source handover includes a locally verified backend; promoting it requires a separate production release decision and the operational evidence below.

- Company admins control source, domains, hosting, provider assets and recovery.
- Incoming developers can clone, install, run gates and find current/remaining scope.
- API/schema/configuration/event/attribution/AI and legal/security decisions are versioned.

- A named primary/backup maintainer and on-call rota cover the accepted service objective.
- Release, rollback, provider failure, credential rotation, rights and restore drills have evidence.
- Capacity reports include workload, tested envelope, headroom and observed cost.
- Open risks, access reviews, patch cadence, support/security contacts and renewals have owners.
- Source/remote equality, artifact parity and documentation links are verified at handover.

The public technical overview can be shared separately from the private repository. Existing Google Docs retain content/ID; explicitly authorized updates use revision-aware edits and a validated temporary PDF before canonical replacement. Reference projects are not modified. See the documentation index for current guides/evidence.

Development and contribution

Source guide: CONTRIBUTING.md

Start from source

Clone the owner-controlled repository. Use Node 24 LTS, npm ci and Python 3.11+ for deployment tests. Read the overview and documentation index. Use the URL printed by npm run dev for static development, or start the core for real local HTTP/auth/ledger/AI flows. HTTP regression tests need local loopback permission.
Create a focused branch. Record acceptance criteria and configuration/data owners before implementation. Later features ship UI, API, data, security, tests and docs together. Verify new packages on their real registry and review licenses/advisories.

Gates

npm test
npm run typecheck
npm run lint
npm run format:check
python3 -m unittest discover -s tests -p 'test_*.py'
npm run build
Install pre-commit, Gitleaks, Bandit and Semgrep from official distributions, then:
pre-commit install
pre-commit run --all-files
gitleaks git --redact --no-banner
Hooks use system-installed staged-diff Gitleaks, Bandit for Python and Semgrep security-audit rules. Preserve edit scanners; fix findings without suppression/bypass. Report missing gates. Tracked automated tests live in tests/; personal/manual tools stay ignored and are improved in place rather than deleted.

Exercise affected real UI/HTTP flows, keyboard/focus and responsive states. Backend features require negative authorization, retry and duplicate tests. Obtain fresh-context review, then report criteria/gates/behavior/verdict. Document escaped defects privately.

Configuration and secrets

Use src/config.ts, src/data/ and deploy/settings.json for public values. Backend variables enter backend/settings.ts and .env.example; policy, prompts and CRM mapping data live in backend/data/. Keep actual credentials, private keys and customer records out of source, fixtures and public docs. Verify exclusions before recording access details. Review history as well as staging. An exposed credential requires revocation/rotation; deleting text alone does not revoke it. GitHub sensitive-data guidance.

Safe Git delivery

Verify authenticated account and approved remote. Use the owner's approved name/email; commit attribution and a verified-signature badge are distinct. Preserve visibility, branch protection and CI settings. Review the diff and stage explicit public paths. Run hooks, fetch/reconcile remote changes, push normally, verify branch SHA equality and record remaining local work. Never force-push routinely or enable Actions as a side effect. Current Actions remain disabled pending billing/execution approval.

Documentation and release

Update affected guides and phase status when exit gates pass. Preserve scoped historical evidence. Public docs exclude private infrastructure/access details. Existing Google Docs are edited in place when requested; validated PDF replacement follows successful updates. Production changes require drift snapshot, local edits, local gates, reviewed artifact deployment, real smoke flows and hash/digest parity. Source delivery and server promotion are separate. See operations.

Workspace and onboarding checklist

Source guide: docs/SETUP-CHECKLIST.md
Updated 2026-10-12. The repository is initialized, source is pushed and the static frontend is published. Earlier read-only metadata/network restrictions were resolved; they are historical, not current blockers. See publication and current documentation verification.

Clean checkout

- Obtain approved access to the owner-controlled private repository.
- Use Node 24 LTS/npm and Python 3.11+; run npm ci.
- Read the overview, frontend guide and

backend runtime. Static development works without the core; the runtime guide starts the connected local engine.

- Run local test/type/lint/format/build and Python gates in CONTRIBUTING.md.
- Install system Gitleaks/Bandit/Semgrep and pre-commit; run pre-commit install and exercise the hooks. Verify actual execution, not only configuration presence.
- Run npm run dev and use its printed URL; verify affected real browser flows.

Privacy and configuration preflight

Before recording private access, verify actual exclusions:
git check-ignore CREDENTIALS.md PROJECT-DOSSIER.md memory/ .env manual-tests/
git status --short
Private recovery, narrative and memory belong to this project and stay ignored. Public documents derive from verified project facts and exclude sensitive details. Current public values belong to src/config.ts, src/data/ and deploy/settings.json. The frontend reads no application env variables. Backend runtime variables enter backend/settings.ts and .env.example. Keep actual secrets outside source.

Verified safeguards and remaining gates

- Git; Current state: Initialized normally; owner attribution and remote verified
- Private exclusions; Current state: Verified against actual Git before credential synchronization
- Security pipeline; Current state: Local edit/commit scanners exercised; findings fixed without bypass
- Frontend; Current state: Fifteen routes and static HTTPS artifact verified
- Recovery; Current state: Previous release retained and archive/file parity verified
- Remote CI; Current state: Workflow prepared; Actions disabled pending billing/execution approval
- Local core; Current state: Implemented; gates/evidence in CORE-VERIFICATION.md
- Provider activation; Current state: Native connectors tested with fixtures; company access/mappings remain
- Legal/scale/uptime; Current state: Approval, benchmarks and operating evidence remain

The source package includes both frontend and backend, their automated tests and configuration examples. Local acceptance is in core verification; company-account and hosted-service release gates are in production readiness. Private runtime state is never part of a clean checkout.
Temporary developer tooling may need reinstalling after reboot. Do not assume a machine-specific tool path exists in a clean checkout. No global instruction files are modified by onboarding. No purchase or cloud provisioning follows from setup.

References

Git ignore verification, pre-commit installation, documentation index, operations.

AdRevora documentation

Source guide: docs/README.md
Updated 2026-10-12. The technical overview introduces the complete product. The build plan owns phase scope/effort; specialist guides define implementation and release contracts.
The latest source handover includes frontend and executable backend. Start with production readiness for implemented capabilities, account activation, security/legal release gates, 1M+ scaling and 99.9% service objectives.

Product and implementation

For the executable local engine, start with runtime, account activation, implementation criteria and current core evidence. Other verification records retain their original static-release scope.

- Technical overview; Scope: Product, users, delivery status, architecture, workflows and handover
- Production readiness; Scope: Current acceptance, future release stages, accountable owners and evidence
- Ten-phase build plan; Scope: Dependencies, twenty architecture artifacts, estimates and exit gates
- Frontend guide; Scope: Routes, components, data/configuration and backend connection points
- Design; Scope: Visual system, responsive and accessible interaction
- Backend guide; Scope: Implemented local API/modules and proposed production architecture
- Integrations and data; Scope: Meta/GHL/Google, Pixel/CAPI, schema, identity and financial attribution
- AI and governance; Scope: Evidence, agents, approval, spending and ownership

Release and maintenance

- Compliance; Scope: US/EU applicability, privacy, regulated expansion and accountable owners
- Security policy; Scope: Current protections, planned controls and private reporting
- Capacity and reliability; Scope: Million-plus workloads, SLOs, benchmarks, recovery and cost
- Operations; Scope: Monitoring, release/rollback, incidents, retention and handover
- Deployment; Scope: Renderer usage and settings boundaries
- Static runbook; Scope: Current HTTPS publication and hash parity
- Contributing; Scope: Clean checkout, tests, review and safe Git delivery
- Setup; Scope: Current prerequisites and resolved historical restrictions

Acceptance and evidence

- Documentation specification; Scope: Criteria for this handover
- Documentation verification; Scope: Current gates, review and source delivery

- Hero revision; Scope: Autoplay/artwork acceptance and seven-width live checks
- Publication specification; Scope: Static publication criteria
- Publication verification; Scope: HTTPS, isolation, recovery, parity and source
- Website specification; Scope: Multi-page requirements
- Website verification; Scope: Historical multi-page pre-publication results
- Frontend specification; Scope: Original frontend criteria
- Frontend verification; Scope: Historical initial frontend results

Historical records retain their original scope. Current status is in the root README and current verification. Documentation changes do not change the deployed artifact.

Owners and synchronization

Every feature updates its guide and acceptance evidence. Technical lead owns contracts, integration lead event mappings, finance owner money definitions, privacy/security owners their controls, SRE capacity/operations and product owner phase acceptance. Public docs exclude credentials, real lead records and private infrastructure details. Existing Google Docs retain their IDs; explicitly requested updates preserve content and refresh a validated PDF in the project's local project-view folder.

A comprehensive Google Doc and PDF mirror were first prepared after an earlier verified GitHub handover. Current synchronization is recorded in the delivery verification; updates keep the same document ID, preserve prior material and put current developments first. Generated PDF/media mirrors remain ignored. Replace the canonical local PDF only after a fresh export passes content and layout checks.

Previous documentation baseline — preserved historical material

The following earlier handover is retained in full for continuity. Its implementation-status statements describe that earlier baseline; the current edition and source guides above govern the latest local capabilities and future release gates.
Updated 2026-10-12. This single-document handover covers current frontend delivery and the planned connected platform, architecture, integrations, security, compliance, scale, reliability and operations.
Source repository: https://github.com/Zahidulislam2222/AdRevora-AI
Published website: https://adrevora.zahidul-islam.com
Source documentation baseline verified: e6506ae44cf403bc02d8afa0e58be06812d7c19a.
Repository remains private; this overview is prepared for controlled sharing.

AdRevora AI — technical overview

Updated 2026-10-12. Product brand: **AdRevora AI**. Platform: **AdRevora OS**.

Purpose and initial business outcome

AdRevora combines campaign economics with lead quality, CRM stages, appointments, verified sales, collected revenue and known costs. It sits above supported advertising APIs and preserves a provider-independent financial and attribution model.
The first connected journey is Meta ad → webinar registration → lead → GHL contact → Platinum appointment → completed consultation → enrollment → verified revenue → AI recommendation. The supplied offer price is $5,000; payments, discounts, refunds and chargebacks determine actual money. Primary KPI: cost per Platinum enrollment. Secondary KPI: revenue per lead.
Later funnels cover real estate listings/closed transactions, property acquisition and profit, paid insurance policies and active producers. Definitions, regulated scope and provider permissions are reviewed separately for every expansion.

Implemented scope

The published frontend has fifteen routes: home, Platform, Solutions, Integrations, Security, Resources, About, Contact, Privacy, Terms, three resource articles, AdRevora OS and a not-found entry. React/TypeScript/Vite emit per-route HTML; Motion and Lenis support transitions/scrolling with reactive reduced-motion behavior. Fonts, artwork and the automatic muted film are self-hosted.
Workspace filtering, campaign search, evidence, approvals, creative drafts, activity and CSV operate on reference records in browser memory. Integrations are disconnected. Contact

preparation validates and downloads a local brief; no message is transmitted. No backend API, login, database or payment connection is running in this release.
A versioned static container behind HTTPS has verified isolation, headers, route behavior, restoration and file parity. Historical release gates include fifteen JS and four Python tests, fifteen live routes and seven hero viewport widths. See publication, hero verification and current documentation verification.

Users and authority

- **Marketing Director**; Primary needs: Winners/losers, creative fatigue, tests, funnel economics; Decision boundary: Approved advertising changes within company policy
- **COO**; Primary needs: Uncontacted leads, follow-up, appointments, CRM and technical bottlenecks; Decision boundary: Workflow/routing and operational changes
- **CEO**; Primary needs: Spend, sales, revenue, profit, allocation and major decisions; Decision boundary: Major budget, offers, pricing, markets and strategy
- **Analyst**; Primary needs: Scoped exploration, evidence and reporting; Decision boundary: No financial/provider execution
- **Administrator**; Primary needs: Membership, security and connection administration; Decision boundary: Human-controlled account management

Executive reporting does not imply permission to change provider ownership or account security. Authorization is tenant/business-unit scoped and enforced by the future API.

Architecture and stack

Reuse the current frontend. Proposed backend: modular TypeScript/NestJS API plus independent workers, PostgreSQL for transactions/ledger/identity/policy, a durable queue and transactional outbox, Redis-compatible cache, encrypted object storage, scoped analytical read models and an OIDC identity service. Exact versions/providers/regions are approved in phase 2. Next.js is an optional architecture decision, not a required rewrite. Add analytical storage or services when measured workloads justify them.
Architecture relationships (the repository includes the rendered diagram):

- Dashboard users → CDN and WAF
- CDN and WAF → React frontend
- React frontend → Tenant-authorized API
- Approved websites → Consent and event capture
- Consent and event capture → Regional ingestion
- Meta / GHL / later Google ↔ Provider adapters
- Provider adapters → Regional ingestion
- Regional ingestion → Durable queue and outbox
- Durable queue and outbox → Identity / funnel / finance workers
- Identity / funnel / finance workers → PostgreSQL source of truth
- PostgreSQL source of truth → Scoped analytical read models
- Tenant-authorized API → PostgreSQL source of truth
- Tenant-authorized API → Scoped analytical read models

- Scoped analytical read models → Evidence-bound AI workflows
- Evidence-bound AI workflows → Human approval and policy engine
- Human approval and policy engine → Restricted executor in later phases
- Restricted executor in later phases → Provider adapters
- Durable queue and outbox → Consent-checked conversion delivery
- Consent-checked conversion delivery → Meta / GHL / later Google

Interactive APIs, ingestion, provider sync, analytical reporting and AI have independent scaling/rate budgets. Dashboard refreshes read local aggregates rather than causing provider/LLM calls. Current static hosting and future regional services have separate capacity envelopes.

Tracking, identity and revenue

Preserve external IDs, historical names, UTMs and permitted click identifiers at capture. Keep contact links, appointment transitions, enrollment evidence, payments and refunds distinct. A won opportunity does not establish collected revenue. Metrics expose currency, cohort/date basis, model/window, data completeness and freshness.

Browser/server copies share a durable conversion ID for the same occurrence. Retries reuse that identity; separate conversions get new IDs. Receipt deduplication, domain-event idempotency and financial-posting idempotency protect different layers. Source counts and provider attribution counts remain separate.

First-touch and last eligible paid-touch views begin the internal attribution model. Unknown/partial matches remain visible. Multi-touch credit never exceeds one sale; network claims are not summed as separate revenue. Incrementality needs experiments. See integrations and data.

AI, security and compliance

The Director coordinates strategy, creative, campaign drafts, analysis, budget recommendations, retargeting and attribution. These are logical workflows, not a requirement for eight paid model sessions. Deterministic tools calculate money; AI explains evidence and drafts proposals. Recommendations use SCALE, MAINTAIN, TEST, FIX or PAUSE with freshness, limitations and expiration.

Publication, pausing, budgets and creative changes require human approval in the initial platform. Later bounded execution uses exact payload approval, live permission checks, budget reservations, idempotency, reconciliation and a kill switch. Ownership, payment methods and security stay human-controlled. AI/governance.

Required security includes tenant isolation, least-privilege OAuth, MFA, secret storage, verified ingestion, safe exports, audit and independent assessment. Secrets and raw lead details stay out of prompts/logs. US/EU readiness uses an applicability register, privacy/consent/rights/retention/transfer controls and sector-specific review before relevant processing. Security and compliance define gates.

Scale and availability

The target covers **more than one million simultaneous dashboard users AND more than one million simultaneous tracked visitors**, independently measured. Request/event rates, regions, payloads, tenant skew, retention and peak duration define capacity. Incremental load/failure tests establish the accepted envelope.
The target is **99.9% monthly availability** for defined critical console/API and event acceptance journeys. Routine releases target no user-visible interruption with compatible migrations, canaries, draining and rollback. Recovery, latency, freshness, errors and cost have measurable contracts in capacity/reliability.

Ten-phase delivery and architecture approval

1. Frontend design and implementation, with explicit design acceptance.
2. Architecture approval, authentication, tenancy and secure foundation.
3. Meta/GHL read integrations and reconciled reporting.
4. Tracking audit, consent-aware Pixel/CAPI and diagnostics.
5. CRM journey, revenue ledger and attribution.
6. AI analysis, creative/draft workflows, approvals and reporting.
7. Initial connected production pilot, security and operational acceptance.
8. Google/GA4, additional businesses and SaaS expansion.
9. Million-plus capacity proof, resilience and controlled execution.
10. Final acceptance, company handover and ongoing operations.

The initial connected MVP spans phases 1–7. Every later feature delivers frontend, backend, data, tests, security and documentation together. The build plan owns detailed dependencies and exit gates; frontend publication does not close other gates.
Approve twenty architecture items before integration development: diagrams, stack, Meta, Pixel, CAPI, GHL, schema, attribution, event map, AI, security, permissions, screens, scope, timeline, development cost, monthly cost, testing, deployment and docs. Draft coverage exists; approved migrations and live provider permission evidence follow in their implementation phases.

Estimates, ownership and handover

Baseline planning: 800–1,300 engineering hours for phases 1–7; 1,880–3,580 for all ten. At the assumed $60–$120 blended rate, arithmetic is $48,000–$156,000 and $112,800–$429,600 before contingency/external services. Suitable-team calendar allowances are 12–20 weeks for the initial connected MVP and 9–18+ months overall, subject to discovery, reviews and measured scope. These are planning assumptions.
Pilot allowances and the large-scale cost model are in the roadmap. Million-user pricing requires measured demand and current vendor calculations; no purchase or paid execution follows from this document.
Company-controlled source/accounts/assets, API/schema/configuration guides, release manifests, recovery drills, dashboards, user/operator instructions and named maintenance owners form the handover. Start with contributing and operations. Public material excludes

recovery and private incident records. The supplied brief is a requirements reference; distribution/brand rights need owner review.

AdRevora AI ten phase build plan

AdRevora AI is the proposed product brand; AdRevora OS is the proposed platform name. Build one advertising intelligence platform connecting spend to leads, appointments, customers, revenue, and profit. Begin with frontend design, then complete frontend, backend, data, infrastructure, tests, and documentation together for each feature.
Status updated 2026-10-12: frontend source and static HTTPS publication exist. Phase 1 design acceptance remains open; phases 2–10 and their exit gates remain planned. Connected backend/provider workflows, legal approval, capacity proof and measured availability are pending. Brand availability is unverified. Start with the technical overview and documentation index.

Planning acceptance criteria

These criteria precede application implementation. The plan must:

1. Define exactly ten ordered phases, dependencies, deliverables, and exit gates.
2. Put frontend design first and combine the implementation disciplines afterward.
3. Deliver the Bootcamps Meta/GHL revenue funnel before expanding the product.
4. Specify architecture, stack, schema, tracking, attribution, AI, and permissions.
5. Include US/EU applicability review, privacy controls, security, and ownership.
6. Define separate million-user dashboard and tracking workloads and measurable SLOs.
7. Include GitHub, CI/CD, staged deployment, rollback, and local/live parity.
8. Give explicit estimate assumptions, cost controls, dependencies, and risks.
9. Define documentation and handover understandable before every feature is finished.
10. Distinguish proposals from measured results and identify current setup blockers.

Scope and release boundaries

The initial funnel is Meta ad → free webinar/event → lead → GHL contact → Platinum appointment → completed consultation → verified Platinum enrollment → revenue → AI recommendation. Its primary KPI is cost per Platinum enrollment; secondary KPI is revenue per lead. The supplied offer price is $5,000; actual collected revenue, discounts, payment plans, refunds, and chargebacks determine financial reporting.
The source brief calls this the client's "Phase 1 MVP." In this ten-phase program, that MVP spans **Phases 1–7**. Phase 1 below means frontend design only. The first technical revenue website implementation is in Phase 5; AI analysis completes it in Phase 6; the controlled production pilot is in Phase 7.

- **Initial MVP**; Included: Bootcamps, Meta, Pixel/CAPI, GHL, verified revenue, recommendations, creative drafts, daily reports, approvals and audit; Deferred: Google, other business units, autonomous changes, paid SaaS subscriptions

- **Expansion**; Included: Google Ads/GA4, four-company reporting, additional business funnels, SaaS onboarding; Deferred: TikTok/LinkedIn connectors until separately prioritized
- **Scale release**; Included: Measured regional capacity, disaster recovery, limited explicitly approved automation; Deferred: Unrestricted AI spending, guarantees of attribution or uptime

All production assets belong to the company: repositories, domains, cloud accounts, Meta Business assets, datasets, GHL installations, database, AI workflows, and creative library. Company administrators grant scoped developer access. No personal developer account owns a critical production asset. No existing tracking asset is replaced or deleted without an approved audit/change plan.

The ten phases

The repository/scanner preflight is now active and static publication is verified; see the setup checklist. Keep those gates green throughout the program. Phase 1 frontend delivery exists; owner acceptance of the complete planned screen/behavior inventory remains open. Phase 2 extends the working repository into the backend/delivery foundation after architecture approval.

- **1**; Outcome: Frontend design and complete clickable product interface; Dependency: Supplied brief and example data; Effort estimate: 60–100 hours
- **2**; Outcome: Approved architecture and secure executable foundation; Dependency: Design review and repository access; Effort estimate: 100–160 hours
- **3**; Outcome: Meta/GHL connections and reconciled campaign reporting; Dependency: Foundation and company-owned provider access; Effort estimate: 100–160 hours
- **4**; Outcome: Audited consent-aware Pixel/CAPI tracking; Dependency: Approved tracking/privacy architecture; Effort estimate: 140–220 hours
- **5**; Outcome: CRM funnel, revenue ledger, and explainable attribution; Dependency: Event and CRM ingestion; Effort estimate: 140–220 hours
- **6**; Outcome: Evidence-based AI recommendations, creative drafts, approvals; Dependency: Trusted funnel metrics; Effort estimate: 140–240 hours
- **7**; Outcome: Bootcamps MVP security validation and production pilot; Dependency: Complete MVP acceptance and cost approval; Effort estimate: 120–200 hours
- **8**; Outcome: Google and four-business SaaS expansion; Dependency: Stable pilot and expansion approval; Effort estimate: 360–640 hours
- **9**; Outcome: Proven capacity, resilience, and controlled automation; Dependency: Measured workload, funded tests, operating team; Effort estimate: 600–1,400 hours
- **10**; Outcome: Final release, company handover, and ongoing operations; Dependency: All contractual gates accepted; Effort estimate: 120–240 hours

Estimates are engineering planning assumptions, not quotations or guaranteed dates. Phase 9 has the greatest uncertainty; benchmarking can require a revised scope and budget. Security and reliability start in Phase 2 and run through every phase.

Phase 1 Frontend design and product interface

Goal: settle the product experience before connecting real accounts or building backend workflows. A minimum navigation/data contract prevents design rework; the formal technical architecture package and approval remain in Phase 2.

Design and frontend deliverables

- Brand tokens, typography, colors, spacing, reusable components, chart conventions, responsive layouts, focus states, and accessible contrast.
- Product shell with company, platform, date, currency, and attribution-model filters.
- CEO overview, marketing workspace, COO operations, campaign/ad/creative drilldown, lead journey, appointments, revenue, tracking health, AI recommendations, creative library, campaign drafts, approval queue, reports, integrations, audit, and settings.
- Design SaaS onboarding, tenant/team roles, consent/privacy controls, and Google views now; mark future functions as planned. Use example data and visibly label workspace mode.
- Show loading, empty, error, partial attribution, stale sync, disconnected provider, insufficient evidence, permission denied, and pending-approval states.
- Provide a clickable scenario from campaign to enrollment and recommendation, with working navigation and filters. No local interaction implies a real account change.
- Store example fixtures and maintained UI copy in validated data files. Define component inputs and draft API shapes; no provider credentials or paid AI calls.

Verification and exit gate: owner reviews all screens and the Bootcamps journey; keyboard and responsive checks pass; the design targets WCAG 2.2 AA; fixture totals reconcile; role and approval states are visible. Record screenshots, screen inventory, design decisions, component catalog, and acceptance findings. Design approval closes this phase; backend implementation follows. WCAG 2.2

Phase 2 Architecture approval and secure foundation

Goal: approve the full architecture before substantial integration development, then make a small secure application run locally with the approved design shell.

Before feature implementation: deliver all twenty architecture items mapped later in this plan. Confirm legal scope, company ownership, threat model, initial workload, cloud/account choice, provider access strategy, and cost assumptions. Record approval with scope/version; unresolved safety or financial blockers cannot be assumed approved.

Coordinated implementation

- Create the monorepo, typed configuration boundary, validated content/config schemas, dependency locks, environment example, API contracts, and database migrations.
- Implement tenant onboarding, authentication, MFA policy, scoped roles, session revocation, server authorization, tenant-scoped database access, and audit records.
- Connect the real frontend shell to session/tenant APIs with permission/error states.
- Separate web/API, tracking ingestion, provider workers, reporting, and AI execution boundaries. Use local fake connectors; test that the UI never receives provider tokens.
- Prepare containers, infrastructure definitions, telemetry, health/readiness checks, and local development services. Provision hosted resources only after cost approval.

- Verify the repository/hook preflight, retain the owner-controlled private GitHub repository, and push the reviewed foundation. Check Actions billing before enabling workflows; keep optional paid AI review commented out.

Verification and exit gate: real login/tenant-switch/revocation flows pass; cross-tenant reads/writes are rejected; tests, types, lint, security, and build gates pass; configuration inventory has owners; architecture approval is recorded. A developer follows setup instructions from a clean checkout. No backend work proceeds without verifying the installed hooks and current gates remain active.

Phase 3 Meta and GHL connections with trusted reporting

Goal: replace workspace campaign data with normalized, reconciled provider data.

Coordinated implementation

- Build company-owned Meta OAuth/account selection and GHL installation flows, with encrypted tokens, state validation, scope checks, revocation, and safe refresh locking.
- Before live Meta calls, verify current supported API versions, required permissions, business verification, app review/access levels, and approved test accounts. Record exact findings; an SDK version alone does not prove account permissions.
- Read Meta campaigns, ad sets, ads, creatives, leads where authorized, and Insights. Persist external IDs, historical names, account timezone/currency, source windows, sync checkpoints, pagination, and query versions. Preserve raw evidence under policy.
- Read relevant GHL contacts, opportunities, calendars/appointments, and configured funnel fields. Define explicit source owners for enrollment and payment evidence.
- Implement authenticated webhook receivers, durable queues, idempotency, provider rate-limit handling, backoff, dead-letter review, backfill, and reconciliation jobs.
- Connect dashboards and integration health UI to real read models. Display freshness, partial sync, scope failures, and provider-versus-internal metric labels.

Verification and exit gate: a bounded account/date sample reconciles to provider reports after documenting timezones, currency, definitions, and reporting lag. Token expiry/revocation, duplicate webhooks, pagination, retries, and rate limits have real-flow or documented sandbox evidence. Unauthorized scopes/tenants fail closed. No publishing, pausing, budget adjustment, or CRM routing changes occur.

GHL documents scoped OAuth installation and webhook integration. Verify endpoint support and signature requirements again when implementing. GHL OAuth, GHL webhooks

Phase 4 Pixel and CAPI with consent and diagnostics

Goal: prove the tracking architecture before trusting conversion counts.

Audit before changes: inventory accessible company datasets/pixels, websites, funnels, tag containers, plugins, CRM automations, server event senders, ownership, domains, consent behavior, payloads, and diagnostics. Find overlapping triggers and legacy integrations; retain evidence and obtain approval for migration/removal.

Coordinated implementation

- Approve the business event dictionary and provider mapping. Reuse suitable existing datasets; create assets only where the audit justifies them under company ownership.

- Implement browser tags only on approved public funnels, with consent controls. Do not put ad tracking on the authenticated AdRevora console by default.
- Generate one durable conversion identity per real occurrence; pass it through the browser and backend where both represent that occurrence. Reuse it across retries.
- Validate event time, event name, action source, approved source URL, value/currency, and permitted matching fields. Never fabricate click identifiers or attendance.
- Implement a durable CAPI outbox with consent/purpose checks both when queued and immediately before sending. Withdrawal cancels pending marketing delivery.
- Enforce field allowlists and normalization/hashing only as the provider requires. Hashing is not anonymization or permission to send insurance/financial details.
- Add tracking-health screens for event source, deduplication, freshness, rejection, consent suppression, match quality, and retry status, with sanitized diagnostics.

Verification and exit gate: use Meta Test Events/diagnostics on approved assets; show one logical conversion from browser+server, stable identity after retry, and separate identity for the next conversion. Test consent denied/withdrawn, duplicate tags, invalid fields, delayed events, and CRM-only conversions. Verify reception, not merely an HTTP success response. Record before/after audit and rollback plan.

Meta's official SDK identifies `event_id` with `event_name` as event identity inputs. Its current documentation and actual dataset tests must validate the complete deduplication behavior.

Meta SDK event identity

Phase 5 CRM journey and revenue attribution

Goal: demonstrate the complete ad-to-revenue chain with reproducible accounting.

Coordinated implementation

- Persist touchpoints and lead-form identifiers at capture; resolve identity through tenant-scoped contact/link tables, explicit confidence, and correction history.
- Model appointment bookings, reschedules, cancellations, no-shows, completed consultations, enrollments, invoices, payments, refunds, and chargebacks separately.
- Build payment/revenue reconciliation against the approved source. A GHL "won" opportunity is expected revenue until actual sale/payment evidence supports it.
- Implement first-touch and last eligible paid-touch attribution with configurable lookback rules; add a descriptive multi-touch view without claiming causation.
- Build the lead timeline, evidence drawer, funnel conversion view, revenue ledger, reconciliation exceptions, cost per enrollment, revenue per lead, ROAS, and profit UI.
- Support duplicate leads, merged contacts, multiple purchases, late webhooks, missing IDs, organic/direct touches, cross-device gaps, and corrected financial events.

Verification and exit gate: trace an authorized test/sandbox $5,000 enrollment through the whole chain without charging a real customer. Replay the input and show one enrollment and one revenue posting. Demonstrate an unmatched sale, a multi-touch journey, a refund, and a contact merge. Reconcile source totals and explain attribution coverage. No touchpoint is invented to make dashboard numbers look complete.

Phase 6 AI intelligence and approvals

Goal: produce useful recommendations from verified metrics without giving the language model independent authority over accounts or money.

Coordinated implementation

- Implement the eight logical AI roles described later as bounded workflows sharing versioned evidence. Start with deterministic orchestration; no agent swarm is required.
- Calculate financial metrics in tested code. Feed minimized aggregates and approved content to a provider-neutral LLM gateway. Model/URL/limits/pricing come from config.
- Generate campaign briefs, copy/concepts/scripts, daily CEO/COO/marketing reports, fatigue hypotheses, funnel diagnoses, and evidence-linked recommendations.
- Assign SCALE / MAINTAIN / TEST / FIX / PAUSE using configurable evidence thresholds. Include sample size, lag, uncertainty, freshness, and alternative explanations.
- Implement creative history, approved claims/disclosures, content policy checks, campaign draft editor, approval queue, and immutable decision/execution history.
- Keep MVP campaign drafts inside AdRevora. Optional creation of paused Meta objects needs separately verified API behavior and explicit approval; “paused” is not a guaranteed provider-native draft and must never imply authorization to activate.
- Enforce token/call quotas and per-tenant/global AI cost ceilings outside prompts; fall back to deterministic reports when the provider fails or the quota is reached.

Verification and exit gate: benchmark recommendations against curated cases, including expensive leads with stronger downstream sales, insufficient samples, sales-process failures, stale data, and adversarial CRM text. Reject unauthorized, expired, repeated, or changed-payload actions in backend tests. No prompt can override authority. Any live paid AI evaluation requires a separate verified-cost approval.

Phase 7 Bootcamps production pilot

Goal: launch the narrowly scoped MVP after legal, security, and operational gates.

Coordinated implementation

- Complete all MVP UI states and end-to-end journeys; close escaped defects in the defect log and rerun the gates affected by their fixes.
- Review privacy notices, legal applicability, data processing agreements, consent, retention/deletion, ownership, ad claims, accessibility, and incident obligations.
- Perform independent security review and penetration testing; resolve exploitable critical/high findings before launch. Verify tenant isolation and credential recovery.
- Run agreed pilot load/soak tests, database restore, worker replay, provider outage, rollback, and availability-alert exercises. Report tested limits, not future capacity.
- Deploy immutable artifacts to company-owned staging, then an approved production canary with real monitoring. Infrastructure and test spending require prior approval.
- Operate the pilot for at least 30 days, reconcile actual funnel data, and measure availability, freshness, attribution coverage, cost, and user acceptance.

Verification and exit gate: complete the first meaningful AI website implementation with verifiable events and revenue; owner accepts the MVP; security/legal reviewers sign their

scoped findings; restore/rollback succeed; measured pilot results meet the agreed SLOs. A 30-day result is evidence for that window, not a perpetual guarantee. Advertising changes remain human-controlled.

Phase 8 Google and multiple business SaaS expansion

Goal: expand through the existing canonical model and preserve tenant isolation.
Coordinated implementation

- Add company-owned Google Ads and GA4 connections with current approved access, consent settings, conversion mapping, identifiers, reconciliation, and diagnostics.
- Choose the currently supported offline conversion ingestion route for the specific Google account/app; verify Data Manager API eligibility rather than copying a historical upload endpoint. GA4 Measurement Protocol supplements tracking and requires its own validation; a successful HTTP response is not sufficient proof.
- Add the four-company KPI matrix and drilldowns by provider, campaign, creative, audience where actually available, market, offer, date, and attribution model.
- Onboard Realty seller/listing/closed-transaction, Properties acquisition/profit, and BigLife consumer/recruiting funnels one at a time with signed event dictionaries.
- Build SaaS signup/invitation, tenant admin, entitlements, usage metering, quotas, export/offboarding, support access, and optional subscription billing. Use test mode first; live billing and subscriptions require explicit commercial/cost approval.
- Add configurable stage exclusions to retargeting recommendations; avoid advertising registration to an attendee. Audience sync writes need scoped approval and policy review.
- Test with multiple example tenants and isolated real pilot companies; verify no cross-company joins, audience contamination, cache leaks, or double-counted sales.

Verification and exit gate: reconcile each connector and business funnel; a sale appears once in cross-platform internal totals; security/legal review covers new jurisdictions and regulated uses; updated UI and API docs describe provider limitations. Do not add new networks merely because the adapter interface exists.
Google's current conversion guide contains access restrictions affecting legacy uploads, so implementation must select the supported route at that time. Google offline conversions, GA4 Measurement Protocol

Phase 9 Capacity proof resilience and controlled automation

Goal: establish measured capacity for both requested million-user workloads and introduce limited automation only when governance and evidence permit it.
Coordinated implementation

- Validate the workload contract below; benchmark 1k, 10k, 100k, then 1M+ concurrent sessions/visitors. Use open-loop request/event rates as well as session counts; small tests or extrapolation cannot substitute for the agreed final test.
- Add regional tenant cells, partitioned ingestion/analytics, precomputed reports, independent worker pools, connection pooling, cache isolation, admission control, fair per-tenant quotas, and protection from noisy tenants. Size from measurements.

- Plan US/EU data placement and recovery so disaster routing does not silently move personal data across an unapproved jurisdiction. Exercise capacity after losing an AZ.
- Demonstrate failover/restore, replay, backlog drain, canary rollback, and privacy deletion under load. Trace acknowledged events through durable storage and recovery.
- Provide dashboards for traffic, latency, errors, saturation, queue age, tenant cost, attribution coverage, provider lag, budget anomalies, and burn-rate alerts.
- Implement narrow opt-in automation: approved creative rotation/test variants, bounded pausing, or budget changes within signed policies. Shadow mode precedes canary mode.
- Use a separate executor with capability allowlists, live-state checks, atomic budget reservations, approval expiry, idempotency, kill switch, and before/after audit.
- Document compensating actions. Spent money, delivered impressions, and third-party side effects cannot be undone; restoration of settings is not financial reversal.

Verification and exit gate: independent performance/security review accepts measured tests at the exact contracted workload, geography, data size, failure mode, and latency/error targets. Include test costs, bottlenecks, safety margin, and largest verified capacity. If 1M+ is not exercised successfully, label it unverified and do not close that criterion. No uncontrolled external provider/AI calls in a load generator.

Phase 10 Final release handover and operations

Goal: leave a reproducible company-owned product that another developer can run, explain, deploy, recover, and maintain.

Coordinated implementation

- Close contractual acceptance items; finish UI polish, operational screens, migration tooling, support/offboarding, and documentation against the shipped release.
- Push reviewed source, migrations, infrastructure definitions, CI, lockfiles, and public-safe docs to the company repository; tag the approved immutable release.
- Verify secret/private-record exclusion, branch protections, dependency/license inventory, artifact signatures/SBOM, vulnerability gates, and company administrator access.
- Rehearse deployment and rollback from GitHub with company maintainers; compare local reviewed source with deployed artifacts, manifests, and served frontend build identity.
- Deploy gradually, verify parity and smoke checks, complete availability observation, publish measured limits, and obtain product/security/legal/operations acceptance.
- Transfer operational responsibility, train users, rotate temporary developer access, confirm recovery under company control, and schedule dependency/API deprecation, consent-policy, incident-response, restore, and capacity reviews.

Verification and exit gate: a developer who did not write the implementation can use a clean checkout and the docs to run it, trace a sample sale, deploy to staging, restore backup data, and roll back. Provide a signed gate matrix and unresolved-risk register. No feature or capacity criterion is described as delivered without evidence.

Proposed architecture and stack

This is the baseline for Phase 2 approval. Use a modular backend with independently deployable workers first; extract services where ownership, scaling, or isolation requires it. Separate ingestion and analytics from interactive transactions from the start. Framework choice alone does not establish scalability.

Architecture relationships (the repository includes the rendered diagram):

- Dashboard users → CDN and WAF
- CDN and WAF → Next.js console
- Next.js console → Authenticated tenant API
- Approved websites and funnels → Consent and browser events
- Consent and browser events → Regional event ingestion
- Meta accounts and datasets ↔ Provider adapters
- GoHighLevel ↔ Provider adapters
- Google Ads and GA4 later ↔ Provider adapters
- Provider adapters → Regional event ingestion
- Regional event ingestion → Durable queues and outbox
- Durable queues and outbox → Identity and funnel workers
- Identity and funnel workers → PostgreSQL transaction ledger
- Identity and funnel workers → Aggregates and analytics store
- PostgreSQL transaction ledger → Aggregates and analytics store
- Authenticated tenant API → PostgreSQL transaction ledger
- Authenticated tenant API → Aggregates and analytics store
- Aggregates and analytics store → AI Director and bounded workflows
- AI Director and bounded workflows → Recommendations and internal drafts
- Recommendations and internal drafts → Human approval and policy checks
- Human approval and policy checks → Restricted executor in later release
- Restricted executor in later release → Provider adapters
- Durable queues and outbox → Consent checked conversion delivery
- Consent checked conversion delivery → Meta accounts and datasets
- PostgreSQL transaction ledger → Audit and operational monitoring
- **Frontend**; Proposed choice: React/Next.js and TypeScript, reusable design tokens; Reason and constraint: Accessible dashboards, shared contracts, deployable containers; use caching correctly across replicas
- **Backend**; Proposed choice: TypeScript/NestJS modular API and workers; Reason and constraint: One primary language, explicit boundaries, typed validation; Python worker only if a measured need justifies another stack
- **Transaction data**; Proposed choice: Managed PostgreSQL with RLS and pooler; Reason and constraint: Reliable financial/identity transactions; tenant authorization also enforced in services
- **Analytics**; Proposed choice: SQL aggregates initially; ClickHouse or warehouse after benchmark; Reason and constraint: Prevent complex reporting scans from saturating the transactional database

- **Jobs**; Proposed choice: Durable managed queue with DLQ; transactional outbox; Reason and constraint: Retries/replay and isolation from slow providers; no claim of network-wide exactly-once delivery
- **Cache**; Proposed choice: Managed Redis-compatible cache; Reason and constraint: Scoped aggregates, rate limits, coordination; not the financial source of truth
- **Storage**; Proposed choice: Encrypted object storage, scoped signed downloads; Reason and constraint: Creative assets, short-retention approved raw evidence, export lifecycle
- **AI**; Proposed choice: Provider-neutral gateway with schema-validated workflow outputs; Reason and constraint: Replace models via config; minimize PII; enforce budget and action policy outside prompts
- **Identity**; Proposed choice: Company-owned OIDC provider with MFA; Reason and constraint: Avoid storing user passwords in application tables; evaluate license and regional fit
- **Hosting**; Proposed choice: Proposed AWS regional containers, managed DB/queue, CDN/WAF; Reason and constraint: Multi-AZ baseline; regional cells later; company approves provider and costs
- **Delivery**; Proposed choice: GitHub, containers, declarative infrastructure, staged promotion; Reason and constraint: Reviewed source to reproducible artifact; short-lived deployment credentials
- **Observability**; Proposed choice: OpenTelemetry plus metrics/logs/traces and on-call alerts; Reason and constraint: Correlate failures without putting tokens or CRM payloads in public logs

Exact versions, packages, cloud SKUs, regions, provider endpoints, and licenses are selected and verified in Phase 2. Check registries and security advisories before adding dependencies. Proposed products are not installed or purchased by this plan. Next.js deployment and caching, PostgreSQL row security, AWS reliability guidance

Configuration and maintained content ownership

- **Tokens, keys, encryption/signing material**; Single owner: Environment or company secret manager; private recovery register; Required verification: Ignore verified before writing; never in prompts/browser/source/logs
- **URLs, versions, hosts, regions, timeouts, limits, retries, model IDs**; Single owner: Central typed settings populated from environment; Required verification: Every variable in `.env.example`; validation at startup; no adapter fallback duplication
- **Prices, quotas, attribution windows, authority thresholds, feature flags**; Single owner: Versioned validated policy/config data; Required verification: Effective date and audit; provider changes need config edits only
- **Brand/copy, prompts, reports, funnel/event dictionaries, workspace data**; Single owner: Validated content/data/template packages; Required verification: No long maintained content embedded in control flow
- **Stable provider syntax**; Single owner: Named provider protocol modules; Required verification: One definition; do not confuse true syntax with changeable API versions

Canonical schema and event architecture

Logical tables below become migrations and an ER diagram in Phase 2. Every tenant record and foreign-key relationship carries tenant scope; provider identifiers are strings and uniqueness includes tenant/provider/account. Separate tenant from business unit so one company group can report across permitted units without crossing customers.

- **Access**; Core entities and important fields: tenants, business_units, users, memberships, roles, scoped_permissions, sessions
- **Connections**; Core entities and important fields: provider_connections, external_accounts, token_secret_refs, scopes, sync_cursors, webhook_receipts
- **Ads**; Core entities and important fields: campaigns, ad_sets, ads, creatives, offers, markets, daily_insights; external IDs and historical name versions
- **Tracking**; Core entities and important fields: events, event_deliveries, consent_receipts, touchpoints; event/received times, source, schema version, purpose, provenance
- **CRM**; Core entities and important fields: contacts, identity_links, contact_merges, leads, lead_forms, opportunities, pipeline_stage_history
- **Funnel**; Core entities and important fields: appointments, appointment_transitions, consultation_events, enrollments; source evidence and business definitions
- **Money**; Core entities and important fields: orders, invoices, payments, refunds, chargebacks, revenue_postings, cost_postings, fx_rates; currency, effective date, evidence
- **Attribution**; Core entities and important fields: attribution_models, attribution_runs, attribution_credits; model version, eligibility window, evidence, unknown reason
- **AI and action**; Core entities and important fields: analysis_runs, evidence_snapshots, recommendations, campaign_drafts, approvals, policies, budget_reservations, executions
- **Operations**; Core entities and important fields: outbox, job_runs, reconciliation_exceptions, audit_events, retention_tasks, deletion_requests

Use integer minor currency units or precise decimal arithmetic. Preserve original currency and versioned FX conversion; never sum unrelated currencies directly. Keep occurrence time and receipt time separate; provider reports retain account timezones. Aggregate and partition event tables by measured time/tenant workload; avoid a mandatory full-history scan per dashboard visit. Archive or delete raw data according to policy.

Initial Bootcamps event map

Names here are internal proposals. Standard/custom Meta mappings, permitted data, source values, time limits, and optimization eligibility need approval against the current API and account. Do not send every internal business event to an ad network.

- **Landing viewed**; Authoritative source: Approved landing page; Browser: Yes after consent; Backend or CRM: Optional receipt; External delivery decision: PageView where permitted
- **Event registered / lead accepted**; Authoritative source: Successful form or Meta Lead Ad; Browser: For website forms after success; Backend or CRM: Yes; preserve

lead-form ID; External delivery decision: Choose one approved Lead or registration mapping per occurrence
- **Event attended**; Authoritative source: Verified attendance system; Browser: Never inferred from a visit; Backend or CRM: Import/integration; External delivery decision: Custom mapping only if justified
- **Platinum appointment booked**; Authoritative source: GHL appointment record; Browser: Only if the same confirmed booking returns to browser; Backend or CRM: Yes; External delivery decision: Approved Schedule/custom mapping
- **Consultation completed**; Authoritative source: Defined CRM transition and operator evidence; Browser: No; Backend or CRM: Yes; External delivery decision: Internal first; optional allowed downstream mapping
- **Platinum enrolled**; Authoritative source: Verified enrollment record; Browser: Optional confirmation of that exact occurrence; Backend or CRM: Yes; External delivery decision: Internal business event; avoid duplicating payment Purchase
- **Payment collected**; Authoritative source: Approved payment/accounting source; Browser: Optional same confirmed payment; Backend or CRM: Yes; External delivery decision: Purchase if policy/consent/provider rules allow; actual value and currency
- **Refunded or charged back**; Authoritative source: Payment/accounting source; Browser: No; Backend or CRM: Yes, ledger adjustment; External delivery decision: Follow supported adjustment route; no invented negative Purchase

Pair browser and server copies only when they refer to the same occurrence in the same dataset with matching event identity. Use opaque IDs, not emails, as event IDs. Lead, booking, enrollment, and payment are different events. A replay uses the same identity; a new appointment/payment gets a new one. Deduplicate ingestion separately using provider receipt identity and logical event identity. Financial idempotency survives queue replay and worker crashes. Internal source-of-truth counts remain independent of provider attribution counts.

Attribution and financial definitions

Capture business unit, provider/account, campaign/ad set/ad/creative identifiers and historical names where available, UTMs, permitted click/browser identifiers, landing page, lead form, creation time, first/last touches, contact, appointment, opportunity, sale, and money evidence. Validate URL parameters; strip PII and sensitive query text from exported URLs. Names and UTMs alone do not prove an exact ad match.

Store immutable touchpoint evidence and version the attribution rules. Prefer explicit lead-form IDs and validated click/ad parameter links; use consented verified contact identity to connect later sales. Show deterministic match, partial match, or unknown. Do not fingerprint people or assume cross-device linkage. Never promise every sale can be attributed. Platform-reported modeled/view-through results are a separate view.

- **CPL**; Definition: Approved ad spend / distinct defined leads
- **Cost per appointment**; Definition: Spend / qualifying unique appointment occurrences, with cancellation policy shown
- **Cost per Platinum enrollment**; Definition: Spend / verified enrollments, with refund/cohort policy shown

- **CAC**; Definition: Agreed acquisition costs / new customers; show when only ad spend is included
- **Revenue per lead**; Definition: Agreed net collected or recognized revenue / cohort leads; basis labeled
- **ROAS**; Definition: Attributed revenue / ad spend, with revenue basis/model/window stated
- **Profit**; Definition: Revenue minus refunds, ad spend, fulfillment/COGS, fees, commissions, and other agreed costs
- **Profit per marketing dollar**; Definition: Agreed contribution profit / marketing cost; explicit definition
- **Attribution coverage**; Definition: Eligible revenue or sales with sufficient evidence / total eligible revenue or sales

Zero denominators return “not enough data,” not infinity. Missing cost inputs make profit unavailable or explicitly partial. Distinguish lead cohort from calendar-period reporting; late sales/restatements can change historical results. Refunds adjust the ledger without deleting the original sale.

Start with first-touch and last eligible paid-touch views. Add a transparent multi-touch comparison later, distributing at most 100% of a sale across eligible touches. Platform claims can overlap; never add Meta-attributed plus Google-attributed revenue as if they were distinct sales. Attribution describes observed credit; proving incremental lift requires separate experiments. Two enrollments are an example, not sufficient statistical proof for an automatic budget increase.

AI workflows and authority

- **Ads Director**; Evidence and output: Ranked opportunities, economics, executive decision brief; Authority: Recommend only
- **Campaign Strategist**; Evidence and output: Offer, audience policy, funnel, objective, KPI, campaign brief; Authority: Draft only
- **Creative Director**; Evidence and output: Approved claims, historical creative results, copy/concepts/scripts; Authority: Draft only
- **Campaign Builder**; Evidence and output: Approved brief and assets to internal draft; Authority: No publication in MVP
- **Performance Analyst**; Evidence and output: Reproducible full-funnel metrics and alternative diagnoses; Authority: Analyze only
- **Budget Optimizer**; Evidence and output: Evidence/lag thresholds and five-state recommendation; Authority: No direct budget access
- **Retargeting Manager**; Evidence and output: Stage-aware segment/exclusion proposals; Authority: Audience writes approved separately
- **Attribution Agent**; Evidence and output: Explain evidence and compute approved model through deterministic tools; Authority: Cannot invent identity or revenue

Every output includes tenant, evidence snapshot, as-of time, model/prompt/policy versions, rationale, uncertainty, and expiration. Treat CRM text and imported creative content as untrusted input. Keep secrets/PII out of prompts; disable cross-tenant retrieval and choose

retention/training terms through vendor review. Historical learning begins with versioned queries and evaluations, not automatic training on customer data.

Permissions matrix

- **Read scoped dashboards and reports**; Analyst / AI: Allowed by assigned scope; Marketing Director: Allowed; COO: Allowed; CEO / company admin: Allowed
- **Generate analysis, creative concepts, internal drafts**; Analyst / AI: Allowed; Marketing Director: Allowed; COO: Allowed; CEO / company admin: Allowed
- **Publish/pause ads, start tests, change creative or approved budget**; Analyst / AI: Propose only; Marketing Director: Approve within company policy; COO: Escalation when operations change; CEO / company admin: Major strategy/budget approval
- **Change CRM routing or sales process**; Analyst / AI: Propose only; Marketing Director: Request; COO: Required approval; CEO / company admin: Cross-company/major decisions
- **New offers, pricing, markets, significant vendor costs**; Analyst / AI: Propose only; Marketing Director: Request; COO: Request; CEO / company admin: Required approval
- **Ownership, payment methods, security, account users/partners**; Analyst / AI: No AI execution; Marketing Director: No delegated AI execution; COO: No delegated AI execution; CEO / company admin: Human administrator only
- **Privacy request handling and support access**; Analyst / AI: Assigned minimal permissions; Marketing Director: No implicit access; COO: Assigned process owner; CEO / company admin: Designated privacy/security owners

Scope every approval to tenant, business unit, account, action, exact payload hash, maximum exposure, policy version, expiration, and approver. Recheck live state and authorization at execution. Modified payloads invalidate approval; replay cannot execute twice. UI role hiding is not access control.

Future automated budgets require pessimistic reservations, shared account/campaign ceilings, pacing/headroom, reconciliation, and emergency stop controls. Account for platform pacing, reporting lag, in-flight requests, and manual changes. An app cannot promise an absolute spend cap it cannot enforce at the provider; use verified provider limits where supported and document residual exposure. Recommendation labels alone never authorize spending.

US and EU legal and privacy work

The launch gate is a documented applicability assessment by qualified US/EU advisers, followed by implemented controls and evidence. “Follows every US/EU law” is not a testable software specification. Identify company establishments, user/lead locations, target markets, regulated activities, controller/processor roles, vendors, and data flows. Maintain a versioned legal register; recheck current law before each regulated launch.

- **EU GDPR**; Work required before applicable processing or release: Define roles/lawful bases, notices, purpose/minimization, rights handling, retention, processor agreements, transfer mechanism, security and breach process; assess DPIA, DPO/representative, profiling and Article 22 applicability

- **EU device tracking**; Work required before applicable processing or release: Assess ePrivacy rules and member-state implementation; obtain applicable consent before nonessential tracking, support withdrawal, and gate browser/server marketing delivery consistently
- **US state privacy**; Work required before applicable processing or release: Build a state applicability matrix; implement required access/deletion/correction/opt-outs, GPC and targeted-advertising/sharing controls where applicable; assess sensitive/minor data and retention rules
- **AI governance**; Work required before applicable processing or release: Assess provider/deployer roles, prohibited practices, AI literacy, transparency and risk classification; financial/insurance/recruiting expansions need renewed review, not an assumption they are ordinary marketing
- **Housing and recruiting**; Work required before applicable processing or release: Review discrimination risks, protected-characteristic proxies, targeting/exclusions, applicable Meta special-category rules and business disclosures; no eligibility or hiring decisions in the MVP
- **Insurance and finance**; Work required before applicable processing or release: Determine federal/state/sector regulator, privacy/security and advertising obligations; restrict payloads from carrying underwriting, policy or account details; assess GLBA/FTC and insurance-regulator jurisdiction
- **Communication and content**; Work required before applicable processing or release: Review marketing email/call/text rules if introduced, claim substantiation, testimonial/IP rights, required disclaimers, and applicable accessibility duties

Sources for these review areas: GDPR, ePrivacy Directive, California privacy rights, EU AI Act framework, HUD housing advertising guidance context, FTC Safeguards Rule scope. Historical guidance is context; advisers must verify its current status and applicability.
Engineering evidence includes a data-flow inventory; purpose-specific consent receipts; server-side suppression; authenticated rights requests; deletion across DB, cache, analytics, exports and vendors; deletion tombstones replayed after restoring backups; documented lawful retention/legal holds; vendor/transfer controls; and privacy-safe logs. Apply retention exceptions explicitly rather than deleting required financial records. Region selection alone does not establish lawful international transfers. The system does not use CAPI to bypass consent, opt-outs, browser preferences, or policy restrictions.

Security controls and delivery gates

Use a scoped OWASP ASVS verification baseline with stronger review for tenant isolation, financial actions, identity, and external execution. Scanners supplement design and manual review; a passing scan is not a security certification. OWASP ASVS
Required controls: least privilege; MFA; session revocation; encryption in transit and at rest; key/secret rotation; private database networking; parameterized queries; schema/payload validation; request size limits; OAuth callback/state protections; CSRF defenses; restrictive browser policy; SSRF/egress restrictions; signed webhooks with replay handling; upload

validation; tenant-specific cache/storage keys; scoped exports; auditable time-limited support access; and tested backup/recovery procedures.
Every nontrivial implementation change follows this sequence:

1. Write acceptance criteria and a failing test/repro; inventory changeable values.
2. Implement the smallest feature on a scoped branch with UI/API/data/docs together.
3. Run unit/integration and meaningful regression tests, types, lint, build, dependency checks, gitleaks, Semgrep, and Bandit where Python exists. Exercise edit/commit hooks.
4. Exercise the real HTTP/CLI/UI behavior, external sandbox where available, negative authorization, duplicate/retry paths, and telemetry. Record unavailable tests.
5. Obtain fresh-context review from a reviewer/new session that did not write the change.
6. Resolve findings and record criteria met, every gate, behavior exercised, and verdict.
7. After an escaped defect, append what escaped, which gate missed it, and the added gate to the private defect log. Keep personal scripts; improve them rather than deleting.

Before release, scan tracked/staged files and relevant history for credentials, private keys, unexpected URLs/models/paths, and duplicated defaults. Verify all private paths are ignored and absent from staging/history. Verify every environment variable belongs to typed settings and the example. Report real secrets found, changeable values moved, and intentional protocol constants separately. Configuration regression tests must reject reintroduced provider literals and obvious secret material in source.

Availability capacity and disaster recovery

User-confirmed target: more than one million simultaneous dashboard users AND tracked website visitors. For conservative planning, the final test proposal models 1.1M of each at the same time. The actual request/event distribution must be agreed before capacity purchasing; “one million idle sessions” is not active-use evidence.

Proposed workload contract

- **Dashboard**; Explicit planning assumption: 1.1M active sessions, average one API request per 10 seconds; Derived rate: 110,000 API requests/second
- **Tracking**; Explicit planning assumption: 1.1M concurrent visitors, average one event per 30 seconds; Derived rate: About 36,667 events/second
- **Combined origin-facing work**; Explicit planning assumption: Dashboard requests plus tracking events before caching/batching; Derived rate: About 146,667 operations/second
- **Tracking continuity scenario**; Explicit planning assumption: The example event rate sustained 24 hours; Derived rate: About 3.168 billion events/day

These are arithmetic scenarios, not measured demand or proven capacity. Define peak duration, geography, arrival bursts, journey mix, tenant skew, data size, API endpoints, cache-hit ratio, payload size, retention, report jobs, and connection lifetimes. At an assumed 1 KB per event, the sustained scenario alone is about 3.17 TB/day before indexes/replication; real payload measurements may change it substantially. Cache hits reduce origin work, not the correctness/authorization requirements.

Do not map website activity one-to-one to external Meta/Google calls or AI inference. Ad networks have independent limits; consent, eligibility, batching, supported limits, and policy determine delivery. Sync provider data once per connection/schedule and serve dashboards from local scoped aggregates. Expensive exports/reports run as jobs.

Proposed service objectives

- **Availability**; Proposed target and measurement: At least 99.9% monthly for critical console/API and event acceptance, measured separately with external probes and request SLIs
- **Availability arithmetic**; Proposed target and measurement: In a 30-day month, 0.1% of time is 43.2 minutes; this is not zero downtime
- **Deployment**; Proposed target and measurement: Target zero user-visible failed critical requests caused by a routine release; prove through canary/rollback drills
- **Dashboard latency**; Proposed target and measurement: p95 ≤500 ms and p99 ≤1.5 s for agreed interactive endpoints at the signed workload; report cache-hit and miss results
- **Event acceptance**; Proposed target and measurement: p95 ≤300 ms for valid regional requests after durable acknowledgment; overload must reject/retry explicitly
- **Error rate**; Proposed target and measurement: <0.1% platform-caused failures under the agreed steady load; publish endpoint and time-window definitions
- **Data freshness**; Proposed target and measurement: Internally process 99% of valid received events within 5 minutes; display external provider availability/lag separately
- **Revenue**; Proposed target and measurement: Reconcile the agreed source daily; unresolved differences visible, never silently hidden
- **Recovery**; Proposed target and measurement: Proposed core-data RPO ≤5 minutes and RTO ≤60 minutes for regional disaster; approve and drill per data class
- **AZ failure**; Proposed target and measurement: No loss of acknowledged durable events during tested in-region AZ failure; verify actual queue/DB guarantees

The final SLI contract defines valid requests, example coverage, maintenance handling, provider-dependent journeys, and reporting windows. Track user-visible end-to-end availability even when a third party caused the interruption. Keep last successful data visible with freshness warnings when Meta/GHL/AI is unavailable. Error-budget burn can stop risky rollouts. A provider SLA is not the application's measured SLO.

Use multi-AZ compute/data/queue from the production pilot, health checks, independent worker scaling, circuit breakers, connection pools, tested backups, and observability. In Phase 9 add regional cells and justified disaster replication, with consent/residency constraints. Avoid synchronous global writes on every page view. Keep one authoritative financial write path per tenant/cell; reconciliation handles asynchronous replicas.

Capacity evidence must include scripts, workload definition, data volume, tested infrastructure, cost, p50/p95/p99, errors, saturation, queue backlog/drain, tenant skew, failover results, and safety margin. Use distributed generators and gradual ramps; funded final tests include at least a 2-hour peak and a 24-hour representative soak, plus bursts and failure exercises. Controlled provider adapters protect third parties; approved bounded contract tests separately validate real connectors. No claim above the largest successfully exercised workload.

GitHub and deployment procedure

1. Resolve ordinary `.git` write access; initialize in place, verify actual ignores, install/test scanner hooks, then connect a company-owned private repository.
2. Push small reviewed commits and docs. Protect main, require independent reviews and checks, avoid deploy credentials in forks, and pin workflow action revisions.
3. Check actual GitHub entitlement/spending settings before running Actions or storing artifacts. Do not turn a push into unapproved metered usage. Use OIDC deployment credentials and narrowly scoped company environments.
4. Build once from the reviewed commit; sign the container/static artifact, record commit and digest, scan dependencies/image, and promote the same artifact.
5. Compare deployed manifests/source evidence to local reviewed release; snapshot live versions before changing them. Reconcile unexplained live-ahead drift locally.
6. Apply backward-compatible expand/migrate/contract database changes; keep old and new releases compatible during rollout. Destructive cleanup follows a later release.
7. Deploy staging, validate fixtures/integrations, then release to production canary with approval and tested rollback criteria. Drain workers/connections correctly.
8. Confirm local/deployed file hashes where files are deployed; for built artifacts, verify source commit, signed image/static digests, manifest parity, and served build identity across replicas. Never compare source bytes with compiled output bytes.
9. Run authorized smoke flows, verify metrics/alerts/audit, and record release evidence. Rollback restores compatible artifacts/config; financial or external side effects require reconciliation/compensation, not a blind database rewind.

GitHub documents metered usage beyond private-repository allowances. Verify the actual company account rather than assuming CI is free. [GitHub Actions billing]

Timeline people and costs

Phases 1–7 total **800–1,300 engineering hours**; all ten total **1,880–3,580 hours** under the assumptions above. Re-estimate after architecture approval, provider access, pilot evidence, and initial scale benchmarks. Requirements expansion can exceed these ranges; no fixed-price commitment follows from this planning document.

Suggested delivery team: senior full-stack/technical lead, integration/data engineer, fractional product designer/QA, and fractional security/SRE specialists. Phase 9 needs dedicated SRE/performance ownership. Qualified legal review is a separate workstream. Operational 24/7 coverage requires a rota and incident ownership, not one developer.

Assuming roughly 80–100 productive team hours/week, allow **12–20 calendar weeks for the controlled MVP** including dependency/review time, with the 30-day observation potentially extending acceptance. Budget **9–18+ months for the full program** including multi-business rollout, million-user verification, and handover. These are sequencing estimates, not market benchmarks or promises. A solo developer needs a revised schedule. Meta reviews, account permissions, CRM cleanup, legal decisions, and procurement can extend the critical path without adding coding hours.

At an explicitly assumed blended labor rate of **$60–$120/hour**, the arithmetic is **$48,000–$156,000 for Phases 1–7** and **$112,800–$429,600 for all ten**, before contingency, legal review, independent penetration testing, cloud, taxes, and operations. A fixed-price offer requires discovery and a bounded acceptance contract; quote the architecture milestone separately. Reserve an additional 20–30% planning contingency, then replace assumptions with a supplier/team estimate.

Monthly infrastructure and API planning

For an assumed small internal pilot with up to 100 concurrent console users and up to 100,000 collected events/day, reserve **$500–$2,500/month for core hosting**, plus **$50–$500/month for limited text AI/reporting**. These are internal budget allowances, not verified vendor prices or assurances of sufficient capacity. Existing GHL plans, ad spend, support/on-call labor, legal/security services, subscriptions, image/video generation, and large load tests are excluded. The cloud design may exceed the allowance.
The million-user scenario has **no defensible monthly quote until measurements and vendor calculations exist**. It must not inherit the pilot allowance. Price each region/cell and the sustained versus peak workload before approving Phase 9.
Monthly cost model:

- Compute: replica/node hours × current regional rate, including failure headroom.
- Database: HA compute + storage + I/O + replicas + backup/restore retention.
- Events/analytics: queue operations + ingest/scan/storage + archive lifecycle.
- Network: CDN/WAF/request fees + inter-AZ/inter-region transfer + internet egress.
- Identity/observability: active-user licenses + logs/traces/metrics + alerting.
- AI: input/output tokens by model × current unit price, plus tools/media and retries.
- Delivery/operations: CI minutes/storage, support, incident rota and recovery tests.

Phase 2 produces a saved region/SKU/usage estimate using current provider calculators, and Phase 9 replaces assumptions with benchmark measurements and observed unit costs. AWS Pricing Calculator
Before any paid execution, provide the required ⚠️ **MONEY** warning with verified amount, coverage, recurrence, and explicit confirmation in a fresh turn. Variable usage requires verified unit pricing plus an explicit approved maximum exposure; if a cap cannot be enforced, disclose it and resolve the exposure before proceeding. Budget alerts are not automatic hard stops. This plan authorizes no purchase, subscription, paid inference, load infrastructure, advertising change, or cloud provisioning.

Documentation and developer handover

Documentation grows with each feature. Never leave documentation until Phase 10. Use plain explanations, diagrams, runnable examples, and exact evidence links.

- **README**; Initial owner phase: 1–2; Ongoing content: Product purpose, current status, start-here links, completed/planned scope, quickstart
- **Design guide and screen inventory**; Initial owner phase: 1; Ongoing content: Tokens, components, responsive/accessibility behavior, interface scenarios

- **Architecture overview and ADRs**; Initial owner phase: 2; Ongoing content: Context/container/data-flow diagrams, decisions and reasons, alternatives/tradeoffs
- **API and schema docs**; Initial owner phase: 2 onward; Ongoing content: OpenAPI contracts, auth/scope/error examples, ER diagram, migrations
- **Integration guides**; Initial owner phase: 3 onward; Ongoing content: Company ownership, setup, scopes/versions, webhook verification, sync/retry troubleshooting
- **Tracking dictionary and audit report**; Initial owner phase: 4; Ongoing content: Sites/assets, event mapping, deduplication, consent, diagnostics, rollback
- **Attribution and finance guide**; Initial owner phase: 5; Ongoing content: Identity confidence, models/windows, formulas, refunds, reconciliation, unknowns
- **AI workflow and authority guide**; Initial owner phase: 6; Ongoing content: Evidence, prompts/policy versions, evals, approvals, forbidden actions
- **Legal/privacy and security records**; Initial owner phase: 2 onward; Ongoing content: Public-safe overview; sensitive assessments/evidence in controlled private storage
- **Testing and release evidence**; Initial owner phase: Every phase; Ongoing content: Criteria, commands, gate results, real flows, review verdict, known gaps
- **Operations runbooks**; Initial owner phase: 2 onward; Ongoing content: Deploy/rollback, alerts, provider outages, restore, drift, keys, rights requests
- **Capacity and cost reports**; Initial owner phase: 7 and 9; Ongoing content: Measured envelope, workload, limitations, SLOs, calculators/unit costs
- **User and maintainer guides**; Initial owner phase: 7 onward; Ongoing content: CEO/COO/marketing workflows, developer onboarding, support/handover

Maintain the private seven-section `PROJECT-DOSSIER.md` as the project narrative; public case studies derive from it after scrubbing. Actual credentials belong only in ignored approved storage and the private recovery register. Memory is confined to ignored `memory/`; personal scripts to ignored `manual-tests/`; genuine CI tests remain tracked. Existing Google Docs and `my-project-view/` remain untouched unless explicitly requested. Public docs contain no real customer data, secrets, account identifiers, production infrastructure details, or unverified success claims.

Architecture package coverage

The full package is approved before substantive integration development in Phase 2. Entries below identify the draft in this plan and the detailed deliverable to finalize.

- **1 Architecture diagram**; Draft coverage and Phase 2 artifact: Proposed architecture → approved context/container/data-flow diagrams
- **2 Technology stack**; Draft coverage and Phase 2 artifact: Proposed stack → pinned versions, dependency/license review and ADRs
- **3 Meta integration plan**; Draft coverage and Phase 2 artifact: Phase 3 → verified API/version/permission/account-access matrix
- **4 Pixel architecture**; Draft coverage and Phase 2 artifact: Phase 4/event map → site/asset inventory and tag/consent placement
- **5 CAPI architecture**; Draft coverage and Phase 2 artifact: Phase 4 → payload/source/retry/deduplication and delivery specification

- **6 GHL integration**; Draft coverage and Phase 2 artifact: Phase 3/5 → endpoints/scopes/webhook and field/source mappings
- **7 Database schema**; Draft coverage and Phase 2 artifact: Canonical schema → ER diagram and reviewed initial migrations
- **8 Attribution model**; Draft coverage and Phase 2 artifact: Attribution section → versioned model and evidence/unknown rules
- **9 Event map**; Draft coverage and Phase 2 artifact: Bootcamps event map → approved names, source owners and provider mapping
- **10 AI architecture**; Draft coverage and Phase 2 artifact: AI roles/authority → workflow contracts, evals and LLM gateway policy
- **11 Security**; Draft coverage and Phase 2 artifact: Security section → threat model and verification/control matrix
- **12 Permissions**; Draft coverage and Phase 2 artifact: Permissions table → endpoint/action and tenant-scope authorization matrix
- **13 Dashboard wireframes**; Draft coverage and Phase 2 artifact: Phase 1 → approved clickable screens and behavior inventory
- **14 MVP scope**; Draft coverage and Phase 2 artifact: Release boundaries → signed requirements and acceptance scenarios
- **15 Timeline**; Draft coverage and Phase 2 artifact: Effort/timeline → resourced dependencies and milestone schedule
- **16 Development cost**; Draft coverage and Phase 2 artifact: Labor assumptions → bounded architecture/MVP quote and contingency
- **17 Monthly costs**; Draft coverage and Phase 2 artifact: Cost model → saved current vendor estimates with workload assumptions
- **18 Testing**; Draft coverage and Phase 2 artifact: Phase gates → automated/manual/contract/security/load evidence plan
- **19 Deployment**; Draft coverage and Phase 2 artifact: Deployment procedure → environment, migration, rollback and parity runbooks
- **20 Documentation**; Draft coverage and Phase 2 artifact: Documentation table → repository index, owners and handover checklist

Risks dependencies and current blockers

- **Meta permissions/version support**; Mitigation and decision point: Verify official docs/account behavior before connector work; start company app review early
- **Incorrect CRM stages/payment truth**; Mitigation and decision point: Approve definitions and source ownership before attribution; expose reconciliation exceptions
- **Consent/identity gaps**; Mitigation and decision point: Preserve unknown attribution; do not overstate exact-ad matching or bypass choices
- **Low sales volume and delayed outcomes**; Mitigation and decision point: Show lag/sample size; require human decisions; evaluate by mature cohorts
- **Regulated housing/insurance/recruiting**; Mitigation and decision point: Specialist applicability and targeting/content review before each expansion

- **Million-user cost and architecture uncertainty**; Mitigation and decision point: Signed workload, incremental benchmarks, funded final tests; no capacity claim from a design
- **Third-party outages and pacing**; Mitigation and decision point: Durable ingestion, circuit breakers, freshness states, bounded execution and reconciliation
- **Personal ownership or lost access**; Mitigation and decision point: Company-controlled assets, scoped access, recovery drill and handover
- **Single-person operations**; Mitigation and decision point: Assign on-call and backup maintainers before contractual uptime commitments

Current delivery prerequisites: actual Git exclusions, installed security hooks, source ownership and static HTTPS publication are verified. Remote Actions remain disabled. Earlier permission restrictions were resolved without metadata bypass.
Open implementation gates: company provider accounts and supported versions/scopes, tracking audit, architecture/legal approval, authoritative CRM/finance definitions, backend security, load budget, independent testing and operating ownership. Meta docs returned HTTP 429 during this review; no current API version/permission is asserted. See integration requirements, compliance and capacity/reliability.

Planning and delivery verification

The original planning criteria cover all ten scopes, architecture, governance, capacity, estimates and handover. Coverage of a plan is separate from phase acceptance. Current frontend and source delivery evidence is in publication verification and hero revision. Current documentation gates are in documentation verification.
The published frontend has fifteen routes; the current hero uses reduced-motion-aware film autoplay and full desktop artwork coverage. Static publication does not close backend, legal, million-user or availability gates. The next implementation milestone is architecture approval and the secure executable foundation in phase 2, with any remaining phase 1 design acceptance resolved with the owner.

Frontend developer guide

React 19, TypeScript 6, Vite 8, Motion 14 and Lenis 1 power the current frontend. Exact versions are in the manifest/lockfile. Use Node 24 LTS and `npm ci` at the root.

Code map

- **`App.tsx`**; Responsibility: Home, routes, transitions and scrolling lifecycle
- **`components/SiteShell.tsx`**; Responsibility: Shared accessible header/navigation/footer
- **`components/Pages.tsx`**; Responsibility: Supporting pages, articles and contact brief
- **`components/Dashboard.tsx`**; Responsibility: Workspace views, filters, dialogs and local decisions
- **`components/Primitives.tsx`**; Responsibility: Reusable interface primitives

- **`data/site.ts`**; Responsibility: Typed page registry, metadata, navigation and copy
- **`data/content.ts`, `data/interface.ts`, `data/templates.ts`**; Responsibility: Maintained content, labels and creative templates
- **`data/workspace.ts`**; Responsibility: Reference business records
- **`config.ts`**; Responsibility: Assets, motion, input, filenames and display settings
- **`lib/routing.ts`**; Responsibility: History routes and offline hash compatibility
- **`lib/contact.ts`**; Responsibility: Normalized validation and local brief preparation
- **`lib/metrics.ts`**; Responsibility: Economics and safe CSV export
- **`lib/motion-preference.ts`**; Responsibility: Reactive reduced-motion subscription
- **`../vite.config.ts`**; Responsibility: Escaped route metadata and per-route HTML emission

Development

```
npm ci
npm run dev
npm test
npm run typecheck
npm run lint
npm run format:check
npm run build
npm run preview
```

Use the printed URL. Ten top-level public pages, three articles, workspace and not-found entry must support direct paths and back/forward. Former console links remain compatible. Unknown hosted URLs return a real HTTP 404.

Keep focus visible, controls named, dialog focus contained/restored and tabs keyboard operable. Verify 320px through wide desktop layouts and internal table scrolling. Target WCAG 2.2 AA; full accessibility assessment remains a separate gate.

The muted inline looping film autoplays without controls. Reduced motion at load and runtime disables decorative motion and smooth scrolling; media failure keeps the poster. No paid call runs on page view or video playback.

Contact preparation creates a local download and sends nothing. Preserve values when editing. Workspace approvals bind to the displayed evidence/reporting period; local state clears on refresh. CSV output protects against spreadsheet formulas.

Backend connection points

Introduce typed data adapters as real APIs arrive; keep provider requests out of components. Preserve disconnected/loading/empty/stale/error, permission-denied and insufficient-evidence states. Read currency, timezone, model, cohort and freshness from the API. Server authorization applies to every request; hiding UI is insufficient.

Backend contracts describe session, tenant, reports, journeys, recommendations and approvals. Provider secrets never enter bundles. Maintained copy belongs in `data/`; public behavior settings in `config.ts`. Any future public env variable must be safe, validated and listed in `.env.example`; none is required now.

Release

Deploy only `dist/` and reviewed hosting configuration. Follow the static runbook, contribution gates and current hero evidence. Preserve local manual tooling; store private screenshots/manifests in ignored folders and publish scrubbed results.

Backend design and implementation contract

Status: planned implementation. No executable API, worker, migration or authentication service exists here yet. Implement this contract through phases 2–7 of the roadmap, connecting UI, API, data, tests and docs together.

Boundaries

Proposed starting point: modular TypeScript/NestJS API and independent workers. Approve versions, licenses and hosting in phase 2; extract services when measured isolation or scaling needs justify the operating cost.

- **Identity/tenancy**; Responsibility: OIDC, memberships, scoped roles, revocation; Constraint: Never trust browser tenant scope
- **Connections**; Responsibility: Approved OAuth, encrypted token references, refresh and sync state; Constraint: No provider tokens in browser
- **Ingestion**; Responsibility: Verify/validate, durably receive and deduplicate; Constraint: Bind tenant/account to verified installation
- **Conversion delivery**; Responsibility: Consent-aware outbox, CAPI mapping, retries and diagnostics; Constraint: Recheck permission before sending
- **Funnel/identity**; Responsibility: Contact links, stage history, appointments and enrollments; Constraint: No name-only identity merge
- **Finance**; Responsibility: Payments, refunds, costs, FX and reconciliation; Constraint: Opportunity value is not cash
- **Attribution**; Responsibility: Versioned touch eligibility, credit and unknown reasons; Constraint: Never invent identifiers
- **Reporting**; Responsibility: Scoped aggregates, freshness and asynchronous exports; Constraint: No full-history scan per page
- **AI/approval**; Responsibility: Evidence, bounded output and exact proposal approval; Constraint: Policy is enforced outside prompts
- **Execution**; Responsibility: Later approved provider writes; Constraint: No asset/payment/security administration
- **Operations/privacy**; Responsibility: Audit, retention, deletion/replay and jobs; Constraint: Preserve lawful financial retention

Draft API contracts

These resource families are design contracts, not current website endpoints. Version OpenAPI beside implementation. Tenant selectors supplement membership checks.

- **Session**; Contract: Read user/scopes/tenants, revoke; Protection: OIDC validation, secure cookies, CSRF
- **Connection**; Contract: OAuth start/callback, health, revoke; Protection: Role, state, exact callback and scope checks
- **Reports**; Contract: Summary/campaign/ad/creative economics; Protection: Unit scope, bounded dates/pagination
- **Journey**; Contract: Lead→appointment→enrollment→money; Protection: PII permissions and access audit
- **Tracking**; Contract: Health/rejection/dedup/consent diagnostics; Protection: Sanitized payloads, restricted access
- **Recommendation**; Contract: Five-state result and evidence snapshot; Protection: Version, freshness and uncertainty
- **Draft**; Contract: Internal campaign/creative draft; Protection: Validated schema; no implicit publication
- **Approval**; Contract: Approve/reject exact proposal; Protection: Payload hash, policy, expiry and replay protection
- **Export**; Contract: Async scoped report and expiring download; Protection: Rechecked authorization, safe CSV
- **Webhook**; Contract: Provider-specific installation receiver; Protection: Current signatures, bounded body, durable receipt
- **Privacy**; Contract: Verified rights request and progress; Protection: Identity verification and legal-hold review
- **Health**; Contract: Separate liveness/readiness; Protection: No sensitive public diagnostics

Use opaque IDs, UTC occurrence/receipt times, reporting timezone, schema versions, minor currency units/currency codes and correlation IDs. Internal retryable mutations require tenant/action-scoped idempotency keys; reject changed payload reuse. External webhooks use verified provider identities. Safe field errors and appropriate HTTP statuses must not leak cross-tenant existence. Long reports return a job receipt.

Authorization and database

Derive tenant scope from session membership, enforce it in services, RLS, constraints, cache/storage keys, jobs, exports and AI retrieval. Runtime database roles cannot bypass policies. Pool connections use transaction-local tenant context, with tests for context leakage. Include tenant scope in appropriate foreign/unique keys. Business units inside group reporting still require explicit permission.

Separate admin, marketing, operations, finance and privacy/support capabilities. Support grants are justified, expiring and audited. Automated workers get minimal roles. PostgreSQL RLS.

The data guide specifies logical entities. Phase 2 adds reviewed migrations, indexes, ER diagram and reproducible local startup. Unique constraints protect provider identity, event occurrence, money posting and execution. Expand/migrate/contract changes preserve old/new API/worker compatibility. Backfills are bounded; assess locks and replication lag before large indexes or data rewrites.

Durability and external side effects

Transactions commit domain changes and outbox together. Queue delivery can repeat; handlers plus unique constraints produce one logical effect. An external timeout can hide success: reconcile ambiguous outcomes before a non-idempotent retry. Record attempts, receipt/occurrence times, consent, schema/policy and correlation identity.
Use bounded backoff/jitter, provider/account rate budgets and dead-letter review. Replay rechecks current privacy/permission. Revocation stops jobs and marks data stale. Drain workers safely during releases; separate interactive and batch capacity.

Configuration boundary

Create one typed, validated settings module before runtime variables. Categories: service origins/ports, OIDC, DB, queues/cache/storage, encryption references, provider URLs/versions/scopes, retries/timeouts, retention, models/AI budget, action ceilings and telemetry. Constructors receive settings instead of inventing defaults.
Every implemented variable enters `.env.example` with safe values/ownership. Missing required production secrets fail startup; actual values live in the approved secret store. Version maintained event/attribution/approval policies as data. Do not add unimplemented backend variables to the current frontend environment file.

Implementation and acceptance

Build real login/tenant isolation first, then reconciled Meta/GHL reporting, consented tracking, persisted revenue journey, evidence-bound AI/approvals and the controlled pilot. Test cross-tenant negatives, signature/replay, refresh races, duplicate payments, outbox crashes, consent withdrawal, deletion after restore, money/FX/cohort arithmetic, stale approvals, concurrent budget reservations and degraded providers.
Run bounded contract tests against approved provider environments. Never load-test Meta/GHL/Google. Contribution gates, security and operations govern release.

Integrations, tracking, data and attribution

Status: implementation contracts for phases 2–5 and 8. Current accounts are disconnected. Integration work begins after company access, architecture and applicable privacy approvals.

Provider register and installation

Maintain a private versioned register per connector: company-owned app/account, verified API version, requested/granted scopes, access level/review status, supported endpoints, webhook verification, limits, token lifecycle, retention/terms and test evidence. Provider IDs are opaque strings. Do not scrape or use undocumented/deprecated functionality.

- **Meta**; Initial data: Accounts, campaigns, ad sets, ads, creatives, Insights, authorized Lead Ads; Write boundary: Internal drafts first; approved executor later; Implementation prerequisite: Current version, company assets, app review/permissions and authorized test data
- **GHL**; Initial data: Contacts, opportunities, pipelines, calendars/appointments, tags and approved fields; Write boundary: Selected events/fields only after operations approval; Implementation prerequisite: Installation/location scopes, stage definitions and webhook verification
- **Google Ads/GA4**; Initial data: Campaign spend, approved conversions and reporting; Write boundary: Approved conversion ingestion and later scoped changes; Implementation prerequisite: Developer/access approval, conversion route and consent requirements
- **Finance/enrollment source**; Initial data: Enrollment evidence, payments, refunds, fees/costs; Write boundary: Source-owned records; controlled reconciliation; Implementation prerequisite: Approved authoritative source and immutable posting identity

OAuth exchanges happen on the backend, with state binding, exact callbacks, minimum scopes, encrypted secret references and serialized refresh. Never store provider tokens in browser storage or prompts. Revocation disables jobs and requires explicit reconnection. Webhook tenant/account binding comes from verified installation metadata, not body trust. Meta's documentation returned HTTP 429 during this documentation review. No current API version or permission set is asserted as verified. Verify official docs and actual account access before coding the connector. Meta Marketing API. GHL OAuth and webhooks are the current implementation starting points; verify event-specific support/signatures. For Google, select the supported ingestion path for the actual account at implementation; see conversion management.

Synchronization and trusted reporting

Capture bounded historical backfill, pagination/cursors, account currency/timezone, query/API versions, receipt timestamps and sync checkpoints. Reconcile a bounded account/date sample against each provider UI/export with documented metric definitions, reporting lag and attribution windows. Refresh approved partitions for late conversions.

Jobs have per-provider/account limits, bounded retries, backoff/jitter, circuit breakers, dead-letter review and reconnect states. Persist webhook receipts before acknowledging success. A duplicate receipt can update attempt telemetry without repeating business effects. Dashboard refreshes use scoped local read models; exports and backfills run as jobs.

Audit tracking before modifying assets

Inventory accessible company datasets/pixels, domains, websites, funnels, tag containers, CMS plugins, existing CAPI senders and GHL automations. Record ownership, event triggers, consent state, diagnostics and the route from business occurrence to receipt. Find missing triggers, duplicate tags, obsolete senders and incorrect values/timezones.
Reuse suitable assets. New datasets require an approved ownership/business rationale. Snapshot configuration and rollback before migration/removal. Inspect actual network payloads and provider diagnostics; a homepage tag is not a complete revenue architecture. Do not place advertising tags in the authenticated AdRevora console by default.

Event dictionary

Internal names are proposals; approve standard/custom external mappings, allowed fields, source values, event-age limits and optimization eligibility against the current account.

- **Landing viewed**; Authority: Approved public page; Browser: After required consent; Backend/CRM: Optional receipt; External decision: PageView if permitted
- **Event registration/lead accepted**; Authority: Successful form or Meta Lead Ad; Browser: Website success only; Backend/CRM: Preserve form/provider identity; External decision: One approved Lead/registration mapping
- **Attendance**; Authority: Verified attendance system; Browser: Never inferred from a page view; Backend/CRM: Source integration; External decision: Internal first; optional approved custom mapping
- **Platinum appointment**; Authority: Confirmed GHL booking; Browser: Only same confirmed booking; Backend/CRM: Appointment identity/status; External decision: Schedule/custom mapping if permitted
- **Consultation completed**; Authority: Defined CRM stage and operator evidence; Browser: No; Backend/CRM: Verified transition; External decision: Internal first
- **Platinum enrollment**; Authority: Verified enrollment source; Browser: Optional exact confirmation; Backend/CRM: Enrollment identity; External decision: Avoid duplicate payment reporting
- **Collected payment**; Authority: Approved finance/payment record; Browser: Optional same payment; Backend/CRM: Actual value/currency; External decision: Purchase if permitted
- **Refund/chargeback**; Authority: Finance source; Browser: No; Backend/CRM: Reversing ledger posting; External decision: Only supported adjustment route

Every event stores tenant/unit, stable identity, schema, occurrence/receipt times, source provenance, consent/purpose and allowed evidence. Business stages are distinct; a lead is not a booking and an enrollment is not automatically a payment.

Deduplication and CAPI

For browser/server copies of one conversion, propagate one opaque durable event ID and the same provider event name within the same dataset. Browser and server senders must preserve

this identity; queue retries reuse it. A subsequent booking/payment gets a new identity. Never use an email as the event ID or randomly regenerate it on retry.
Deduplicate provider receipts, logical events and financial postings independently. CAPI deliveries use an outbox, current consent checks at queue/send time, approved matching fields, provider-required normalization/hashing, valid occurrence time and source/value/currency. Hashing is not anonymization. Strip sensitive query strings and insurance/financial details from URLs and external payloads.
Validate browser+server reception/dedup with Meta Test Events and diagnostics on approved assets. Test repeated delivery, duplicate tags, second real conversion, browser-only, CRM-only, denied/withdrawn consent, invalid fields, delayed events and partial failure. HTTP success alone is insufficient. Meta deduplication reference. Record supported behavior against current docs/account tests before production.

Logical schema

Phase 2 turns these domains into reviewed migrations and an ER diagram. All tenant relationships preserve tenant scope; uniqueness includes provider/account where needed.

- **Access**; Entities: tenants, business_units, users, memberships, roles, sessions
- **Connections**; Entities: installations, external_accounts, secret_refs, granted_scopes, cursors, receipts
- **Advertising**; Entities: campaigns, ad_sets, ads, creatives, offers, markets, historical_names, daily_insights
- **Tracking**; Entities: events, deliveries, consent_receipts, touchpoints
- **CRM**; Entities: contacts, identity_links, merges, leads, lead_forms, opportunities, stage_history
- **Funnel**; Entities: appointments, appointment_transitions, consultations, enrollments
- **Finance**; Entities: orders, invoices, payments, refunds, chargebacks, revenue_postings, costs, FX
- **Attribution**; Entities: model_versions, runs, credits, evidence_links, unknown_reasons
- **AI/actions**; Entities: analysis_runs, evidence_snapshots, recommendations, drafts, approvals, reservations, executions
- **Operations/privacy**; Entities: outbox, job_runs, audit_events, reconciliation_exceptions, retention/deletion_tasks

Architecture relationships (the repository includes the rendered diagram):

- TENANT → BUSINESS_UNIT (contains, one to many)
- BUSINESS_UNIT → CAMPAIGN (scopes, one to many)
- CAMPAIGN → AD (groups, one to many)
- AD → TOUCHPOINT (identifies, one to many)
- CONTACT → TOUCHPOINT (links, one to many)
- CONTACT → APPOINTMENT (books, one to many)
- CONTACT → ENROLLMENT (joins, one to many)
- ENROLLMENT → PAYMENT (collects, one to many)
- PAYMENT → REVENUE_POSTING (records, one to many)
- REVENUE_POSTING → ATTRIBUTION_CREDIT (allocates, one to many)

- TOUCHPOINT → ATTRIBUTION_CREDIT (receives, one to many)

The diagram is conceptual; tenant constraints, refund references, consent and remaining entities are specified in migrations. Use precise decimals/minor units, original currency, versioned FX and distinct occurrence/receipt/account-reporting times. Partition/archive high-volume events based on measured workload and approved retention.

Identity and attribution

Capture unit/provider/account/campaign/ad-set/ad/creative IDs, historical names, UTMs, approved click/browser identifiers, page/form, creation time and permitted first/last touches. Link GHL contact, appointments, opportunity, enrollment and finance evidence. Validate parameters; names and UTMs alone do not prove an exact ad match.

Prefer explicit lead-form IDs and validated ad/click links. Contact matching is scoped, confidence-based and correctable with history. Handle merges without duplicating money. Do not fingerprint or infer an unobserved cross-device journey. Expose deterministic, partial and unknown outcomes with reason codes and coverage.

Initial models: first eligible touch and last eligible paid touch. Window, eligibility, lookback and refund/cohort policy are versioned product data. Later multi-touch credits sum to at most 100% of a posting. Keep network-reported views separate: Meta and Google may both claim the same sale. Attribution credit does not prove causal lift.

A $5,000 collected payment links to verified enrollment/contact and eligible recorded touch, then the actual account/ad/campaign. If evidence is missing, retain $5,000 in total revenue with unknown attribution. If two campaigns touched the person, show model-specific credit and the touch timeline. Refunds append reversing postings and restate metrics without deleting the original transaction.

Financial definitions

- **CPL**; Definition and required label: Approved ad spend / distinct defined leads
- **Cost per appointment**; Definition and required label: Spend / qualifying unique appointments; cancellation policy
- **Cost per enrollment**; Definition and required label: Spend / verified enrollments; refund and cohort basis
- **CAC**; Definition and required label: Agreed acquisition costs / new customers; disclose ad-only basis
- **Revenue per lead**; Definition and required label: Net collected or recognized revenue / cohort leads; basis
- **ROAS**; Definition and required label: Attributed revenue / ad spend; model/window/currency
- **Profit**; Definition and required label: Revenue less refunds, ads, fulfillment/COGS, fees, commissions and agreed costs
- **Attribution coverage**; Definition and required label: Eligible revenue/sales with adequate evidence / all eligible revenue/sales

Missing costs mean unavailable or explicitly partial contribution, not audited profit. Zero denominators produce insufficient-data states. Never sum currencies without approved FX. Distinguish acquisition cohort from calendar-period cash reporting.

End-to-end acceptance journey

On approved accounts, prove ad identity → consented Pixel/CAPI event → lead → GHL contact → appointment → completed consultation → enrollment → real money posting → report → AI recommendation. Also prove unmatched sale, multiple touches, installments, refund, repeated webhook, delayed outcome, contact merge and consent withdrawal. Reconcile source totals daily, expose exceptions and retain a sanitized evidence trace.

AI workflows, approval and governance

Status: implementation requirements. The current workspace uses local reference records; no model or provider action is connected. Implement these flows in phase 6, then add narrowly approved execution after pilot and resilience acceptance.

Workflow responsibilities

- **Ads Director**; Inputs and output: Reconciled economics → ranked priorities and executive brief; Initial authority: Recommend
- **Strategist**; Inputs and output: Offer/funnel/audience policy → objective, CTA, budget/KPI brief; Initial authority: Draft
- **Creative Director**; Inputs and output: Approved claims and creative history → hooks/copy/concepts/scripts; Initial authority: Draft
- **Campaign Builder**; Inputs and output: Approved brief/assets → internal campaign/ad-set/ad draft; Initial authority: No publication
- **Analyst**; Inputs and output: Entire funnel → evidence and alternative diagnoses; Initial authority: Analyze
- **Budget Optimizer**; Inputs and output: Mature cohorts, cost/uncertainty → five-state recommendation; Initial authority: No budget write
- **Retargeting Manager**; Inputs and output: Verified stage/exclusions → next-action segment plan; Initial authority: Propose
- **Attribution Agent**; Inputs and output: Deterministic credits/ledger → explain trace and unknowns; Initial authority: No invented identity/money

Implement a bounded workflow/state machine before introducing distributed agents. Use a provider-neutral model gateway with configured model IDs, schema validation, timeouts/retries and per-tenant cost caps. Historical learning starts with versioned queries, creative metadata and evaluated prompts; customer-data model training needs separate terms, purpose and approval.

Recommendation contract

Every recommendation includes tenant/account/campaign, reporting/cohort window, source timestamps, attribution model, financial basis, sample size, conversion lag, evidence hash, model/prompt/policy versions, uncertainty, expiration and rationale.

- **SCALE**; Interpretation: Economics support a bounded human-reviewed increase
- **MAINTAIN**; Interpretation: Current economics justify continuation within approved limits
- **TEST**; Interpretation: Run a defined hypothesis with a capped budget and success/stop rule
- **FIX**; Interpretation: A tracking, landing-page, follow-up, appointment, offer or sales issue needs review
- **PAUSE**; Interpretation: Evidence supports human review of a stop; no automatic initial action

Good CTR with poor landing conversion suggests a page/offer issue; strong lead volume with weak appointments suggests follow-up; strong appointments with low closes suggests sales/offer issues. Separate these hypotheses from conclusions. Low sample counts, stale data, unknown revenue and immature cohorts reduce confidence. Two enrollments alone do not establish a reliable scaling rule.

Human authority

- **Publish/pause, launch a test, change creative/budget within policy**; Required approver: Marketing Director
- **Routing, sales process or significant operations**; Required approver: COO
- **Major spend, new offer/pricing/market, significant vendor or strategy change**; Required approver: CEO
- **Asset ownership, users/partners, payment methods, security**; Required approver: Human administrator only; outside AI executor

Approvals bind exact tenant, business unit, account, action/payload hash, evidence, policy version, ceiling, approver and expiry. Changed payload/evidence or revoked permission invalidates approval. Recheck live provider state and authorization just before execution; rejected/expired approvals never become executable by prompt text.

Future spending controls

A separate restricted executor has only approved write capabilities. Enforce account, company and campaign ceilings, shared reservations/locks for concurrent requests, pacing, cool-down, permitted delta, max action count and a global/company kill switch. A daily platform budget is not a guaranteed invoice cap: account for provider delivery behavior, attribution lag and pending changes. Reserve worst approved exposure and reconcile live spend; fail closed when trustworthy spend or policy is unavailable.

Record idempotency identity, before/after state, execution result and uncertain external outcomes. Reversal uses compensating action; served ads, spent money and delivered data cannot be literally undone. Stop new changes and escalate when reconciliation fails. Test

simultaneous requests, timeout-after-success, replay, stale approval, ceiling changes and kill-switch behavior before enabling automation.

Model security and evaluation

Treat CRM text, imported ads and web content as untrusted data. Separate instructions from evidence; allowlist tool schemas/actions. No secrets, raw contact payloads or cross-tenant retrieval enters prompts. Minimize aggregates and assess vendor training, retention, residency and subprocessors. Log safe IDs/versions/cost rather than prompts containing sensitive source material.
Evaluation covers arithmetic grounded in tools, unsupported claims, wrong tenant, privacy-sensitive content, missing disclaimers, prompt injection, ambiguous attribution, low samples, model refusal/outage and unauthorized execution. Store a versioned dataset and release thresholds; a model upgrade needs evaluation rather than an automatic swap.

Ownership and audit

Company-controlled accounts own code, domains, cloud, Meta/GHL assets, database, creative history, prompts and workflows. Developers receive scoped access, revoked at handover. Audit captures actor, tenant, request, evidence, approvals, before/after, correlation, timestamps and outcome; sensitive details follow retention/access policy.
Define named product, technical, finance, security/privacy and operations owners in private governance records. The requirements brief does not establish client consent or permission for production accounts. Compliance and operations define expansion/release controls.

Security policy and implementation requirements

Private reporting

Contact the repository owner privately through GitHub or the established project channel. Do not publish credentials, customer data or infrastructure details in issues. Include affected commit/component, impact and safe reproduction. A dedicated monitored security mailbox, response SLA and bounty are not configured; establish response ownership before wider onboarding.

Current protections

The deployed scope is static frontend. No advertising token, customer database, login or connected AI runs in the browser. Assets are self-hosted. HTTPS, restrictive headers and a nonroot/read-only/resource-limited privately bound container are verified within that scope. See publication evidence.

Local Gitleaks, Bandit and Semgrep supplement tests/review. Repository Actions remain disabled. Re-evaluate CSP when adding approved backend/auth origins; do not replace specific rules with wildcards. Passing scans do not establish security certification.

Required platform controls

- **Session**; Controls: OIDC, MFA, secure cookies, CSRF, revocation; Acceptance evidence: Real login/revocation and session negatives
- **Tenant**; Controls: Membership scope, endpoint checks, RLS, tenant-aware jobs/cache/storage; Acceptance evidence: Cross-tenant read/write/export/retrieval tests
- **Provider**; Controls: Least-privilege OAuth, state/callback checks, encrypted tokens, refresh locking; Acceptance evidence: Install/revoke/expiry and permission register
- **Ingestion**; Controls: Verified signatures, bounded payload, replay/idempotency, durable receipts; Acceptance evidence: Tampered/duplicate/out-of-order tests
- **AI**; Controls: Minimal inputs, untrusted-content isolation, allowlisted tools, output schemas; Acceptance evidence: Injection and unauthorized-action evaluations
- **Money/actions**; Controls: Exact approval, live policy check, reservations, ceilings, kill switch; Acceptance evidence: Race/replay/stale-approval/reconciliation tests
- **Runtime**; Controls: TLS, private data services, restricted egress, minimum privileges, patching; Acceptance evidence: Configuration/security assessment
- **Files**; Controls: Scoped expiring exports, safe CSV, upload validation; Acceptance evidence: Unauthorized download and unsafe-file tests
- **Secrets**; Controls: Approved vault, separated environments, rotation, redacted logs; Acceptance evidence: Leak scan and rotation/recovery drill
- **Availability**; Controls: Multi-AZ, overload controls, backup/restore, on-call; Acceptance evidence: Failure/load/restore reports

Use a versioned OWASP ASVS baseline. Threat-model cross-tenant access, token theft, forged events, duplicate money, injection, overspend, privacy replay and provider compromise. Keep detailed assessments private.

Lifecycle and launch gate

Classify/minimize data; enforce consent, retention, rights and deletion after restore. Hashing identifiers does not make them anonymous. Restrict sensitive insurance/financial fields from advertising/AI payloads. Review vendor terms, data use and transfers.
Assign dependency patch ownership. Every release requires tests, types/lint/build, secret/SAST/configuration scans, real flows and independent review. Connected production pilots require independent penetration testing, incident/recovery drills and closure of launch-blocking findings. Any future SOC 2/ISO audit needs separate operating evidence. Contain incidents, preserve safe evidence, rotate affected credentials, reconcile external effects and assess notification duties with the privacy owner. Recover from verified artifacts and record corrective gates. See operations and compliance.

US/EU compliance and privacy implementation plan

Research reviewed: 2026-10-12. This is the engineering/legal work register for the future connected platform. The current release is static frontend with disconnected accounts and a local contact brief. Applicable obligations require qualified review, implemented controls and operating evidence before relevant processing starts.

Scope and accountable decisions

Before onboarding, record legal entity/establishment, target countries/states, user and lead locations, business sector, age scope, collected fields, purposes, vendors, data flow, hosting/transfer destinations and controller/processor responsibilities. The company controls its marketing purposes; AdRevora's future SaaS role depends on its contracts and actual processing. Do not assume one role for every activity.
Maintain a versioned applicability register with legal basis, owner, controls, evidence, decision date, next review and release gate. Counsel owns legal interpretation; privacy owner rights/consent; security owner safeguards; finance owner retention and money; marketing owner claims/disclosures; product owner accepted scope. Reassess when adding a network, country, business, data type, AI use or sales communication channel.

EU/EEA data protection

The GDPR provides the primary review framework for roles, lawful basis, transparency, rights, privacy by design, security, processor terms, impact assessment and international transfers. Engineering controls:

- **Purpose/basis**; Required implementation evidence: Field-level inventory, lawful-purpose decisions, notices and minimization
- **Roles/vendors**; Required implementation evidence: Controller/processor mapping, agreements, subprocessor/change review
- **Rights**; Required implementation evidence: Verified intake, scoped access/correction/deletion/export/restriction/objection workflows
- **Retention**; Required implementation evidence: Approved schedule per data class, deletion jobs, financial/legal-hold exceptions
- **High-risk processing**; Required implementation evidence: DPIA/DPO/representative/profiling and automated-decision applicability decisions
- **Transfer**; Required implementation evidence: Approved mechanism and transfer assessment where required, vendor/access locations
- **Security/breach**; Required implementation evidence: Risk-based safeguards, incident ownership and notification decision evidence

Track rights deadlines by jurisdiction rather than hardcoding one global period. A qualified owner determines whether Article 22 or other decision safeguards apply. Under GDPR Article 33, notify the competent authority where required without undue delay and, where feasible, within 72 hours of awareness; assess affected-person duties separately. Incident escalation must enable those decisions rather than wait for a complete technical postmortem. Keep legally

required financial records under explicit restricted retention; deletion is not indiscriminate destruction.

Device tracking and marketing delivery

Assess national ePrivacy implementations for device storage/access, pixels and links. The EDPB final technical-scope guidance is a starting point. Where required, obtain valid consent before nonessential tracking, provide usable withdrawal and keep browser/server decisions consistent.
Consent receipts record purpose, notice/policy version, timestamp, source and permitted identifiers without collecting unnecessary data. A marketing outbox checks permission both when queued and before sending. Withdrawal suppresses pending sends and future eligible processing. CAPI, hashing and server-side architecture do not bypass choices. Test no-consent, reject-all, accept-selected, withdrawal, region/unknown state and retries. Keep authenticated console analytics separate from customers' advertising tags.

US state privacy

Use an applicability matrix for each targeted state, thresholds, exemptions and data activities. California review starts with the Attorney General CCPA overview and CPPA FAQs. Covered processing may require access, correction/deletion and sale/sharing or targeted-ad opt-outs, including qualifying opt-out preference signals such as Global Privacy Control. Review sensitive/minor data and service-provider/contractor terms. CPPA's current regulation register also covers audit, risk-assessment, automated-decision and insurance developments; verify applicability and operative dates for the actual business.
Engineering acceptance includes signal handling through capture/outbox/replay, required links/notices, verified requests, vendor propagation and a legal exception record. Do not infer that a GLBA-related exemption removes all state privacy duties for a company or every dataset. Maintain state breach-notification decisions privately.

AI governance

Assess provider/deployer roles, prohibited practices, AI literacy, transparency and risk classification under the EU AI Act framework. The Commission's Article 50 guidance states transparency obligations apply from 2 August 2026, with a limited legacy-system marking transition. Its enforcement timeline, updated 6 October 2026, distinguishes later high-risk implementation dates. Recheck the enacted rules and classification before launch rather than treating all AI as one category.
Inventory interactive AI and generated media, determine applicable user disclosures and content marking, and preserve provenance. Review insurance, recruiting, education or eligibility uses separately; marketing recommendations do not authorize hiring, underwriting or other consequential decisions. Keep initial scope outside those functions. Human oversight, prompt/model versions, evaluation, vendor terms and incident response are implementation gates. AI/governance.

Sector, advertising and communications

- **Housing/real estate**; Required review and controls: Fair-housing duties, protected-characteristic proxies, audience exclusions and claims; Primary starting point: HUD digital-advertising guidance context
- **Insurance/finance**; Required review and controls: Applicable federal/state regulator, GLBA scope, licensing, disclosures and data restrictions; Primary starting point: FTC GLBA overview
- **Advertising claims**; Required review and controls: Substantiation, endorsements/results claims, material disclosures, generated-media rights; Primary starting point: FTC substantiation policy
- **Email/calls/texts**; Required review and controls: Channel-specific permission, opt-outs, suppression, records and sender requirements if introduced; Primary starting point: FTC CAN-SPAM guide, FCC telemarketing
- **Accessibility**; Required review and controls: Applicable accessibility duties and WCAG 2.2 AA engineering target; Primary starting point: W3C WCAG 2.2
- **Children/vulnerable groups**; Required review and controls: Age scope and sensitive targeting restrictions; no child-directed product in initial scope; Primary starting point: FTC COPPA resources

Historical guidance is context; counsel verifies continuing status and governing law. Provider advertising policies, special categories, audience restrictions and required business verification are additional release requirements. They do not replace law. Assess any DSA/platform or other sector obligations from the product's actual service role before expansion; do not apply a blanket checklist without an applicability decision.

Data lifecycle and release evidence

Classify public marketing content, business analytics, identity/contact data, financial records, consent, audit and secrets separately. Default external AI input to approved aggregates; exclude raw CRM/free text and sensitive financial/insurance fields. Hashes and pseudonymous IDs remain protected where identifiable. Protect exports and logs.

Rights/deletion jobs reach transactional DB, analytics, cache, object exports and vendors where required. Keep deletion tombstones and reapply them after backup restore before replay. Enforce purpose on replay so withdrawn events are not resent. Test legal holds and permitted financial retention without reactivating marketing.

Before connected launch, retain an approved applicability/data-flow register, notices/terms and company identity/contact, vendor agreements/transfer decisions, consent/network traces, rights/retention tests, security review, breach exercise, AI classification/evaluations and sector sign-off. Name responsible owners and review cadence. Certifications and a statement of universal legal compliance require evidence beyond software design; public pages are updated to match the processing actually enabled.

Scalability, availability and recovery

Status: future acceptance contract. The published static website is not a benchmark of the connected platform. Keep measurements, estimates and approved targets distinct.

Signed workload before capacity purchase

The owner requires more than one million simultaneous dashboard users **and** more than one million simultaneous tracked visitors. Plan an overlapping 1.1M of each; measure interactive work and tracking work independently.

- **Dashboard**; Planning assumption: 1.1M active sessions, one API request per 10 seconds; Arithmetic rate: 110,000 API requests/second
- **Tracking**; Planning assumption: 1.1M visitors, one event per 30 seconds; Arithmetic rate: About 36,667 events/second
- **Combined before caching/batching**; Planning assumption: Both workloads at once; Arithmetic rate: About 146,667 operations/second
- **Tracking sustained 24 hours**; Planning assumption: Same event rate; Arithmetic rate: 3.168 billion events/day

At an assumed decimal 1 KB/event, raw tracking alone is about 3.168 TB/day before indexes, replicas, queue overhead or logs. Peak concurrency does not imply all-day traffic. Sign arrival distributions, peak/soak duration, regions, session activity, endpoint mix, report/export jobs, payload/event sizes, tenant skew, retention, data history, cache hit/miss and network conditions. Idle connections are a separate test.

Scaling boundaries

- **Static UI/assets**; Growth method: CDN and immutable hashed files; Guardrail: Keep HTML/artifact compatibility; verify headers
- **Interactive API**; Growth method: Stateless regional replicas, autoscaling and scoped caches; Guardrail: Per-tenant admission limits and no synchronous analytics scan
- **Event ingestion**; Growth method: Regional collectors and partitioned durable queues; Guardrail: Acknowledge only after durable acceptance
- **Provider sync/CAPI**; Growth method: Account-scoped workers, batching where supported; Guardrail: Provider budgets and consent; not one call per page view
- **Reports/analytics**; Growth method: Incremental materialization and analytical store; Guardrail: Async heavy jobs, query limits and freshness
- **Transaction/finance**; Growth method: Pooled PostgreSQL, indexed scoped queries, tenant cells; Guardrail: Single authoritative financial write path per tenant
- **AI**; Growth method: Async evidence-based jobs and cached authorized results; Guardrail: Token/cost/concurrency caps; no inference per visitor
- **Storage/export**; Growth method: Object lifecycle and expiring scoped downloads; Guardrail: Residency, deletion and access enforcement

Begin the connected pilot with multi-AZ compute, DB and durable queues. Introduce regional cells when benchmarks justify them. A tenant routes to its authorized cell; keep failure blast

radius bounded. Data residency and cross-region replication require privacy approval. Avoid synchronous global financial writes. Tenant skew and noisy neighbors require per-tenant limits and isolation, not only fleet-wide scaling.
Capacity planning records sustainable throughput per instance/cell, DB connections, queue lag, bytes/event, cost/tenant and failure headroom. An autoscaler cannot create capacity beyond cloud quotas, DB limits or provider permissions; review those first.

Service objectives

Proposed production targets become accepted only after measurable evidence:

- **Console/API availability**; Target: At least 99.9% per calendar month; Measurement: Successful eligible critical requests / eligible requests, plus external user-journey probes
- **Event acceptance availability**; Target: At least 99.9% monthly, separately; Measurement: Durable accepted eligible events / eligible attempts; explicit rejection/retry counted by agreed contract
- **Interactive latency**; Target: p95 ≤500ms, p99 ≤1.5s; Measurement: Agreed endpoints at signed load; cache hit/miss separately
- **Event acceptance latency**; Target: p95 ≤300ms regional; Measurement: Time to durable acknowledgement
- **Platform errors**; Target: <0.1% under agreed steady load; Measurement: Defined endpoint failures/time windows
- **Internal freshness**; Target: 99% processed within 5 minutes; Measurement: Received valid event → available read model
- **Recovery**; Target: Core-data RPO ≤5min, RTO ≤60min; Measurement: Approved regional disaster drill, per data class
- **In-region AZ failure**; Target: No loss of acknowledged durable events; Measurement: Queue/DB guarantee plus actual failure exercise
- **Routine deployments**; Target: No user-visible failed critical requests caused by release; Measurement: Canary, draining, compatibility and rollback evidence

99.9% corresponds to 43.2 minutes in a 30-day time-based month; request-based error budgets use eligible request counts instead. Do not interchange the two. Final contracts define valid attempts, maintenance, customer/network errors, regions and provider journeys. Keep user-visible end-to-end failures visible even when a provider is at fault. A cloud SLA is not measured application availability. Google SRE SLO guidance.
Keep last trustworthy data with timestamps during provider outages; stop unsafe actions when source freshness fails policy. Isolate AI/provider work from core event acceptance. Use error-budget burn to freeze risky releases and prioritize recovery.

Benchmark and resilience gates

Ramp through 1k, 10k, 100k and 1.1M active dashboard users with matching tracking load. Use distributed generators, representative data/tenant skew, cold/warm caches, auth, expensive endpoints and real payload sizes. Final proposal includes at least a two-hour peak and 24-hour

representative soak, plus bursts. Production-scale generators and vendor traffic require a separately verified budget and spend cap.
Use controlled provider adapters for load generation; separately perform bounded real provider contract tests. Never send the million-user load to external networks or CRM. Measure p50/p95/p99, error rate, saturation, DB/queue lag, backlog drain, autoscaling, cache misses, egress and observed unit cost. Exercise AZ loss, replica/worker termination, DB failover, provider/AI outage, queue duplication, region recovery and canary rollback.
Acceptance requires the signed workload, generator/config versions, artifact/infra identity, data size, complete results, safety margin, failures/remediation and independent review. Capacity claims stop at the largest successfully exercised envelope.

Backup, restore and deployment continuity

Encrypted backups/PITR belong to company accounts with least-privilege recovery access. Record retention and geographical/legal constraints. Test restore in isolation, reconcile finance totals, reapply privacy deletion tombstones and verify replay cannot send withdrawn marketing events. Measure actual RPO/RTO and publish scrubbed results.
Use expand/migrate/contract schemas, compatible old/new workers, graceful draining, canaries, readiness probes and rollback thresholds. Keep source/artifact/config identity and local/deployed digest parity. External side effects require compensating reconciliation; restoring a DB does not reverse ads already served or cash spent.

Cost and operating team

Pilot budget allowances are in the roadmap; they do not price this million-user workload. Price compute/HA headroom, DB/storage/I/O, queue/analytics, CDN/WAF/network transfer, observability, identity, backups, AI, CI and 24/7 staffing using measured units and current regional vendor calculators. Record baseline/peak/ stress scenarios and per-tenant/event/report cost. Alerts do not impose hard caps.
Assign SRE, incident commander, backup maintainer and an on-call rota before contractual uptime. Operations defines incident and release ownership. No new resources or paid load tests are activated by writing this contract.

Release, operations and developer handover

Current operation is static HTTPS frontend. Backend and data procedures below become required as those services are implemented. Company-owned accounts and reviewed local source remain authoritative; private recovery records hold actual access details.

Current website runbook

Use deployment tooling and the static runbook. Build locally, deploy only static output/generated configuration, retain the previous release, verify real browser/HTTP behavior and prove every

deployed file hash. Do not hand-edit live code. A documentation-only source push does not promote a website release.

Future environment and delivery contract

Separate development, staging and production identity, data, secrets, queues and quotas. Use reserved test records and approved provider test environments; never copy customer data into development without a documented lawful minimization process. Adopt declarative company-owned infrastructure, verified native inventories, short-lived CI grants and pinned/scanned artifacts. Remote Actions stay disabled until actual billing and execution are approved; source pushes do not enable workflows.

Each release records source commit, dependency lock, artifact/container digest, configuration/schema version, migration plan, acceptance/gates/review, operator, observability and rollback trigger. Snapshot deployed manifests/files before changing anything; reconcile live-ahead drift into local first. Promote staging, then a bounded production canary with approval. Verify served identity and local/artifact parity.

Use readiness, graceful connection/worker draining, backward-compatible migrations and old/new worker tests. Retain rollback artifacts and restore procedures. Database recovery is not a reversal of external financial/ad effects; reconcile/compensate them explicitly.

Observability and alerting

- **Critical API/event SLO and burn rate**; Owner/action: On-call SRE; incident/freeze release
- **DB saturation/replication, queue age/backlog, ingestion rejection**; Owner/action: Platform owner; isolate overload and drain safely
- **Provider freshness, rate limits, auth/revocation**; Owner/action: Integration owner; reconnect or degrade read-only
- **Ledger/source difference and attribution coverage**; Owner/action: Finance/data owner; quarantine/reconcile exceptions
- **Approval execution, spend reservation and policy denial**; Owner/action: Marketing/security owner; stop unsafe changes
- **AI schema/evidence/cost failure**; Owner/action: AI owner; deterministic fallback and suspend affected workflow
- **Consent/rights/deletion failures**; Owner/action: Privacy owner; suppress affected processing and assess obligations
- **Backup/restore and certificate expiry**; Owner/action: SRE; repair before recovery/security exposure

Use structured metrics/traces/logs with correlation/tenant-safe IDs. Redact tokens, contact fields and full provider bodies. Keep cardinality/retention/cost bounded. Public status communications expose service impact, not private infrastructure.

Incident procedure

1. Assign incident commander, technical lead and communications/privacy contacts.
2. Contain dangerous execution, compromised access or affected processing; preserve safe evidence and audit continuity. Use scoped/global kill switches where appropriate.
3. Determine affected tenants, time, money/data and service impact. Keep a factual timeline.
4. Restore a verified artifact or fail over under the approved runbook. Reconcile queues, payments and uncertain provider mutations before re-enabling work.
5. Privacy/legal owners decide notification duties and deadlines. Use approved channels; do not send unsolicited customer communications as a tooling side effect.
6. Verify recovery against SLO/freshness/data/finance checks; remove temporary privileges.
7. Record cause, contributing conditions, corrective tests and owners/deadlines. Update private incident/defect logs and publish only an approved scrubbed summary.

Recovery and data lifecycle

Schedule encrypted backups/PITR and isolated restores. Measure RPO/RTO per data class, verify finance totals and file/artifact hashes, reapply deletion tombstones and consent before replay, and validate current schema/permissions. Test retention/legal holds. A successful backup job without a restore test is insufficient acceptance evidence.
Provider outages use durable queues, bounded retries and stale timestamps. Preserve last trustworthy metrics; fail closed on advertising execution when authority/economics are unknown. Replaying a dead-letter message requires cause review, current consent, idempotency and bounded scope. Secrets rotate through the approved vault and private recovery register, with revoked old grants and tested dependents.

User workflow handover

Marketing reviews full-funnel evidence and creative/test proposals, then explicitly approves a bounded action. COO reviews follow-up/appointments/CRM bottlenecks before routing changes. CEO reviews spend→sales→revenue→profit and major allocation decisions. All users can inspect freshness/model/unknown coverage rather than receiving unexplained scores. Permissions and a clear escalation path apply to every workflow.

Acceptance and maintenance checklist

- Company admins control source, domains, hosting, provider assets and recovery.
- Incoming developers can clone, install, run gates and find current/remaining scope.
- API/schema/configuration/event/attribution/AI and legal/security decisions are versioned.
- A named primary/backup maintainer and on-call rota cover the accepted service objective.

- Release, rollback, provider failure, credential rotation, rights and restore drills have evidence.
- Capacity reports include workload, tested envelope, headroom and observed cost.
- Open risks, access reviews, patch cadence, support/security contacts and renewals have owners.
- Source/remote equality, artifact parity and documentation links are verified at handover.

The public technical overview can be shared separately from the private repository. Existing Google Docs retain content/ID; explicitly authorized updates use revision-aware edits and a validated temporary PDF before canonical replacement. Reference projects are not modified. See the documentation index for current guides/evidence.

Development and contribution

Start from source

Clone the owner-controlled repository. Use Node 24 LTS, `npm ci` and Python 3.11+ for deployment tests. Read the overview and documentation index. No backend/database starts in this release. Use the URL printed by `npm run dev`.

Create a focused branch. Record acceptance criteria and configuration/data owners before implementation. Later features ship UI, API, data, security, tests and docs together. Verify new packages on their real registry and review licenses/advisories.

Gates

```
npm test
npm run typecheck
npm run lint
npm run format:check
python3 -m unittest discover -s tests -p 'test_*.py'
npm run build
```

Install pre-commit, Gitleaks, Bandit and Semgrep from official distributions, then:

```
pre-commit install
pre-commit run --all-files
gitleaks git --redact --no-banner
```

Hooks use system-installed staged-diff Gitleaks, Bandit for Python and Semgrep security-audit rules. Preserve edit scanners; fix findings without suppression/bypass. Report missing gates. Tracked automated tests live in `tests/`; personal/manual tools stay ignored and are improved in place rather than deleted.

Exercise affected real UI/HTTP flows, keyboard/focus and responsive states. Backend features require negative authorization, retry and duplicate tests. Obtain fresh-context review, then report criteria/gates/behavior/verdict. Document escaped defects privately.

Configuration and secrets

Use `src/config.ts`, `src/data/` and `deploy/settings.json` for current values. Future backend variables enter one validated settings module and `.env.example`. Keep actual credentials, private keys and customer records out of source, fixtures and public docs. Verify exclusions before recording access details. Review history as well as staging. An exposed credential requires revocation/rotation; deleting text alone does not revoke it. GitHub sensitive-data guidance.

Safe Git delivery

Verify authenticated account and approved remote. Use the owner's approved name/email; commit attribution and a verified-signature badge are distinct. Preserve visibility, branch protection and CI settings. Review the diff and stage explicit public paths. Run hooks, fetch/reconcile remote changes, push normally, verify branch SHA equality and record remaining local work. Never force-push routinely or enable Actions as a side effect. Current Actions remain disabled pending billing/execution approval.

Documentation and release

Update affected guides and phase status when exit gates pass. Preserve scoped historical evidence. Public docs exclude private infrastructure/access details. Existing Google Docs are edited in place when requested; validated PDF replacement follows successful updates. Production changes require drift snapshot, local edits, local gates, reviewed artifact deployment, real smoke flows and hash/digest parity. Source delivery and server promotion are separate. See operations.

Deployment tooling

This directory implements the static website configuration renderer. Future backend and cloud services are specified in the roadmap; they are not provisioned here.

Render locally

From the repository root with Python 3.11+:

```
python3 deploy/render.py deploy/settings.json artifacts/hosting
python3 -m unittest discover -s tests -p 'test_*.py'
```

The ignored output contains Compose, Nginx and Caddy configuration. Rendering starts no container, changes no DNS and makes no paid request. Unsafe hostnames and unpinned images are rejected. `settings.json` owns hostname, ports, image/release identity, resources, health, cache, headers and timeouts. Stable protocol syntax belongs to the renderer. Secrets never belong in the settings file.

Follow the static runbook: detect drift, build/test locally, package static output, validate/upload/promote, exercise HTTPS/browser flows and prove file parity. Retain the previous verified release and referenced hashed assets. Never use the preview server for production hosting.
Private recovery records hold actual server access/locations. Future API, queue and database operations follow operations and capacity/reliability.